Building Cloud Applications

Use **VCF Automation** to provision VMs, Kubernetes workloads and other Cloud Services by using self-service UI, API, and CLI.

Table 1664: Provisioning Workloads in VCF and vSphere Foundation

Workload Provisioning Method	UI or Utility	Documentation
UI-based workload provisioning	VCF Automation catalog in the namespace allocated to you by your organization administrator.	Getting Started with the Self-Service Catalog in VCF Automation
	Services console in VCF Automation in VCF or Local Consumption Interface invSphere Foundation.	Access the IaaS Services Console or Local Consumption Interface
Automated workload provisioning	`kubectl` command in the namespace allocated to you in VCF Automation by your organization administrator, accessing the namespace and Kubernetes resources in it by using the VCF Consumption CLI (also called VCF CLI).	Installing and Using VCF CLI v9.0

Available Workload Types

You can provision the following workload types by using the out-of-the-box VCF deployment:

- Virtual machines
- Provision and Manage Kubernetes Clusters
- vSphere Pods
- Persistent volumes
- Secret store
- Harbor container registry
- GPU-enabled private AI workloads or Kubernetes clusters

Getting Started with the Tools for Building Applications

Use VCF Automation to provision VMs, Kubernetes workloads and other cloud services by using self-service UI, API, and CLI. The Local Consumption Interface provides an alternative for access to vSphere Supervisor services for users who do not have access to VCF Automation.

Building Applications with VCF Automation

VCF Automation supports two distinct organization types, All Apps organizations and VM Apps organizations, which have different consumption mechanisms. What you can do in VCF Automation depends on your organization membership.

VCF Automation for All Apps provides a user-friendly interface for consuming infrastructure resources as a service, including Kubernetes and other infrastructure components. If you are a member of an All Apps organization in VCF Automation, you have two ways to provision infrastructure:

- **Self-service catalog**. The catalog in VCF Automation for All Apps is the self-service interface where you can provision workloads from blueprints and other objects that the organization administrator curated and published for you. Blueprints contain resources that can be nested, for example, a blueprint can contain a VM inside a vSphere namespace. See Request an Item from the Catalog in for All Apps.
- **IaaS Services**. VCF Automation gives you access to modern services enabled by vSphere Supervisor and cloud native applications. You can provision resources including VMs, Kubernetes, volumes, storage, load balancers and networking objects, without involvement from your administrator. See Access the IaaS Services Console or Local Consumption Interface.

Both catalog and services deployments support day 2 operations on workloads and services.

In **VCF Automation for VM Apps** organizations you deploy catalog items from the VCF Automation for VM Apps self-service catalog. The catalog in VCF Automation for VM Apps is a self-service portal for users to request and manage IT resources from various cloud providers, both private clouds (VMware vSphere) and public clouds (GCP, AWS, Azure), independently through the catalog interface. See Request an Item from the Catalog in for VM Apps.

Catalog items in VCF Automation for VM Apps are pre-configured templates that users can deploy, such as virtual machines, cloud templates, and other infrastructure components. The catalog items are provided to you by your administrator. The items that are available to you and your options at deployment time depend on your project membership.

Important:

VCF Automation for VM Apps does not support IaaS services.

Building Applications with the Local Consumption Interface

The Local Consumption Interface provides an alternative interface to allow access to modern services enabled by enabled by vSphere Supervisor and cloud native applications. Users can provision resources including VMs, Kubernetes, volumes, storage, load balancers and networking objects, without administrator involvement.

The Local Consumption Interface is a vSphere Supervisor service. It can be made available in environments without access to VCF Automation, such as vSphere Foundation. For more information on installing and configuring the Local Consumption Interface, see Using the Local Consumption Interface.

Getting Started with the Self-Service Catalog in VCF Automation

As a consumer, you use VCF Automation to deploy IT resources using a self-service catalog and then manage your workloads and run day-2 actions like updates and patches independently.

What is the VCF Automation Catalog

The VCF Automation catalog is the streamlined user interface that you use to request and deploy infrastructure on a self-service basis. Catalog items include VCF Automation blueprints, VM images, storage, networks, and so on, that you can request for deployment.

The catalog items are provided to you by your organization administrator. The items that are available to you depend on your project membership. At request time, the information that you must provide depends on how the item was configured by your administrator. When you deploy an item, it is provisioned based on the infrastructure available to the selected project. Projects also determine your options at deployment time.

How does the catalog work

The catalog is the simplified user interface that organization administrators make available to users when the administrator's teams do not need full access to developing and building the blueprints. You use the catalog to deploy blueprints to namespaces associated with projects in the organization.

To provide the blueprints, the organization administrator creates blueprints and configures content libraries, which are collections of VM images, and then publishes these items to the catalog.

- Blueprints are associated with projects. Projects link a set of users with one or more target cloud regions.

- For example, UserA is a member of ProjectA and ProjectB, but not ProjectC. UserA sees only the blueprints that were entitled to ProjectA and ProjectB.

When users request a catalog item, where it is deployed depends on the project selected. Projects might have one or more namespaces.

- If UserA and UserB are members of ProjectA, they see the blueprints as catalog items. At deployment time they can deploy to ProjectA, which determines which namespace the catalog item is deployed to.

The availability of the catalog items is determined by project membership. Projects link users, catalog items, and IaaS resources where the items are deployed. The Build & Deploy tab provides a single point for VCF Automation consumers to access their catalog, deployed instances, and available resources.

After a successful request, your users can then manage their deployments by running actions, including dismiss or delete.

Who works with the catalog

- Provider or organization administrators can create blueprints across all vCenter instances and publish them to the catalog.
- Organization administrators manage the catalog within the organization. They can share blueprints with projects within their organization.
- Application team members request and consume the content in the catalog.

Which catalog do I have

Depending on the VCF Automation environment you have access to, your view of the Catalog might look differently. For example, if you are a member of a VM Apps organization in VCF Automation, you access your catalog and deployments on the **Consume** tab, where in All Apps organizations, you access the catalog and deployed catalog instances on the **Build & Deploy** tab.

Request an Item from the Catalog in VCF Automation for All Apps

The catalog in VCF Automation for All Apps contains blueprints, VM images, and other infrastructure objects you can request for deployment. At request time, the information that you must provide or configure depends on how the blueprint was designed by the organization administrator. When you deploy an item, it is provisioned based on the namespaces that are associated with your project.

Using the filter and search to locate a catalog item

Depending on your company goals and project members, the catalog available to you can be extensive. You can use the following tools to locate a catalog item.

1. Search. Enter a search term.
2. Filter. Opens the left panel where you can filter by content type and projects.
3. Sort. If the list is still too long, you can sort in ascending or descending order.

Procedure

The information provided in this article is general because each catalog item is unique. The variation depends on how the blueprint and other items were constructed, including what variables are made available to you at request time.

1. Log in to VCF Automation.
2. Click the **Build and Deploy** tab.

3. Click **Catalog**.
   The available catalog items are available to you based on the project your selected. If you didn't select a project, all catalog items that are available to you appear in the Catalog.
4. Locate the catalog item you plan to deploy.
   You can use the filter, search, or sorting options to find the catalog item.
5. Click **Request**.
6. Provide any required information.
   a. If the catalog item has more than one released version, select the version that you want to deploy.
   b. At minimum, you must provide a deployment name and select a project and a namespace for your deployed catalog instance.
      The project list includes projects that you are a member of. The namespace list includes namespaces associated with the project you selected.
   c. The request form might have other options that you must configure, depending on how the catalog item was designed.
7. Click **Submit**.
   The provisioning process begins and the Catalog tab on the Instances page opens with your current request at the top.

Managing Deployed Instances in VCF Automation for All Apps

You use the **Instances** page in VCF Automation for All Apps to view and manage all resources that are managed within the organization. Resources include virtual machines, Kubernetes clusters, and other custom resources. You can manage, search, filter, and discover these resources across projects and namespaces.

Working with instances

To view your resources, select **Build and Deploy** > **Instances**.

The Instances page supports the following resources:

- The Catalog tab
- Virtual Machines
- Kubernetes Clusters
- Volumes
- Custom Resources

Using the filter and search to locate a resource

Depending on your company goals and project membership, the list of resources available to you can be extensive. You can use the following tools to locate a resource.

- Search. Enter a search term.
- Sort. If the list is still too long, you can click on a column heading to sort in ascending or descending order.
-

Managing Deployed Catalog Instances in VCF Automation for All Apps
The Catalog list on the Instances page includes catalog instances, where each instance can contain multiple resources of different types created from blueprints.

What are catalog instances

You use the Instances > Catalog page to manage your deployed catalog items and the associated resources, making changes to deployments, troubleshooting failed deployments, making changes to the resources, and destroying unused deployments.

The deployments are the provisioned instances of catalog items, blueprints, and resources. If you manage a small number of deployments, the deployment cards provide a graphical view for managing them. If you manage a large number of deployments, the deployment list and the resource list provide more a more robust management view.

To manage your catalog deployments, select **Build & Deploy** > **Instances** > **Catalog**.

Working with deployment cards and the deployment list

You can locate and manage your deployments using the card list. You can filter or search for specific deployments, and then run actions on those deployments.

1. Filter your requests based on attributes.
   For example, you can filter based on owner, projects, lease expiration date, or other filtering options. Or you might want to find all the deployments for two projects with a particular tag. When you construct the filter for the projects and tag example, the results conform to the following criteria: (Project1 OR Project2) AND Tag1.
   The values that you see in the filter pane depend on the current deployments that you have permission to view or manage.Most of the filters and how to use them are relatively obvious. Additional information about some of these filters is provided below.
2. Search for deployments based on keywords or requester.
3. Sort the list to order by time or name.
4. Switch between the deployment card and the deployment grid views.
5. Run deployment-level actions on the deployment, including deleting unused deployments to reclaim resources.

Monitoring Deployed Catalog Instances in VCF Automation for All Apps

You monitor VCF Automation deployment requests to ensure that the resources are provisioned, that the provisioned resources are running, and to resize or destroy the resources as needed.

Monitoring your request status

The deployment cards that appear on the Instances page show the state of the deployment, including in-progress (top) and completed (below). The card includes the number of deployed resources, how long it has been deployed, and the lease expiration date.

The cards also provide the IP addresses and the actions that you can run on the deployment.

If an approval policy is triggered for your request, you might see the request in an in progress state with the name of at least one approver. Approval policies are defined by your organization administrator. The approvers are defined in the policy. The approvers approve requests using an Approvals tab. You might also encounter approvals on day 2 actions.

If a deployment fails, the cards show the error message for the point of failure and the process progress. To learn more about the failure, click the deployment name at review the **History** tab.

Where are resources deployed

To access your successfully provisioned deployments, you might need more than the IP address provided on the card. Click the deployment name and review the deployment details on the **Topology** tab.

You most likely need the IP address for the primary component. As you click on each component, notice the information that is provided is specific to that component.

The availability of the external link depends on the cloud provider. Where it is available, you must have the credential on that provider to access the component.

Tracking deleted deployments

After you delete a deployment, you might want to see a list or review the history of a particular deployment.

To view your deleted deployments, click the filter on the **Instances** page, and then turn on **Deleted Deployments Only** toggle. The list of deployments is now limited to those that are deleted.

If you need the name of delete machines, you can look at the history to retrieve the information.

The deleted deployments are available for 90 days.

Troubleshooting failed deployments

Your deployment request might fail for many reasons. It might be due to network traffic, a lack of resources on the target namespace, or a flawed deployment specification. Or, the deployment succeeded, but it does not appear to be working. You can examine your deployment, review any error messages, and determine whether the problem is the environment, the requested workload specification, or something else.

You use this workflow to begin your investigation. The process might reveal that the failure was due to a transient environmental problem. Redeploying the request after verifying the conditions have improved resolves this type of problem. In other cases, your investigation might require you to examine other areas in detail.

1. To determine if a request failed, select **Instances** > **Catalog** and locate the deployment card.
   Failed deployments are indicated on the card.
   1. Review the error message.
   2. For more information, click the deployment name for the deployment details.
2. On the deployment details page, click the **History** tab.
   1. Review the event tree to see where the provisioning process failed. This tree is useful when you modify a deployment, but the change fails.
   2. The **Details** page provides a more verbose version of the error message.

If you are unable to resolve your problem, contact your organization administrator for additional assistance.

Note:

When vSphere Supervisor converts storage policies that you assign to namespaces into Kubernetes storage classes, it changes all upper case letters into lower case and replaces spaces with dashes. To avoid confusion, use lower case and no spaces in the VM storage policy names.

Managing Individual Resource Instances in VCF Automation for All Apps

Individual resource lists, such as Virtual Machines or Custom Resources, include all resources of the same type created in VCF Automation.

What are resource instances

You use the individual resources lists on the **Instances** page to view your inventory of resource types within an organization. Resources of each type are listed on the Virtual Machines, Kubernetes Clusters, and Volumes tabs on the Instances page.

Listed resources include resources that were provisioned from the VCF Automation catalog, using the IaaS Services console, or using kubectl.

The list of resources visible in the inventory views depends on the rights and project roles assigned to the user. Remember that resources belong to namespace, and namespace belongs to a project.

Working with resource lists

You can use the resource lists to manage the following resource types: virtual machines, Kubernetes clusters, volumes, and custom resources that make up your deployments. In the resource list you can manage them in resource type groups rather than by deployments.

- Virtual Machines
  Individual virtual machines. The machines might be part of larger deployments.
  To control the amount of resources that VCF Automation users might deploy, it is likely that your administrator added approval policies to reject or approve any deployment requests based on the image used or the resources requested.
- Kubernetes Clusters
  Kubernetes clusters that were associated with deployments.
- Volumes
  Storage volumes that were associated with deployments.
- Custom Resources
  Includes custom resources and resource actions based on VCF Operations orchestrator workflows.

Similar to the deployment list view, you can filter the list, select a resource type, search , sort, and run actions.

If you click the resource name, you can work with the resource in the context of the resource details.

- Filter your list based on resource attributes.
  For example, you can filter based on project or other attributes.
- Search for resources based on name or other values.
- Run available day 2 actions that are specific to the resources type and the resource state.
  For example, you might power on a machine if it is off. Or you might resize a machine.

Working with the resource details view

You can use the resource details view to get a deeper look at the selected resource. Depending on the resource, the details can include networks, ports, and other information collected about the machine. The depth of the information varies depending on cloud account type and origin.

To open the details pane, click the resource name or the double arrows.

Data collection for supervisor resources

The catalog uses Kubernetes watch mechanism to collect inventory updates from namespaces in near real time and update the cache.

Request an Item from the Catalog in VCF Automation for VM Apps

As a VCF Automation consumer, you deploy a catalog item that was created in VCF Automation and other sources so that you can deploy it as part of your work processes.

Using the filter and search to locate a catalog item

Depending on your company goals, the catalog available to you can be extensive. In addition to filtering available items based on project in the Projects drop-down menu, you can use the following tools to locate a catalog item.

1. Search. Enter a search term.
2. Filter. Opens the left panel where you can filter by content type and projects.
3. Sort. If the list is still too long, you can sort in ascending or descending order.

Procedure

1. Log in to VCF Automation.
2. Click the **Consume** tab.
3. In the **Projects** drop-down menu, select the project for which you want to view the available catalog items.
   If multiple projects appear in the drop-down menu, you can select more than one project. You can see a list of your projects on the Overview page.
   If you have access to a single project, you can skip this step. All catalog items for that project appear in the catalog.
4. Click **Catalog**.
   The available catalog items are available to you based on the project your selected. If you didn't select a project, all catalog items that are available to you appear in the Catalog.
5. Locate the catalog item you plan to deploy.
   You can use the filter, search, or sorting options to find the catalog item.
6. Click **Request**.
7. Provide any required information.
   a. If the catalog item has more than one released version, select the version that you want to deploy.
   b. At minimum, you must provide a deployment name and select a project for your deployed catalog instance.
      The project list includes projects that you are a member of.
   c. The request form might have other options that you must configure, depending on how the catalog item was designed.
8. Click **Submit**.
   The provisioning process begins and the Catalog tab on the Instances page opens with your current request at the top.

Manage your deployments

As a VCF Automation for VM Apps consumer, you use the Deployments page to manage your deployments and the associated resources, making changes to deployments, troubleshooting failed deployments, making changes to the resources, and destroying unused deployments.

The deployments are the provisioned instances of catalog items, cloud templates, and onboarded resources. If you manage a small number of deployments, the deployment cards provide a graphical view for managing them. If you manage a large number of deployments, the deployment list and the resource list provide more a more robust management view.

To manage your deployments, select **Consume** > **Deployments** > **Deployments**.

You can use the **Projects** drop-down on the Consume tab to view only the deployments for a specific project.

Managing Deployments and Resources in VCF Automation for VM Apps

As a catalog consumer with the necessary permissions, you use the Resources tab to manage your resources. The resources can be deployed catalog items, but they can also be those that were discovered for you project cloud accounts.

How do I manage my deployments in VCF Automation for VM Apps
VCF Automation for VM Apps consumers use the Deployments page to manage deployments and the associated resources, making changes to deployments, troubleshooting failed deployments, making changes to the resources, and destroying unused deployments.

The deployments are the provisioned instances of catalog items, cloud templates, and onboarded resources. If you manage a small number of deployments, the deployment cards provide a graphical view for managing them. If you manage a large number of deployments, the deployment list and the resource list provide more a more robust management view.

To manage your deployments, select **Consume** > **Deployments** > **Deployments**.

You can use the **Projects** drop-down on the Consume tab to view only the deployments for a specific project.

Working with deployment cards and the deployment list

You can locate and manage your deployments using the card list. You can filter or search for specific deployments, and then run actions on those deployments.

Figure 246: Deployments page card view

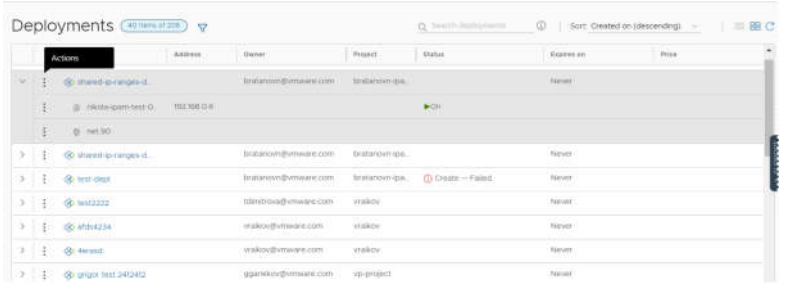
1. Filter your requests based on attributes.
   For example, you can filter based on owner, projects, lease expiration date, or other filtering options. Or you might want to find all the deployments for two projects with a particular tag. When you construct the filter for the projects and tag example, the results conform to the following criteria: (Project1 OR Project2) AND Tag1.
   The values that you see in the filter pane depend on the current deployments that you have permission to view or manage.
   Most of the filters and how to use them are relatively obvious. Additional information about some of these filters is provided below.
2. Search for deployments based on keywords or requester.
3. Sort the list to order by time or name.
4. Switch between the deployment card and the deployment grid views.
5. Run deployment-level actions on the deployment, including deleting unused deployments to reclaim resources.

You can also see deployment costs, expiration dates, scheduled deletion dates, and status.

To adjust what information you see for your deployments, click Manage Columns in the bottom left of the deployment grid and select your preferred columns.

You can switch between the card and grid view in the upper right of the page, to the right of the Sort text box. You can use the grid view to manage a large number of deployments on fewer pages.

Figure 247: Deployment page grid view



Working with selected deployment filters

The following table is a not a definitive list of filter options. Most of them are self-evident. However, some of the filters require a little extra knowledge.

Table 1665: Selected filter information

Filter name	Description
Optimizable Resources Only	If you integrated VCF Operations and are using the integration to identify reclaimable resources, you can toggle on the filter to limit the list of qualifying deployments.
Deployment Lifecycle Status	The Deployment Lifecycle Status and Last Request Status filters can be used individually or in combination, particularly if you manage a large number of deployments. Examples are included at the end of the Last Request Status section below. Deployment Lifecycle Status filters on the current state of the deployment based on the management operations. This filter is not available for deleted deployments. The values that you see in the filter pane depend on the current state of the listed deployments. You might not see all possible values. The following list includes all the possible values. Day 2 actions are included in the Update status. • Create - Successful • Create - In Progress • Create - Failed • Update - Successful • Update - In Progress • Update - Failed • Delete - In Progress • Delete - Failed
Last Request Status filters	Last Request Status filters on the last operation or action that ran on the deployment. This filter is not available for deleted deployments. The values that you see in the filter pane depend on the last operations that ran on the listed deployments. You might not see all possible values. The following list is all of the possible values.

Filter name	Description
	• Pending. The first stage of a request where the action is submitted but the deployment process has not yet started. • Failed. The request experienced a failure during any stage of the deployment process. • Cancelled. The request was cancelled by a user while the deployment process was processing and not yet completed. • Successful. The request successfully created, updated, or deleted a deployment. • In Progress. The deployment process is currently running. Additional deployment states, for example, Initialization and Completion that you see in the deployment History tab are not provided as filters, but you can use the In Progress filter to locate deployments in those states. • Approval Pending. The request triggered one or more approval policies. The process is waiting for a response to the approval request. • Approval Rejected. The request was denied by the approvers in the triggered approval policies. The request does not continue. The following examples illustrate how the how to use the Deployment Lifecycle Status and Last Request Status filters individually or together. • To find all delete requests that failed, select **Delete - Failed** in the Deployment Lifecycle Status filter. • To find all the requests waiting for approval, select **Approval Pending** in the Last Request Status filter. • To find the delete requests where the approval request is still pending, select **Delete - In Progress** in the Deployment Lifecycle Status filter and **Approval Pending** in the Last Request Status filter.

Working with deployment details

You use the deployment details to understand how the resources are deployed and what changes have been made. You can also see pricing information, the current health of the deployment, if the deployment expires and when it's scheduled for deletion, and if you have any resources that need to be modified.

- **Topology** tab. You can use the Topology tab to understand the deployment structure and resources.

- **History** tab. The History tab includes all the provisioning events and any events related to actions that you run after deploying the requested item. If there are any problems with the provisioning process, the History tab events helps you with troubleshoot the failures.
- **User Events** tab. You can use the User Events tab to provide and track the user interactions for your deployment. The events include any initial input values, any required approvals, input values for day 2 changes, and any values that you must provide as part of a deployment or for a day 2 action workflow. Where the request requires input values, you can also enter the values on the Inputs tab in VCF Automation.

  For deployments that include VCF Operations orchestrator workflow user events, where you enter values during the deployment process, there are some situations where the tab does not display the form or where the workflow is canceled. If you have multiple VCF Operations orchestrator instances that do not have tags or where they all have the same tag, the form does not load. Ensure that you correctly tag the instances so that the form displays on the tab. If there are not assignees in the workflow form, the workflow is canceled and the deployment or action fails. Ensure that you workflow form includes assignees.
- **Price** tab. You can use the pricing tab for insights about how much your deployment is costing your organization. Pricing information is provided by your VCF Operations or CloudHealth integrations.
- **Monitor** tab. The Monitor tab data provides information about the health of your deployment based on data from VCF Operations.
- **Alerts** tab. The Alerts tab provides active alerts on the deployment resources. You can dismiss the alert or add reference notes. The alerts are based on data from VCF Operations.
- **Optimize** tab. The Optimize tab provides utilization information about your deployment and offers suggestions for reclaiming or otherwise modifying the resources to optimize resource consumption. The optimization information is based on data from VCF Operations.

How do I monitor deployments in VCF Automation for VM Apps

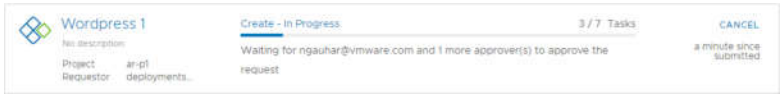
You monitor deployment requests in VCF Automation for VM Apps to ensure that the resources are provisioned, that the provisioned resources are running, and to resize or destroy the resources as needed.

The Deployments page provides information about the current state of the deployment and where the resources are deployed in your provider clouds.

How do I know that my deployment request succeeded

The deployment cards that appear on the Deployments page show the state of the deployment, including in-progress (top) and completed (below). The card includes the number of deployed resources, how long it has been deployed, and the lease expiration date.

If an approval policy is triggered for your request, you might see the request in an in progress state with the name of at least one approver. Approval policies are defined in VCF Automation for VM Apps by your administrator. The approvers are defined in the policy. The approvers approve requests using an Approvals tab. You might also encounter approvals on day 2 actions.

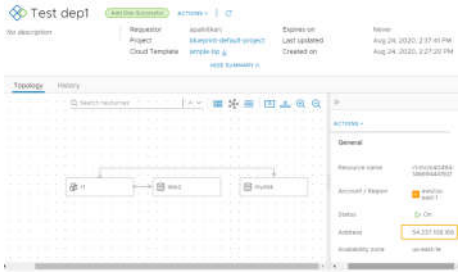


If a deployment fails, the cards show the error message for the point of failure and the process progress. To learn more about the failure, click the deployment name at review the History tab.

For more information about troubleshooting failed deployments, see What can I do if a VCF Automation for VM Apps deployment fails.

Where are my resources deployed

To access your successfully provisioned deployments, you might need more than the IP address provided on the card. Click the deployment name and review the deployment details on the Topology tab.



You most likely need the IP address for the primary component. As you click on each component, notice the information that is provided is specific to that component.

The availability of the external link depends on the cloud provider. Where it is available, you must have the credential on that provider to access the component.

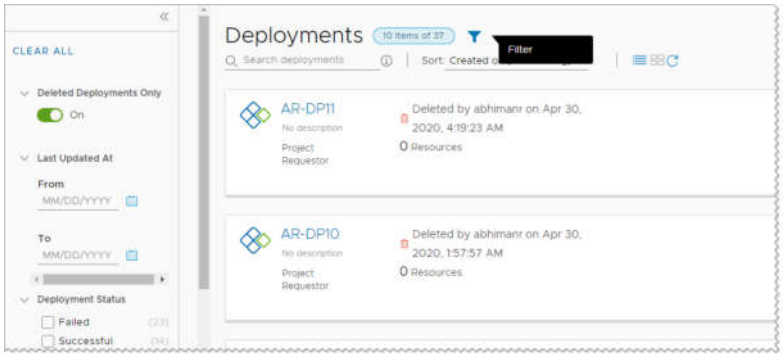
How do I track deleted deployments

After you delete a deployment, you might want to see a list or review the history of a particular deployment.

To view your deleted deployments, click the filter on the **Deployments** page, and then turn on **Deleted Deployments Only** toggle. The list of deployments is now limited to those that are deleted.

If you need the name of delete machines, you can look at the history to retrieve the information.

The deleted deployments are available for 90 days.



What can I do if a VCF Automation for VM Apps deployment fails

Your deployment request might fail for many reasons. It might be due to network traffic, a lack of resources on the target cloud provider, or a flawed deployment specification. Or, the deployment succeeded, but it does not appear to be working. You can examine your deployment, review any error messages, and determine whether the problem is the environment, the requested workload specification, or something else.

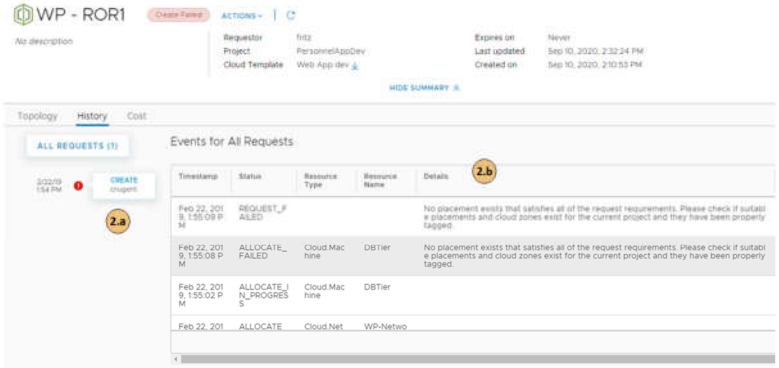
You use this workflow to begin your investigation. The process might reveal that the failure was due to a transient environmental problem. Redeploying the request after verifying the conditions have improved resolves this type of problem. In other cases, your investigation might require you to examine other areas in detail.

1. To determine if a request failed, select **Deployments** > **Deployments** and locate the deployment card.

   Failed deployments are indicated on the card.

   a) Review the error message.
   b) For more information, click the deployment name for the deployment details.

2. On the deployment details page, click the **History** tab.



   a) Review the event tree to see where the provisioning process failed. This tree is useful when you modify a deployment, but the change fails.
   b) The **Details** provides a more verbose version of the error message.

If you are unable to resolve your problem, contact your cloud administrator for additional assistance.

How do I track and respond to requests that require approval in VCF Automation for VM Apps

As a VCF Automation for VM Apps consumer, you received an email notification about a deployment or day 2 action request that you made. Your request must be approved by a designated approver before it proceeds. If you are an assigned approver, you received an email notification about a deployment request that someone made. In either case, you can use these procedures to understand the approval policy workflow related to deployment requests and how to respond to approval requests if are an assigned approver.

How do I track my requests that require approval in VCF Automation for VM Apps

This procedure assumes that you received an email notification about your deployment request is pending approval, or that you noticed that your deployment did not progress.

You receive an email with the name of your deployment and the name of the first approver on the list. The message includes a link to the deployment details where you can track the approvals in the deployment details.

If you received an email about the pending request, you can see the name of your deployment and the name of the first approver on the list. The message includes a link to the deployment details where you can track the approvals in the deployment details.

1. Select **Consume** > **Deployments** > **Deployments**.
2. You requested a deployment or a day 2 action on an existing deployment, but you now see message on your deployment card.
   For example, your card displays `Create - Approval Pending` and lists the names of the approvers.

   Your request triggered one or more approval policies.
3. For information that helps you track the progress of your request, click the deployment name, and then click the **Details** tab.

   When the deployment is first awaiting approval, you only see APPROVAL_IN_PROGRESS. After a few minutes the list of approver names are added in the Details column. If the request requires multiple approvers, the approver list updates as an approver responds. With each update, only the pending approver names remain.
4. When your request is approved or rejected, you receive another email message appropriate to the outcome.

   If the request is rejected, the deployment details **History** tab displays REQUEST_FAILED and the details column provides the name of the approver and the reason for rejecting the request.

How do I respond to an approval request in VCF Automation for VM Apps

As a designated approver for deployment or day 2 action requests made in VCF Automation for VM Apps, you are tasked with approving requests. If you are an assigned approver in the policy, you received an email notification about a deployment request that someone made. If you are ? user with the Manage Approvals custom role who monitors and responds to approval requests, you do not receive a notification.

Some policies might require only your approval, while others require multiple people to approve a request.

If the policy that you are responding to has multiple approvers but only requires one approver, you might see an already approved request in the Approvals page. You do not need to take further action.

If you are managing many requests, you can limit the number of approval requests displayed by using the filter option. For example, rather than all the requests, you can use the Pending for me filter to see only pending approval requests from all levels where you are an assigned approver.

1. If you are an assigned approver, you receive an email that provides the name of the requesting user, the catalog item, and a link to the request in the **Approvals** page in VCF Automation for VM Apps.

   If you are someone who manages approvals, you can select **Inbox** > **Approvals** and continue with the following steps.
2. Locate the approval card for the notification.
3. Review the deployment details and the approval details, and approve or reject the request.

   Required approvals are grouped into sequential levels. For example, if you are an assigned approver for deployment requests where level 1 and level 4 policies apply, then the approval details page shows the required approvals from the level 1 policies grouped under the Level 1 category, and the level 4 policies grouped under the Level 2 category.

   If you are a user with administrative rights, you have the following options.

   - Approve full request. You approve the request at all levels where you are an assigned approver. No other approvals are required so the request is approved.
   - Approve current level. You approve the request at all levels where you are an assigned approver. The request is routed to the next level that is pending approval.
   - Approve as an assigned approver. You approve the request at all levels where you are an assigned approver. If there are other approvers who have not responded, the request remains pending and is not routed to the next level until others approve the request.

   If you reject the request, you must provide a reason that is included in the email message sent to the requester.
4. The system sends an email to the requester indicating that the request was approved or rejected.

How do I track and respond to requests that require user input in VCF Automation for VM Apps

As a VCF Automation for VM Apps consumer, you received an email notification about a deployment request for a VCF Operations orchestrator workflow that requires user input before it proceeds. The deployment might request you or another user to provide input. You can use these procedures to understand how your request is processed and how to respond to user input requests.

How do I track my requests that require user input in VCF Automation for VM Apps

You receive an email with the name of your deployment and the name of the first assignee on the list. The message includes a link to the deployment details where you can track responses to your request on the **User Events** tab.

- A VCF Operations orchestrator workflow that contains a manual user interaction element is added to the catalog.

This procedure assumes that you received an email notification about the user input request, or that you noticed that your deployment did not progress.

1. Select **Consume** > **Deployments** > **Deployments**.
2. Locate your deployment request for the VCF Operations orchestrator workflow.

   You notice a message on your deployment card. For example, your card displays `User Interaction Pending.`
3. To view a summary of your request, click the deployment name.
4. Click the **User Events** tab.

   Until the user input request is answered, you see USER_INTERACTION_IN_PROGRESS and the list of users who must provide input.
5. If you are one of the response assignees, you see an input form.

   Provide the information required for the workflow to proceed, and click **Submit**.
6. When your request is answered or rejected, you receive another email message.

   If the user input request is rejected, your deployment request is canceled.

   The deployment details **History** tab displays REQUEST_FAILED.

How do I respond to a user input request in VCF Automation for VM Apps

When a VCF Operations orchestrator workflow that contains a user interaction element is requested in the VCF Automation for VM Apps catalog, the deployment remains in progress until the assigned users provide the required input.

- A VCF Operations orchestrator workflow that contains a manual user interaction element is added to the catalog.

A VCF Operations orchestrator workflow can sometimes require additional input parameters while it runs. For example, if a certain event occurs while a workflow runs, the workflow can request user interaction to decide what course of action to take. The workflow waits before continuing, either until the assigned user responds to the request for information, or until the waiting time exceeds a possible timeout period.

If multiple users are listed as assignees that can respond to the input request, only one of them must answer or reject the request.

You can use the filter option to limit the number of the user input requests that you manage. For example, rather than all the requests that are waiting for input, you can use the Answered filter to see only requests that were answered by you or other assignees.

1. If a workflow requires your input, you receive an email that provides the name of the requesting user, the name of the requested workflow, and a link to the request in the **User Input Requests** page in VCF Automation for VM Apps.

   You can also select **Inbox** > **User Input Requests** in VCF Automation continue with the following steps.

2. Locate the card for the user input request.
3. Review the request summary and input fields, and answer or reject the request.
   - Answer. Provide the required input, and click **Submit**.
   - Reject. If you reject a user input request, the deployment request for the workflow is canceled.
4. The system sends an email to the requester indicating that the user input request was answered or rejected.

How do I manage resources in VCF Automation for VM Apps
As a VCF Automation for VM Apps cloud administrator or catalog consumer, you can use the Deployments node on the Consume tab to manage your cloud resources.

You can locate and manage your resources using the different views. You can filter the lists, view resource details, and then run actions on the individual items. The available actions depend on the resource origin, for example, discovered compared to deployed, and the state of the resources.

If you are a VCF Automation for VM Apps administrator, you can also view and manage discovered machines.

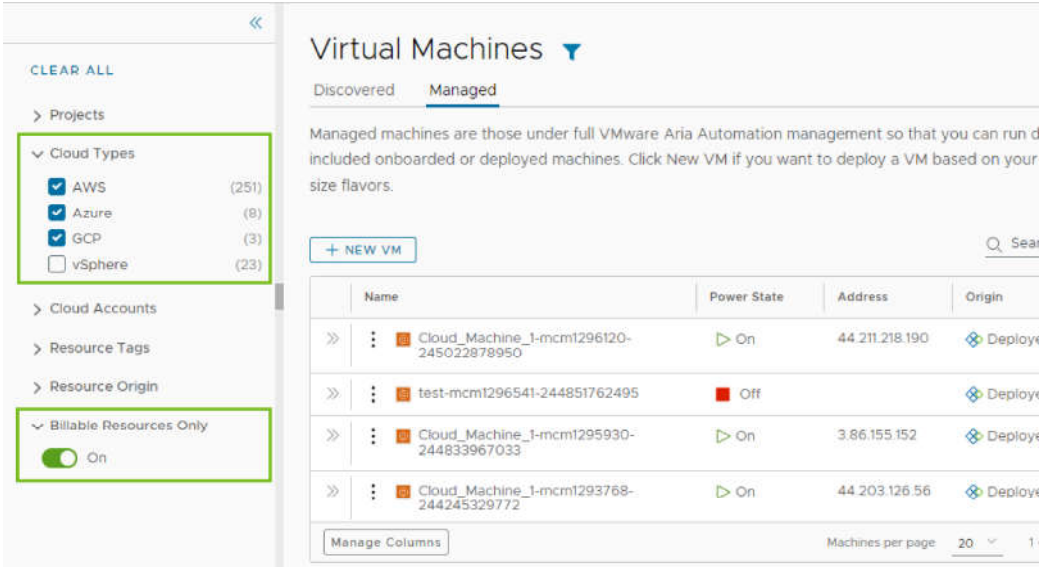
To view your resources, select **Consume** > **Deployments** > **Resources**.

You can use the **Projects** drop-down on the Consume tab to view only the resources that are available for a specific project.

Viewing billable objects

As an VCF Automation or an VCF Automation administrator, you can monitor what billable objects are used in your organization. Counted objects include billable virtual machines, CPUs, and cores that are in use at view time. It might take up to ten minutes for the object count data to refresh.

When viewing resource lists, you can use the Billable Resources Only filter in combination with other available filters.



If you force delete a deployment, the resource count might not match what is displayed in the resource list. You must delete the virtual machine from the corresponding IaaS layer, such as vCenter.

What is actually counted towards the bill depends on your VCF Automation subscription commit contract and entitlement type.

Working with the resource lists

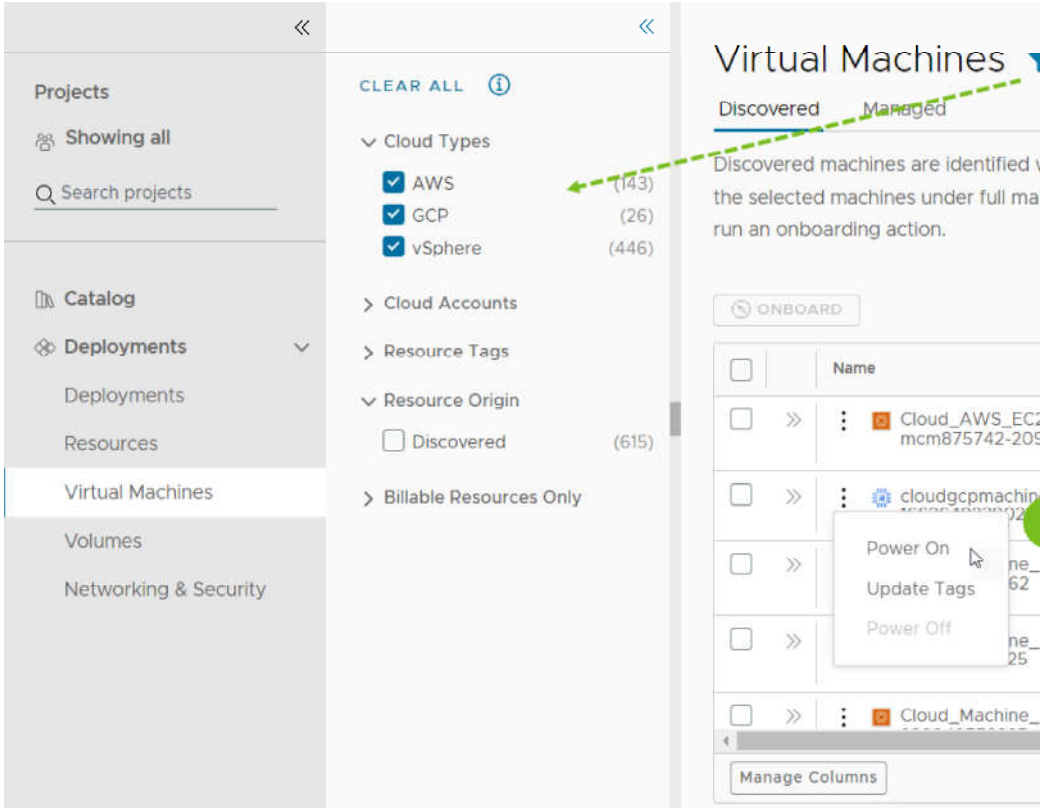
You can use the resource list to manage the machines, storage volumes, and networks that make up your deployments. In the resource list you can manage them in resource type groups rather than by deployments.

Similar to the deployment list view, you can filter the list, select a resource type, search, sort, and run actions.

If you click the resource name, you can work with the resource in the context of the deployment details.

If your administrator turned on the Create new resource setting, VCF Automation viewers and administrators, and project members have the option to **Create New VM**. Administrators can turn on the setting by selecting **Infrastructure** > **Administration** > **Settings**. To control the amount of resources that VCF Automation users might deploy, it is likely that your administrator added approval policies to reject or approve any deployment requests based on the image used or the flavor or size requested.

Figure 248: Resources page list



1. Filter your list based on resource attributes.
   For example, you can filter based on project, cloud types, origin, or other attributes.
2. Search for resources based on name, account regions, or other values.
3. Run available day 2 actions that are specific to the resources type and the resource state.
   For example, you might power on a discovered machine if it is off. Or you might resize an onboarded machine.

List of managed resources by origin

You can use the Resources tab to manage the following types of resources.

Table 1666: Resource origins

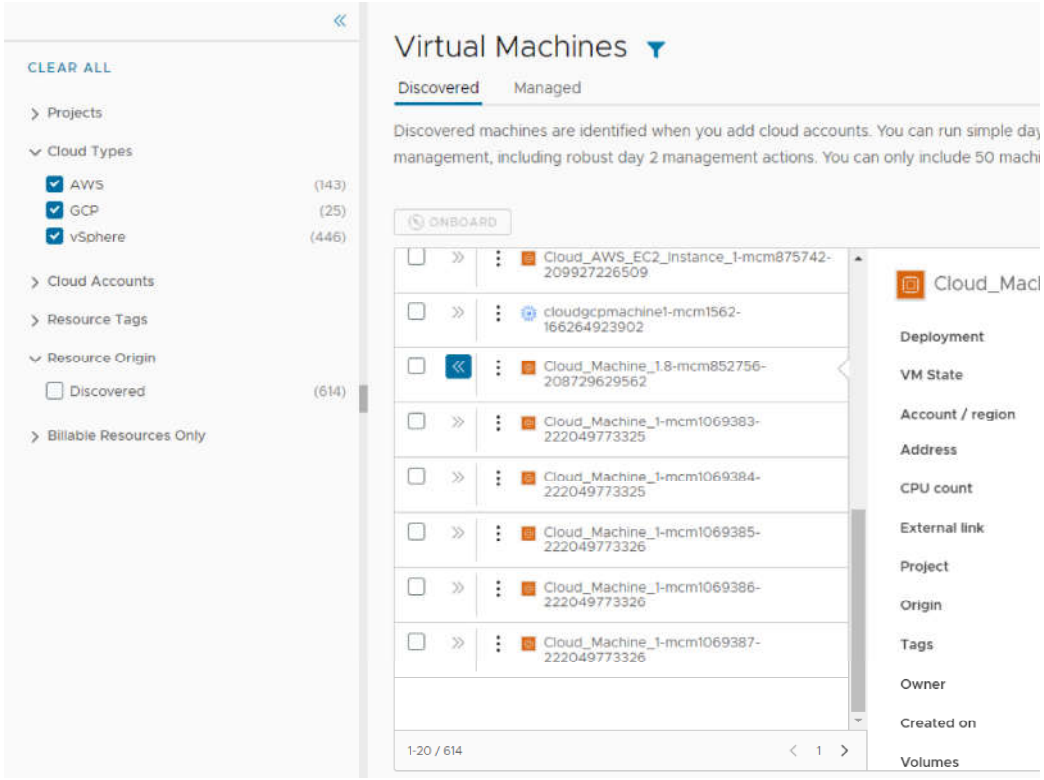
Managed Resource	Description
Deployed	Deployments are fully manage workloads that are deployed cloud templates or onboarded resources. The workload resources can include machines, storage volumes, networks, load balancers, and security groups. You can manage your deployments in the Deployments section.
Discovered	Discovered resources are the machines, storage volumes, networks, load balancers, and security groups that the discovery process identified for each cloud account region that you added. Only VCF Automation Administrators can see and manage discovered resources in the Resources section.
Migrated	Migrated resources are the 7.x deployments that your migrated to VCF Automation. The migrated resources can include machines, storage volumes, networks, load balancers, and security groups. Migrated resources are managed like deployments. You can manage migrated resources in the Deployments section.
Onboarded	Onboarded resources are discovered resources that you bring under more robust VCF Automation management. Onboarded resources are managed like deployments. You can manage onboarded resources in the Deployments section.

What is the resource details view

You can use the resource details view to get a deeper look at the selected resource. Depending on the resource, the details can include networks, ports, and other information collected about the machine. The depth of the information varies depending on cloud account type and origin.

To open the details pane, click the resource name or the double arrows.

Figure 249: Resources details pane



What day 2 actions can I run on resources

The available day 2 actions depend on the resource origin, cloud account, resource type, and state.

Table 1667: List of actions by origin

Resource Origin	Day 2 Actions
Deployed	The actions that are available to run on the resources depend on the resource type, cloud account, and state. For a detailed list, see What actions can I run on deployments or supported resources in VCF Automation for VM Apps.
Discovered	The available actions for discovered resources are limited to virtual machines. Depending on the status, you can perform the following actions.

	• Power Off • Power On Additional vSphere virtual machine action. • Connect to Remote Console
Migrated	Migrated resources have the same day 2 action management options as deployments. The actions that are available to run on the migrated resources depend on the resource type, cloud account, status, and day 2 policies. For a detailed list, see What actions can I run on deployments or supported resources in VCF Automation for VM Apps.
Onboarded	Onboarded resources have the same day 2 action management options as deployments. The actions that are available to run on the onboarded resources depend on the resource type, cloud account, and state. For a detailed list, see What actions can I run on deployments or supported resources in VCF Automation for VM Apps.

How do I work with individual resources in VCF Automation for VM Apps

As a cloud administrator or a project member with resources for your project, you can use the Resources page on the Consume tab to manage your deployed, onboarded, and migrated resources as individual resources by resource type.

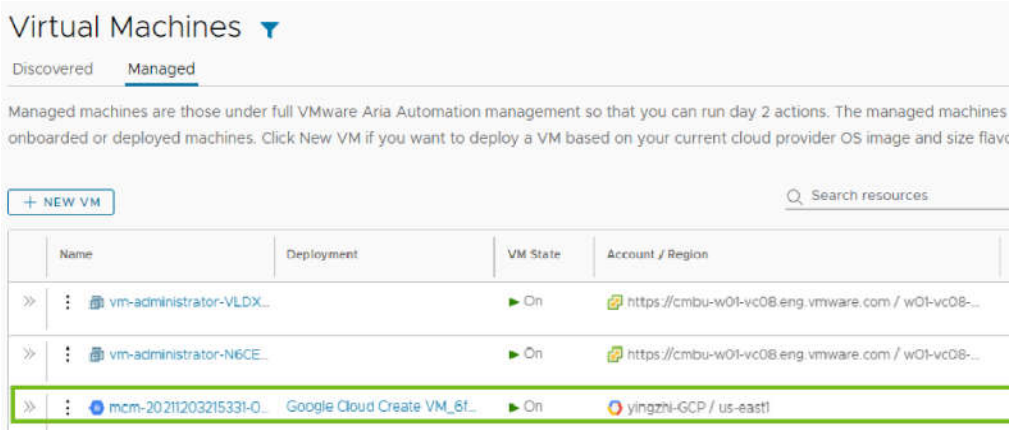
This workflow, which focuses on managing virtual machines, provides a guide for high-level resource life cycle management that you can apply to the other resource types.

Locate virtual machine resources

Deployed, onboarded, and migrated virtual machines are available on the Resources page and the Managed tab on the Virtual Machines page. This example focuses on virtual machines, but you can apply the same workflow to the other resource types.

1. Select **Consume** > **Deployments** > **Virtual Machines** > **Managed**.

2. Locate your virtual machine.
   You can use the filters or the search to locate particular resources.



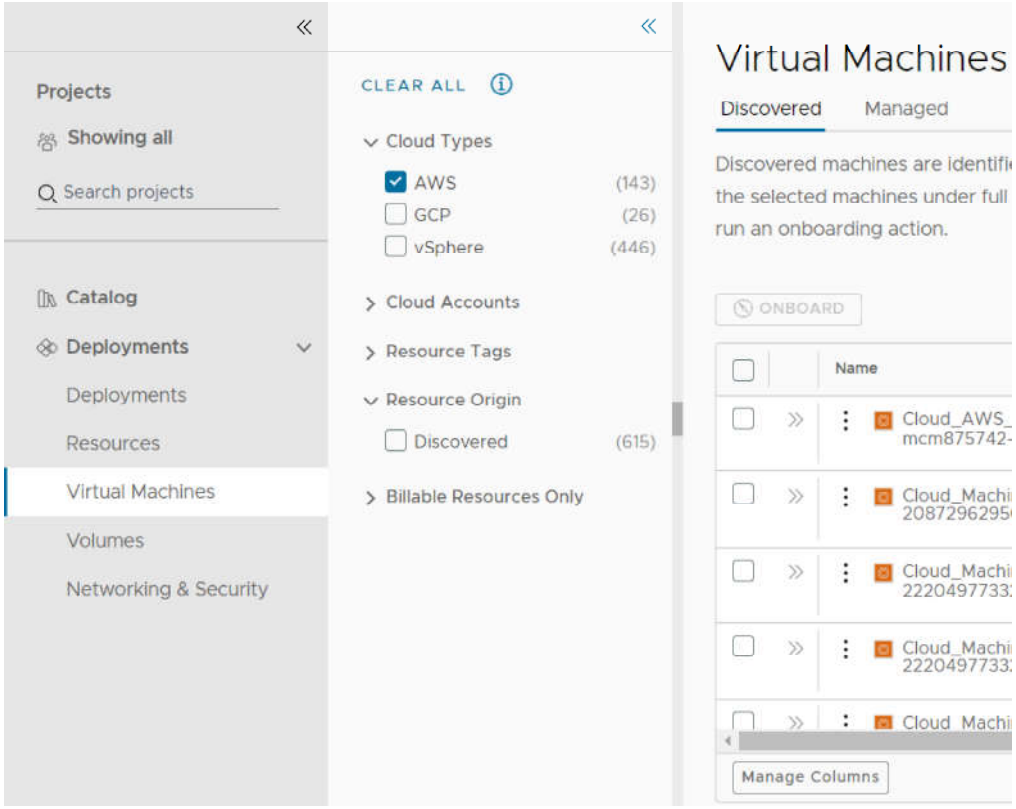
How do I work with discovered resources in VCF Automation for VM Apps
If you are a VCF Automation for VM Apps administrator, you use the Resources page on the Consume tab to manage your discovered machines. Only administrators will see discovered resources on the various pages.

This workflow focuses on managing discovered virtual machines.

Locate discovered virtual machines

Discovered resources are collected from the cloud account region and added to the resources on the Deployments node on the Consume tab. This example focuses on virtual machines, but other resource types are collected, including storage and network information.

1. Select **Consume** > **Deployments** > **Virtual Machines** > **Discovered**.



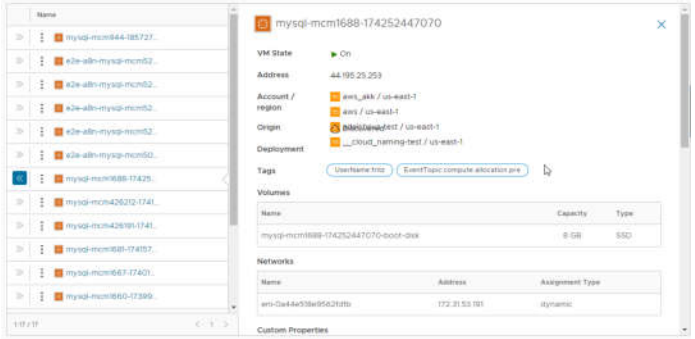
2. To locate the AWS virtual machines, click the **Filter** icon near the page label.
3. In the filter list, expand **Cloud Types** and select **AWS**.
   The list is now limited to discovered AWS virtual machines. You can see deployed, discovered, and other origin types on the Managed tab.
4. To locate a particular machine, you can use the **Search resources** option to search by name, IP address, tags, or values.
   In this example, `mysql` is the search term.

Review virtual machine details

The resource details include all the collected information for the resource. You can use this information to understand the resource and any associations with other resources.

1. Locate the virtual machine in the Virtual Machine list.

2. To view the resource details, click the machine name or click the double arrows in the left column.
   The details pane opens on the right side of the list.



3. Review the details, including storage, networks, custom properties, and other collected information.
4. To close the pane, click the double arrows or click the resource name.

Run day 2 actions on the virtual machine

You use the day 2 actions to manage the resources. The current actions for discovered virtual machines includes Power On and Power Off. If you are managing a vSphere virtual machine, you can also run Connect with Remote Console.

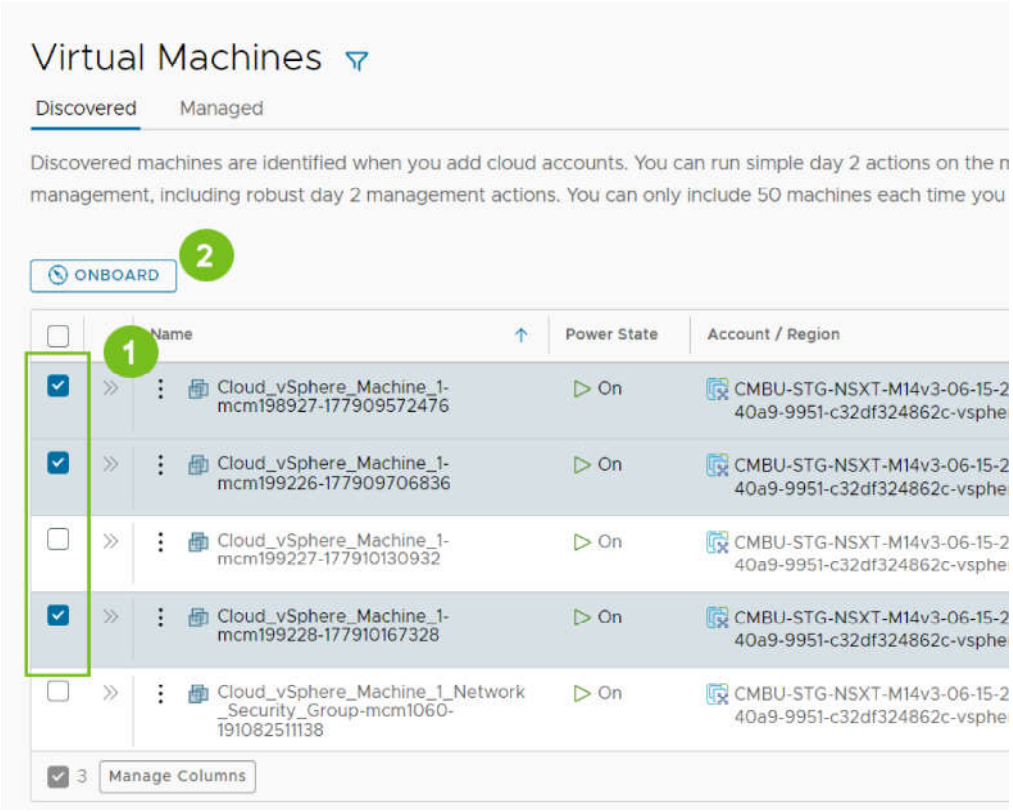
1. Locate the machine in the Virtual Machines list.
2. Click the vertical ellipsis to see the available actions.
   The possible actions for an AWS virtual machine are Power Off and Power On. Power On is not active because the machine is already on.
3. Click **Power Off** and submit the request.

When the process is completed, the machine is powered off. You can now power it back on.

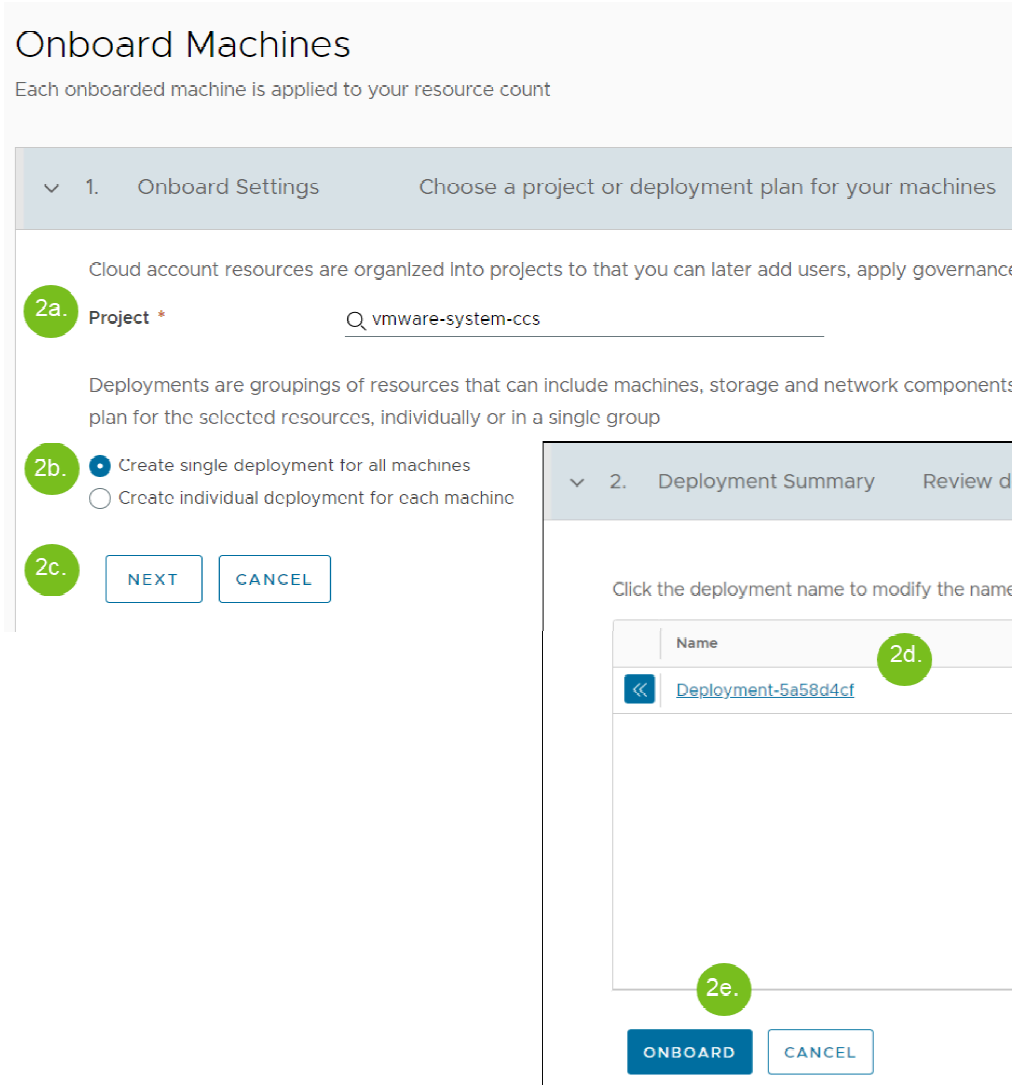
Onboarding virtual machines

You can quickly onboard discovered machines from the Virtual Machines page. Onboarding gets you full day 2 management capabilities for the onboarded resources. See What actions can I run on deployments or supported resources in VCF Automation for VM Apps.

1. On the **Discovered** tab, select the machines that you want to onboard and then click **Onboard**.
   You can onboard up to 50 virtual machines at a time.



2. Follow the prompts in the onboarding wizard.
   a. Select a project for the machines.
   b. Select if you want to group the machines into a single deployment or if you want to create a separate deployment for each machine.
   c. Click **Next**.
   d. Review the deployment summary. You can update the deployment name and owner by clicking the deployment name.
   e. When you're done, click **Onboard**.



   The selected machines are onboarded.
3. To view the deployment, go to **Resources** > **Deployments** and then click the deployment name.

4. You manage your onboarded machines, go to **Virtual Machines** > **Managed**.

What actions can I run on deployments or supported resources in VCF Automation for VM Apps
After you deploy catalog items, you can run actions in VCF Automation for VM Apps to modify and manage the resources. The available actions depend on the resource type and whether the action is supported on a particular cloud account or integrated platform.

The available actions also depend on what your administrator entitled you to run.

As an administrator or project administrator, you can set up Day 2 Actions policies. See How do I entitle deployment users to day 2 actions using policies in VCF Automation for VM Apps.

You might also see actions that are not included in the list. These are likely custom actions that your administrator configured in VCF Automation for VM Apps.

CAUTION:

To change a deployment, you can edit its cloud template and reapply it, or you can use day 2 actions. However, in most cases you should avoid mixing the two approaches.

Lifecycle day 2 changes such as power on/off are usually safe, but others require caution, such as when adding disks.

For example, if you add disks with a day 2 action, and then take a mixed approach by reapplying the cloud template, the cloud template could overwrite the day 2 change, which might remove disks and cause data loss.

Table 1668: List of possible actions

Action	Applies to these resource types	Available for these cloud types	Resource origin	Description
Add Disk	Machines	• Amazon Web Service • Google Cloud Platform • Microsoft Azure • VMware vSphere	• Deployed • Onboarded	Add additional disks to existing virtual machines. If you add a disk to an Azure machine, the persistent disk or non-persistent disk is deployed in the resource group that includes the machine. When you add a disk to an Azure machines, you can also encrypt the new disk using the Azure disk encryption set configured in the storage profile. You cannot add a disk to an Azure machine with an unmanaged disk. When you add a disk to vSphere machines, you can select the SCSI controller, the order of which was set in the cloud template and deployed. You can also specify the unit number for the new disk. You cannot specify a unit number without a selected controller. If you do not select a controller or provide a unit number, the new disk is deployed to first available controller and assigned then next available unit number on that controller. **Note:** Any virtual device that VCF Automation processes must be configured with the SCSI controller. If you add a disk to a vSphere machine for a project with defined storage limits, the added disk must not exceed the storage limits. Storage limits consider the actual capacity for thick and thin resource provisioning so that you cannot over-provision using thin provisioning.

Action	Applies to these resource types	Available for these cloud types	Resource origin	Description
				If you use VMware Storage DRS (SDRS) and the datastore cluster is configured in the storage profile, you can add disks on SDRS to vSphere machines.
Attach SaltStack Resource	Machines	• Amazon Web Services • Google Cloud Platform • Microsoft Azure • VMware vSphere	• Deployed • Onboarded	Attach a SaltStack Resource to a deployment resource so that you can install a Salt minion and update the Salt configuration on the virtual machine. You can use this action to update a configuration on a resource or attach the resource and install the minion on another resource in the deployment. The Attach Salt Resource action is available if you configured the SaltStack Config integration. To apply a configuration, you must select an authentication method. The **Remote access with existing credentials** uses the remote access credentials that are included in the deployment. If you changed the credentials on the machine after deployment, the action can fail. If you know the new credentials, use the Password authentication method. The **Password** and **Private key** use the user name and the password or key to validate your credentials and then connect to the virtual machine using SSH. If you do not provide a value for the Master ID and Minion ID, Salt creates the values for you.
Cancel	• Deployments • Various resource types in deployments	• Amazon Web Service • Google Cloud Platform • Microsoft Azure • VMware vSphere	• Deployed • Onboarded	Cancel a deployment or a day 2 action on a deployment or a resource while the request is being processed. You can cancel the request on the deployment card or in the deployment details. After you cancel the request, it appears as a failed request on the **Deployments** page. Use the **Delete** action to release any deployed resources and clean up your deployment list. Canceling a request that you think has been running too long is one method for managing deployment time. However, it is more efficient to set the **Request Timeout** in the projects. The default timeout is two hours. You can set if for a longer period of time if the workload deployment for a project requires more time.
Change Display Name	Disk	• Amazon Web Service • Microsoft Azure • VMware vSphere	• Deployed • Onboarded	Change the name of a disk to a meaningful display name. This action changes the display name in: • Topology (Node view) • Topology (Tree view) • Side panel • Resource name is day 2 actions, such as "Resize Disk" • All Resources grid view • Volumes gride view
Change Lease	Deployments	• Amazon Web Service • Microsoft Azure • VMware vSphere	• Deployed • Onboarded	Change the lease expiration date and time. When a lease expires, the deployment is destroyed and the resources are reclaimed. Lease policies are set in VCF Automation.

Action	Applies to these resource types	Available for these cloud types	Resource origin	Description
Change Owner	Deployments	• Amazon Web Service • Google Cloud Platform • Microsoft Azure • VMware vSphere	• Deployed • Onboarded	Changes the deployment owner to the selected user or Active Directory group. The selected user or AD group must be an administrator or a member of the same project that deployed the request. Only users or AD groups defined in the project are available to become the owner. Custom groups are not eligible to be the target owner. When a cloud template designer deploys a template, the designer is both the requester and the owner. However, a requester can make another project member the owner. You can use policies to control what an owner can do with a deployment, giving them permissions that are more restrictive or less restrictive.
Change Project	Deployments	• Amazon Web Service • Google Cloud Platform • Microsoft Azure • NSX-T • NSX-V • VMware Cloud Director • VMware Cloud Foundation • VMware Cloud on AWS • VMware vSphere	• Deployed • Migrated • Onboarded	You use the change project action to move a deployment from one project to another project. The change project action is available for deployments with deployed resources, migrated resources, onboarded resources, and deployments with a mixture of deployed, migrated, and onboarded resources. Supported resources include the following resource types and constraints: • Deployments with deployed resources can contain virtual machines, disks, load balancers, networks, security groups, Azure groups, NATs, gateways, custom resources, Terraform configurations, and Ansible and Ansible Tower resources. • Deployments with migrated resources can contain virtual machines, disks, load balancers, networks, security groups, NATs, gateways, and custom resources. • Deployments with onboarded resources can contain virtual machines, disks, and networks. • If you add an unsupported resource type in any deployment type, deployed, migrated, or onboarded resources, you cannot run the change project action. Roles, considerations, and constraints for deployments with deployed, migrated, onboarded, and hybrid/mixed resources: • To change the project of a deployment with deployed or migrated resources, the initiating user must have the following role: – Cloud administrator. • You can only change the project when the target project contains all the cloud zones where the deployment's machines and disks are deployed. The moved deployment is then subject to the configured limits of the target project, including instance count, memory, CPU, and storage. After the move, the current usage is released from the source project.

Action	Applies to these resource types	Available for these cloud types	Resource origin	Description
				• After you move a deployment to the target project, it is subject to the policies of target project. For example, lease, day 2 actions, resource quota, and other polices. To move a deployment, the deployment lease defined by the lease policy of the target project cannot expire in the next 24 hours. • The Change Project action is available for deployments where the custom resources are scoped to be available to any project. The lifecycle and day 2 actions for each custom resource in the deployment must be shared with all projects to ensure that you can continue to manage the custom resource using lifecycle actions or day 2 actions after you move the deployment to the new project. • The extensibility constraints of the target project must match the vRO integration or the same ABX FaaS provider as the source project. The integrations and providers must match so that you can manage the custom resource using lifecycle actions or day 2 actions after you move the deployment to the new project. • The Change Project action is available for deployments with Terraform configurations where all the content sources are shared. If any content sources used in the deployment are not shared, the Change Project action fails validation and does not run. • If a Change Project action fails because a Terraform GitHub repository content source was not shared, you can note the corresponding ID that you must share and then share the source. To share the source repository, select **Infrastructure** > **Integrations** and select the GitHub integration. In the open integration page, click the Projects tab, expand the project that contains the repository for the failed action and turn on the sharing for that repository. You can also use the API to share the content source repositories. • The Change Project action might not be available if you migrated a deployment that contains a custom resource, where the custom resource includes an update action, and you have performed an iterative update of the deployment by redeploying the cloud template. Roles, considerations, and constraints for deployments with onboarded resources: • To move a deployment with onboarded resources, the initiating user must have at least one of the following roles: – Cloud administrator. – Manage Deployments permission. This permission can be defined as a custom role. – Project administrator of the target project. – Project member of the target project and the deployments are shared between all users in the target project.

Action	Applies to these resource types	Available for these cloud types	Resource origin	Description
				• While you can move onboarded resources to a project that does not contain the same cloud zones, if the target project does not have the same cloud zones, any future day 2 actions involving cloud account / region resources that you run might not work. General considerations: • If you are an administrator who is moving the deployment, you might move the deployment to a project where the owner is not a member and therefore loses access. To resolve the problem, you can add the owner to the target project, move the deployment to a project where they are a member, or use the Change Owner action.
Change Security Groups	Machines	• VMware vSphere	• Deployed • Onboarded	You can associate and dissociate security groups with machine networks in a deployment. The change action applies to existing and on-demand security groups for NSX-V and NSX-T. This action is available only for single machines, not machine clusters. To associate a security group with the machine network, the security group must be present in the deployment. Dissociating a security group from all networks of all machines in a deployment does not remove the security group from the deployment. These changes do not affect security groups applied as part of the network profiles. This action changes the machine's security group configuration without recreating the machine. This is a non-destructive change. • To change the machine's security group configuration, select the machine in the topology pane, then click the **Action** menu in the right pane and select **Change Security Groups**. You can now add or remove the association on the security groups with the machine networks.
Connect to Remote Console	Machines	• VMware vSphere	• Deployed • Discovered • Onboarded	Open a remote session on the selected machine. Review the following requirements for a successful connection. • As a deployment consumer, verify that the provisioned machine is powered on.
Create Disk Snapshot	Machines and disks	• Microsoft Azure	• Deployed • Onboarded	Create a snapshot of a virtual machine disk or a storage disk. • For machines, you create snapshots for individual machine disks, including boot disk, image disks, and storage disks. • For storage disks, you create snapshots of independent managed disks, not unmanaged disks. In addition to providing a snapshot name, you can also provide the following information for the snapshot:

Action	Applies to these resource types	Available for these cloud types	Resource origin	Description
				• Incremental Snapshot. Select the check box to create a snapshot of the changes since the last snapshot rather full snapshot. • Resource Group. Enter the name of the target resource group where you want to create the snapshot. By default, the snapshot is created in the same resource group that is used by the parent disk. • Encryption Set Id. Select the encryption key for the snapshot. By default, the snapshot is encrypted with the same key that is used by the parent disk. • Tags. Enter any tags that will help you manage the snapshots in Microsoft Azure.
Create Snapshot	Machines	• Google Cloud Platform • VMware vSphere	• Deployed • Onboarded	Create a snapshot of the virtual machine. If you are allowed only two snapshots in vSphere and you already have them, this command is not available until you delete a snapshot. When creating a snapshot for a Google Cloud Platform machine, you can also create a disk snapshot of the attached disks. The combined snapshot allows you to manage the machine as the attached disks as a single entity.
Delete	Deployments	• Amazon Web Service • Google Cloud Platform • Microsoft Azure • VMware vSphere	• Deployed • Onboarded	Destroy a deployment. All the resources are deleted and the reclaimed. If a delete fails, you can run the delete action on a deployment a second time. During the second attempt, you can select **Ignore Delete Failures**. If you select this option, the deployment is deleted, but the resources might not be reclaimed. You should check the systems on which the deployment was provisioned to ensure that all resources are removed. If they are not, you must manually delete the residual resources on those systems.
	NSX Gateway	• NSX	• Deployed • Onboarded	Delete the NAT port forwarding rules from an NSX-T or NSX-V gateway.
	Machines and load balancers	• Amazon Web Service • Microsoft Azure • VMware vSphere • VMware NSX	• Deployed • Onboarded	Remove a machine or load balancer from a deployment. This action might result in an unusable deployment.
	Security groups	• NSX-T • NSX-V	• Deployed • Onboarded	If the security is not associated with any machine in the deployment, the process removes the security group from the deployment. • If the security group is on-demand, then it is destroyed on the endpoint. • If the security group is shared, the action fails.
	Tanzu Kubernetes clusters	• VMware vSphere	• Deployed • Onboarded	Remove a Tanzu Kubernetes cluster from a deployment.
Delete Disk Snapshot	Machines and disks	• Microsoft Azure	• Deployed • Onboarded	Delete an Azure virtual machine disk or managed disk snapshot. This action is available when there is at least one snapshot.

Action	Applies to these resource types	Available for these cloud types	Resource origin	Description
Delete Snapshot	Machines	• VMware vSphere • Google Cloud Platform	• Deployed • Onboarded	Delete a snapshot of the virtual machine.
Disable Boot Diagnostics	Machines	• Microsoft Azure	• Deployed • Onboarded	Turn off the Azure virtual machine debugging feature. This option is only available if the feature is turned on.
Disable Log Analytics	Machines	• Microsoft Azure	• Deployed • Onboarded	Turn off the ability to run log queries on Azure virtual machine logs. Select the name of the extension that you want to deactivate. If there is no extension name to select, then the log analytics are not currently enabled on this machine. To work with the log analytics, Azure Monitor and the cloud template must be configure to support the workspace. See *Log Analytics*.
Edit Tags	Deployments	• Amazon Web Service • Microsoft Azure • VMware vSphere	• Deployed • Onboarded	Add or modify resource tags that are applied to individual deployment resources.
Enable Boot Diagnostics	Machines	• Microsoft Azure	• Deployed • Onboarded	Turn on the Azure virtual machine debugging feature to diagnose virtual machine boot failures. The boot diagnostics information is available in your Azure console. The Enable option is only available if the feature is not currently turned on.
Enable Log Analytics	Machines	• Microsoft Azure	• Deployed • Onboarded	Turn on the Azure virtual machine to edit and run log queries on data collected by Azure Monitor logs. You provide the extension name. The workspace ID and key must be the values that are configured in Azure. To work with the log analytics, Azure Monitor and the cloud template must be configure to support the workspace. See *Log Analytics*.
Get Terraform State	Terraform Configuration	• Amazon Web Service • Google Cloud Platform • Microsoft Azure • VMware vSphere	• Deployed • Onboarded	Display the Terraform state file. To view any changes that were made to the Terraform machines on the cloud platforms that they were deployed on and update the deployment, you first run the Refresh Terraform State action, and then run this Get Terraform State action. When the file is displayed in a dialog box. The file is available for approximately 1 hour before you need to run a new refresh action. You can copy it if you need it for later. You can also view the file on the deployment History tab. Select the Get Terraform State event on the Events tab, and then click **Request Details**. If the file is not expired, click **View content**. If the file is expired, run the Refresh and Get actions again. 

Action	Applies to these resource types	Available for these cloud types	Resource origin	Description
				You can run other day 2 action on the Terraform resources that are embedded in the configuration. The available actions depend on the resource type, the cloud platform that they are deployed on, and whether you are entitled to run the actions based on a day 2 policy.
Power Off	Deployments	• Amazon Web Service • Microsoft Azure • VMware vSphere	• Deployed • Discovered • Onboarded	Power off the deployment after first attempting to shutdown the guest operating systems. If the soft power off fails, a hard power off still runs.
	Machines	• Amazon Web Service • Google Cloud Platform • Microsoft Azure • VMware vSphere	• Deployed • Onboarded	Power off the machine after first attempting to shut down the guest operating systems. If the soft power off fails, the hard power off still runs.
Power On	Deployments	• Amazon Web Service • Microsoft Azure • VMware vSphere	• Deployed • Onboarded	Power on the deployment. If the resources were suspended, normal operation resumes from the point at which they were suspended.
	Machines	• Amazon Web Service • Google Cloud Platform • Microsoft Azure • VMware vSphere	• Deployed • Discovered • Onboarded	Power on the machine. If the machine was suspended, normal operation resumes from the point at which the machine was suspended.
Reboot	Machines	• Amazon Web Service • VMware vSphere	• Deployed • Onboarded	Reboot the guest operating system on a virtual machine. For a vSphere machine, VMware Tools must be installed on the machine to use this action.
Rebuild	Deployments	• VMware vSphere	• Deployed • Migrated • Onboarded	Rebuild several or all virtual machines in the deployment where the deployment failed or not all requested VMs are provisioned successfully. The rebuild process keeps the same configuration, such as name, ID, IP address, machine data store, and custom properties for each selected machine. A non-persistent disk that is attached to a virtual machine is wiped clean and then recreated as part of the build action. Any attached first class disks are detached and the contents is retained. After you rebuild the machine, you can re-attach the disk. For machines with a missing image, you must select a valid image to rebuild.
	Machines	• VMware vSphere	• Deployed • Migrated • Onboarded	Rebuild a virtual machine where the deployment resulted in a partial deployment, the virtual machine is not usable, or to reprovision a problematic virtual machine after a successful deployment. The rebuild process keeps the same configuration, such as name, ID, IP address, machine data store, and custom properties. A non-persistent disk that is attached to a virtual machine is wiped clean and then recreated as part of the build action. Any attached first class disks are detached and the contents is retained. After you rebuild the machine, you can re-attach the disk.
Reconfigure	Load Balancers	• Amazon Web Service • Microsoft Azure	• Deployed • Onboarded	Change the load balancer size and logging level.

Action	Applies to these resource types	Available for these cloud types	Resource origin	Description
		• VMware NSX		You can also add or remove routes, and change the protocol, port, health configuration, and member pool settings. For NSX load balancers, you can enable or deactivate the health check and modify the health options. For NSX-T, you can set the check to active or passive. NSX-V does not support passive health checks.
	NSX Gateway port forwarding	• NSX-T • NSX-V	• Deployed • Onboarded	Add, edit, or delete the NAT port forwarding rules from an NSX-T or NSX-V gateway.
	Security Groups	• NSX-T • NSX-V • VMware Cloud • VMware vSphere	• Deployed • Onboarded	Add, edit, or remove firewall rules or constraints based on whether the security group is an on-demand or an existing security group. • On-demand security group Add, edit, or remove firewall rules for NSX-T and VMware Cloud on-demand security groups. – To add or remove a rule, select the security group in the topology pane, click the **Action** menu in the right pane, and select **Reconfigure**. You can now add, edit, or remove the rules. • Existing security group Add, edit, or remove constraints for existing NSX-V, NSX-T, and VMware Cloud security groups. – To add or remove a constraint, select the security group in the topology pane, click the **Action** menu in the right pane, and select **Reconfigure**. You can now add, edit, or remove the constraints.
Refresh Terraform State	Terraform Configuration	• Amazon Web Service • Google Cloud Platform • Microsoft Azure • VMware vSphere	• Deployed • Onboarded	Retrieve the latest iteration of the Terraform state file. To retrieve any changes that were made to the Terraform machines on the cloud platforms that they were deployed on and update the deployment, you first run this Refresh Terraform State action. To view the file, run the **Get Terraform State** action on the configuration. Use the deployment history tab to monitor the refresh process.
Remove Disk	Machines	• Amazon Web Service • Google Cloud Platform • Microsoft Azure • VMware vSphere	• Deployed • Onboarded	Remove disks from existing virtual machines. If you run the day 2 action on a deployment that is deployed as vSphere machines and disks, the disk count is reclaimed as it applies to project storage limits. Storage limits consider the actual capacity for thick and thin resource provisioning so that you cannot over-provision using thin provisioning. The project storage limits do not apply to additional disks that you added after deployment as a day 2 action.
Reset	Machines	• Amazon Web Service • Google Cloud Platform • VMware vSphere	• Deployed • Onboarded	Force a virtual machine restart without shutting down the guest operating system.
Resize	Machines	• Amazon Web Service • Microsoft Azure • Google Cloud Platform	• Deployed • Onboarded	Increase or decrease the CPU and memory of a virtual machine.

Action	Applies to these resource types	Available for these cloud types	Resource origin	Description
		• VMware vSphere		
Resize Boot Disk	Machines	• Amazon Web Service • Google Cloud Platform • Microsoft Azure • VMware vSphere	• Deployed • Onboarded	Increase or decrease the size of your boot disk medium. If you run the day 2 action on a deployment that is deployed as vSphere machines and disks, and the action fails with a message similar to “The requested storage is more than the available storage placement,” it is likely due to the defined storage limits on your vSphere VM templates and the content library that are defined in the project. Storage limits consider the actual capacity for thick and thin resource provisioning so that you cannot over-provision using thin provisioning. The project storage limits do not apply to additional disks that you added after deployment as a day 2 action.
Resize Disk	Storage disk	• Amazon Web Service • Google Cloud Platform	• Deployed • Onboarded	Increase the capacity of a storage disk. If you run the day 2 action on a deployment that is deployed as vSphere machines and disks, and the action fails with a message similar to “The requested storage is more than the available storage placement,” it is likely due to the defined storage limits on your vSphere VM templates and the content library that are defined in the project. Storage limits consider the actual capacity for thick and thin resource provisioning so that you cannot over-provision using thin provisioning. The project storage limits do not apply to additional disks that you added after deployment as a day 2 action. For the vSphere Storage DRS, you can relocate the virtual machine within the cluster of the current LUN lacks sufficient space.
	Machines	• Amazon Web Service • Google Cloud Platform • Microsoft Azure • VMware vSphere	• Deployed • Onboarded	Increase or decrease the size of disks included in the machine image template and any attached disks.
Restart	Machines	• Microsoft Azure	• Deployed • Onboarded	Shut down and restart a running machine.
Revert to Snapshot	Machines	• Google Cloud Platform • VMware vSphere	• Deployed • Onboarded	Revert to a previous snapshot of the machine. You must have an existing snapshot to use this action. If you created a snapshot for a Google Cloud Platform machine that included the attached disks, the full snapshot is reverted.
Run Puppet Task	Managed resources	• Puppet Enterprise	• Deployed • Onboarded	Run the selected task on machines in your deployment. The tasks are defined in your Puppet instance. You must be able to identify the task and provide the input parameters.
Scale Worker Nodes	Tanzu Kubernetes clusters	• VMware vSphere	• Deployed • Onboarded	Increase or decrease the number of Tanzu Kubernetes worker node virtual machines in your deployment.
Shutdown	Machines	• VMware vSphere	• Deployed	Shut down the guest operating system and power off the machine. VMware Tools must be installed on the machine to use this action.

Action	Applies to these resource types	Available for these cloud types	Resource origin	Description
Suspend	Machines	• Microsoft Azure • VMware vSphere	• Deployed • Onboarded	Pause the machine so that it cannot be used and does not consume any system resources other than the storage it is using.
Update	Deployments	• Amazon Web Service • Microsoft Azure • VMware vSphere	• Deployed • Onboarded	Change the deployment based on the input parameters. For an example, see How to move a deployed machine to another network in VCF Automation for VM Apps. If the deployment is based on vSphere resources, and the machine and disks include the count option, storage limits defined in the project might apply when you increase the count. If the action fails with a message similar to “The requested storage is more than the available storage placement,” it is likely due to the defined storage limits on your vSphere VM templates that are defined in the project. The project storage limits do not apply to additional disks that you added after deployment as a day 2 action. There are some properties that cannot be updated using this action. See Deployment properties that you cannot update using day 2 actions in VCF Automation for VM Apps.
Update Salt Configuration	SaltStack Config resource	• Amazon Web Service • VMware vSphere	• Deployed • Onboarded	Add or change the Salt environment, apply state files, or provide variables for the selected Salt resource.
Update Tags	Machines and disks	• Amazon Web Service • Microsoft Azure • VMware vSphere	• Deployed • Onboarded	Add, modify, or delete a tag that is applied to an individual resource.
Update Tanzu Version	Tanzu Kubernetes clusters	• VMware vSphere	• Deployed • Onboarded	Update the current Kubernetes version to a later version.
Unregister	Machines	• Amazon Web Service • Google Cloud Platform • Microsoft Azure • VMware vSphere	• Deployed (vSphere only) • Onboarded	Unregister machines and machine clusters to remove them from VCF Automation management and inventory. The machines are not removed from the cloud platform. Unregistered machines are available for onboarding. You can run the onboarding workflow to bring them back under management. For example, you might want to onboard a machine into a new project or a different VCF Automation instance.

Action	Applies to these resource types	Available for these cloud types	Resource origin	Description
				• Deployed vSphere machines considerations. – The unregistered machine, along with any attached disks, is removed from VCF Automation management. If you must continue to manage the disk, you can use the IaaS API to detach the disk before you unregister the machine. – If a deployment has only one machine and you unregister the machine, the deployment remains in VCF Automation. – An unregistered machine can still be a discovered machine that you can onboarded, if needed. – The machine is removed from VCF Automation licensing and metering. – The reservation quota and the storage limit are adjusted when the machine is no longer managed. – When you unregister a machine, the IP address changes from ALLOCATED to UNREGISTERED in AUTOMATION. If the machine is re-onboarded, the IP address changes back to ALLOCATED. If an unregistered machine is deleted in vCenter, the IP address remains UNREGISTERED in VCF Automation. To manually release the IP address, select **Infrastructure** > **Resources** > **Networks**. Select the **IP Addresses** tab and search for IP addresses where the status is Unregistered. Release the IP address as needed. – All NAT rules that target the unregistered machine's NIC are removed from the NAT rules list in VCF Automation when the machine is unregistered. If all the NAT rules target only the NICs on the machine that you want to unregister, the unregister fails. NAT needs at least one rule in the list. You can reconfigure the NAT rules or delete the NAT before you unregister the machine. No changes are made to the NSX DNAT or router configurations. – Unregister extensibility events topics exist and pre and post unregister events are generated by the Event Broker Service. • Onboarded machines considerations. – The unregistered machine, along with the any attached disks, is removed from VCF Automation management. – After you remove the machine, you can then re-run the onboarding workflow for the unregistered machine. For example, you might want to onboard the resource again, this time to a new project.

Action	Applies to these resource types	Available for these cloud types	Resource origin	Description
				• Missing machines considerations. – The machine is listed as missing after migration or a backup and restore operation on the vCenter. Missing commonly means that the database record is unusable. – You can unregister the missing machine to remove it from the database. The unregistered missing machine is not available for onboarding.

How to move a deployed machine to another network in VCF Automation for VM Apps

While maintaining deployments and networks, you might need the ability to relocate machines that you deployed with VCF Automation for VM Apps.

- The VCF Automation network profile must include all subnets that the machine will connect to. In VCF Automation, you can check networks by going to **Infrastructure** > **Configure** > **Network Profiles**.
  The network profile must be in an account and region that are part of the appropriate VCF Automation project for your users.
- Tag the two subnets with different tags. The example that follows assumes that `test` and `prod` are the tag names.
- The deployed machine must keep the same IP assignment type. It can't change from static to DHCP, or vice versa, while moving to another network.

For example, you might deploy to a test network first, then move to a production network. The technique described here lets you design a cloud template in advance to prepare for such day 2 actions. Note that the machine is moved. It isn't deleted and redeployed.

This procedure only applies to `Cloud.vSphere.Machine` resources. It won't work for cloud agnostic machines deployed to VCF Automation.

1. In VCF Automation for VM Apps, go to **Design**, and create a cloud template for the deployment.
2. In the inputs section of the code, add an entry that lets the user select a network.

```
inputs:
  net-tagging:
    type: string
    enum:
      - test
      - prod
    title: Select a network
```

3. In the resources section of the code, add the `Cloud.Network` and connect the machine to it.
4. Under the `Cloud.Network`, create a constraint that references the selection from the inputs.

```
resources:
  ABCServer:
    type: Cloud.vSphere.Machine
    properties:
      name: abc-server
      . . .
      networks:
       - network: '${resource["ABCNet"].id}'
  ABCNet:
```



```
    type: Cloud.Network
    properties:
      name: abc-network
      . . .
      constraints:
       - tag: '${input.net-tagging}'
```

5. Continue with your design, and deploy it as you normally would. At deployment, the interface prompts you to select the `test` or `prod` network.
6. When you need to make a day 2 change, go to **Resources** > **Deployments** > **Deployments**, and locate the deployment associated with the cloud template.
7. To the right of the deployment, click **Actions** > **Update**.
8. In the Update panel, the interface prompts you the same way, to select the `test` or `prod` network.
9. To change networks, make your selection, click **Next**, and click **Submit**.

Deployment properties that you cannot update using day 2 actions in VCF Automation for VM Apps

Day 2 actions are changes that you can make to deployed resources. There are limits to the changes that you can make using the actions. There are some properties that you cannot change using certain actions. Review the list of excluded properties if you expected a change but nothing was modified.

Storage Properties

The following storage properties cannot be modified using the **Update** deployment action.

Table 1669: List of excluded properties

Deployment Component	Property
Cloud.Machine	storage - bootDiskCapacityInGB
Cloud.Machine	storage - maxDiskCapacityInGB
Cloud.Machine	storage - constraints
Cloud.Volume	encrypted
Cloud.Volume	name
Cloud.Volume	constraints
Cloud.Volume	maxDiskCapacityInGB
Cloud.Volume	persistent
Cloud.Volume	tags
Cloud.Vsphere.Disk	datastore
Cloud.Vsphere.Disk	storagePolicy
Cloud.Vsphere.Disk	provisioningType
Cloud.Vsphere.Disk	SCSIController
Cloud.Vsphere.Disk	UnitNumber
Cloud.AWS.Volume	volumeType
Cloud.AWS.Volume	iops
Cloud.Azure.Disk	resourceGroup

Deployment Component	Property
Cloud.Azure.Disk	storageAccountName
Cloud.Azure.Disk	managedDiskType
Cloud.Azure.Disk	diskCaching
Cloud.GCP.Disk	persistentDiskType

Installing and Using VCF CLI v9.0

The VMware Cloud Foundation command-line interface (VCF CLI) is a command-line tool that enables you to interact with VCF Consumption services.

For example, you can use the VCF CLI to:

- Create and manage contexts for vSphere Namespaces
- Create and manage workload clusters
- Manage Kubernetes releases
- Install and manage packages on workload clusters
- Configure the VCF CLI itself

Installing the VCF CLI

The VCF CLI adapts to both internet-connected and internet-restricted environments, offering suitable installation methods for each situation.

- Installing the VCF CLI in Internet Connected Environments
- Installing the VCF CLI in Internet Restricted Environments

Update the VCF CLI

Get the current version of VCF CLI using the below command:

```
vcf version
v9.0.0
...
```

To update your VCF CLI version, follow the instructions on the below page:

Updating the VCF CLI

Interactive Prompts

When you run any VCF CLI command, except **`vcf version`** and **`vcf config set`**, for the first time, the VCF CLI prompts you to accept the Broadcom General Terms and configure the status of Broadcom's Customer Experience Improvement Program (CEIP). If you plan to use VCF CLI commands non-interactively, for example, in automation scripts, ensure that you accept the Broadcom General Terms and configure your CEIP status before executing VCF CLI commands in your scripts.

To configure CEIP, you can set use the **`vcf telemetry cli-usage-analytics update`** command. For more information about **`vcf telemetry cli-usage-analytics update`**, see telemetry in *VCF CLI Command Reference*.

Installing the VCF CLI in Internet Connected Environments

You can install the VCF CLI from a binary release or using a package manager such as Homebrew, Chocolatey, APT, YUM, and DNF.

In order to install VCF CLI , follow the steps below :

1. Installing the VCF CLI
2. Installing VCF CLI Plugins

Installing the VCF CLI

You can install the VCF CLI from a binary release or using a package manager such as Homebrew, Chocolatey, APT, YUM, and DNF.

- Installing VCF CLI Using Package Managers
- Install VCF CLI From Binary Releases

Installing VCF CLI From Binary Release

VCF CLI binary can be downloaded and installed from Broadcom's public artifactory.

Steps to download and install VCF CLI:

1. Visit Broadcom's public artifactory vcf distro link and download the binary based on OS and ARCH type.
2. Unpack the VCF CLI file for your OS and ARCH:

   MacOS:

   ```
   tar -xvf vcf-cli.tar.gz
   ```

   Linux:

   ```
   tar -xvf vcf-cli.tar.gz
   ```

   Windows:

   Use the Windows extractor tool to unzip **vcf-cli.zip**

3. Navigate to the directory containing the extracted CLI binary and then follow the steps below to install the CLI:

   MacOS:

   Install the binary to /usr/local/bin:

   ```
   sudo install vcf-cli-darwin_amd64 /usr/local/bin/vcf
   ```

   Linux:

   ```
   sudo install vcf-cli-linux_amd64 /usr/local/bin/vcf
   ```

   Windows:

   1. Create a new `C:\Program Files\vcf` folder.
   2. Copy the **vcf-cli-windows_amd64.exe** file into the new `C:\Program Files\vcf` folder.
   3. Rename **`vcf-cli-windows_amd64.exe`** to **`vcf.exe`**.
   4. Right-click the **`vcf`** folder, select **Properties > Security**, and make sure that your user account has the **Full Control**permission.
   5. Use Windows Search to search for **env.**
   6. Select **Edit the system environment variables** and click the **Environment Variables** button.
   7. Select the **Path** row under **System variables**, and click **Edit**.
   8. Click **New** to add a new row and enter the path to the **vcf** CLI. The path value must not include the **.exe** extension. For example, `C:\Program Files\vcf`.

4. Check that the correct version of the CLI is properly installed.

   ```
   vcf version
   v9.0.0
   ...
   ```

5. Proceed to Installing VCF CLI Plugins if you intend to use the VCF CLI in an internet-connected environment or to Download VCF CLI Plugins if you intend to use the VCF CLI in an internet-restricted environment. If you plan to use VCF CLI commands in automation scripts, see Interactive Prompts before installing or downloading the plugins.

Installing VCF CLI Using Package Managers
Follow the instructions for your package manager below. This installs the latest version of the CLI available in the package registry.

Tip:

If you want to retain an existing installation of the VCF CLI, move the CLI binary from `/usr/local/bin/vcf` to a different location before following these steps.

Homebrew (MacOS):

```
brew tap vmware/homebrew-vcfcli # Only needs to be done once for the machine
brew install vcf-cli
```

To uninstall the CLI:

```
brew uninstall vcf-cli
```

APT (Debian or Ubuntu)

```
sudo apt update
sudo apt install -y ca-certificates curl gpg

mkdir -p /etc/apt/keyrings
```



```
curl -fsSL https://packages.broadcom.com/artifactory/vcfcli-debian/tools/keys/BROADCOM-PACKAGING-GPG-RSA-
KEY.pub; curl -fsSL https://packages.broadcom.com/artifactory/api/security/keypair/PackagesKey/public | sudo
 gpg --dearmor -o /etc/apt/keyrings/vcf-archive-keyring.gpg

echo "deb [signed-by=/etc/apt/keyrings/vcf-archive-keyring.gpg] https://packages.broadcom.com/artifactory/vcf-
cli-debian noble main"| sudo tee /etc/apt/sources.list.d/vcf.list

sudo apt update
sudo apt install -y vcf-cli
```

To uninstall the CLI:

```
sudo apt remove vcf-cli
```

YUM or DNF (RHEL)

```
cat << EOF | sudo tee /etc/yum.repos.d/vcf-cli.repo
[vcf-cli]
name=vcf cli
baseurl=https://packages.broadcom.com/artifactory/vcfcli-rpm/
enabled=1
gpgcheck=1
#Optional - if you have GPG signing keys installed, use the below flags to verify the repository metadata sig-
nature:
gpgkey=https://packages.broadcom.com/artifactory/api/security/keypair/PackagesKey/public https://pack-
ages.broadcom.com/artifactory/vcfcli-rpm/tools/keys/BROADCOM-PACKAGING-GPG-RSA-KEY.pub
repo_gpgcheck=1
EOF

yum install -y vcf-cli
```

To upgrade the CLI:

```
sudo yum update -y vcf-cli # If upgrade doesn't work , try to clean cache and rebuild cache and try update
 command again.

sudo yum clean all

sudo yum makecache
```

To uninstall the CLI:

```
sudo yum remove vcf-cli
```

Check that the correct version of the CLI is properly installed.

```
vcf version
version: v9.0.0
...
```

Chocolatey (Windows)

```
choco install vcf-cli
```

To uninstall the CLI

```
choco uninstall vcf-cli
```

The **vcf-cli** package is part of the main Chocolatey Community Repository. When a new **vcf-cli** version is released, it may not be available immediately. If the above command fails, run:

```
choco install vcf-cli --version <VCF-CLI-VERSION>
```

Where **VCF-CLI-VERSION** is the VCF CLI version you want to install.

For example:

```
choco install vcf-cli --version 9.0.0
```

Proceed to Installing VCF CLI Plugins. If you plan to use VCF CLI commands in automation scripts, see Interactive Prompts before installing the plugins.

Installing VCF CLI Plugins

A VCF CLI *plugin* is an executable binary that packages a group of CLI commands. The core CLI includes a set of built-in command groups, and CLI plugins extend the CLI with additional command groups. For the list of built-in commands, see System Commands.

Specific plugins are relevant to different products and to different contexts that you connect the CLI to, based on the context type. Each product designates its relevant plugins as either standalone or context-scoped, as described in CLI Core, Plugins, and Plugin Installation.

Context-scoped plugins: When you create a VCF CLI context to connect to a product, context-scoped plugins install automatically if they are not already installed. As a backup, if plugins do not automatically install, you can install them by running **vcf plugin sync** with the CLI set to the desired context.

Standalone plugins: For some but not all products, you need to install standalone plugins by running **vcf plugin install** as described below. To find out whether you need to install standalone plugins for your product, see CLI Core, Plugins, and Plugin Installation.

After completing the steps in Installing VCF CLI From Binary Release or Installing VCF CLI Using Package Managers, follow the instructions below to install the standalone VCF CLI plugins. You can either install all plugins in a plugin group with a single command, or install each plugin individually.

Install all plugins in a group

To install all plugins in a plugin group:

1. Choose the plugin group by listing available plugin groups:
   ```
   vcf plugin group search
   ```
   The output lists plugin groups, resembling:
   ```
   vcf plugin group search
   GROUP                     DESCRIPTION                       LATEST
   vmware-vcfcli/essentials  Essential plugins for the VCF CLI  v9.0.0
   ```
2. In the output, look for the plugin group , such as **essentials** for essentials plugins of VCF CLI.
   - If you do not want to install the latest version listed, list all available versions for the plugin group:
     ```
     vcf plugin group search -n <PLUGIN-GROUP-NAME> --show-details
     ```
     Where **PLUGIN-GROUP-NAME** is the group listed in the previous step.
   - To see the list of plugins included in the plugin group, run:
     ```
     vcf plugin group get <PLUGIN-GROUP-NAME>:<PLUGIN-GROUP-VERSION>
     ```
     Where **PLUGIN-GROUP-NAME:PLUGIN-GROUP-VERSION** is the plugin group and group version for which you want to list the plugins. If you want to list the plugins for the latest version of the plugin group, you can omit **:PLUGIN-GROUP-VERSION** when running **vcf plugin group get**. For more information about how to list available plugin groups, see vcf plugin group.

3. Run one of the following:
   - If you want to install all plugins included in the latest version of the plugin group:
     ```
     vcf plugin install --group <PLUGIN-GROUP-NAME>
     ```
     For example:
     ```
     vcf plugin install --group vmware-vcfcli/essentials
     ```
     The CLI lists the installed version.
   - If you want to install all plugins included in a specific version of the plugin group:
     ```
     vcf plugin install --group vmware-vcfcli/essentials:v9.0.0
     ```
4. Verify that the plugins installed successfully by running:
   ```
   vcf plugin group get <PLUGIN-GROUP-NAME>:<PLUGIN-GROUP-VERSION>
   vcf plugin list
   ```
   Where **PLUGIN-GROUP-NAME:PLUGIN-GROUP-VERSION** is the plugin group and group version from which you installed the plugins. If you installed the plugins from the latest version of the plugin group, you can omit **:PLUGIN-GROUP-VERSION** when running **vcf plugin group get**. Each plugin name and plugin version returned by **vcf plugin group get** must be included in the output of **vcf plugin list**.

Install a single plugin

To install a single plugin:

1. Choose the plugin you want to install:
   ```
   vcf plugin search
   ```
   The output lists plugins, resembling:
   ```
   NAME               DESCRIPTION
    LATEST
   cluster            Kubernetes cluster operations
    v3.3.0
   imgpkg             package, distribute, and relocate your configuration and dependent oci images as
    v9.0.0
                        one oci artifact
   kubernetes-release  Kubernetes release operations
    v3.3.0
   ```
   If you want to list all available versions for the plugin, run:
   ```
   vcf plugin search -n <PLUGIN-NAME> --show-details
   ```
   Where **PLUGIN-NAME** is the plugin you chose above. For more information about this command and its options, see vcf plugin search.
2. Run one of the following:
   - If you want to install the latest version of the plugin:
     ```
     vcf plugin install <PLUGIN-NAME>
     ```
   - If you want to install a specific version of the plugin:
     ```
     vcf plugin install <PLUGIN-NAME> --version PLUGIN-VERSION
     ```
     For more information about **vcf plugin install** and its options, see vcf plugin install.
   - Verify that the plugin installed successfully by running:
     ```
     vcf plugin list
     ```

Updating the VCF CLI

Follow the instructions below to update your VCF CLI version.

Note:

If you want to retain an existing installation of the VCF CLI, move the CLI binary from **/usr/local/bin/vcf** to a different location before following the instructions below.

Update using a package manager

To update the VCF CLI version using a package manager:

1. Follow the instructions for your package manager below. This updates the CLI to the latest available version.
   - **Homebrew (MacOS):**
     ```
     brew update
     brew upgrade vcf-cli
     ```
   - **APT (Debian or Ubuntu):**
     ```
     sudo apt update
     sudo apt upgrade -y vcf-cli
     ```
   - **YUM or DNF (RHEL):**
     ```
     sudo yum update -y vcf-cli # If you are using DNF, run sudo dnf update -y vcf-cli.

     # If upgrade doesn't work , try to clean cache and rebuild cache and try update command again.

     yum clean all
     yum makecache
     ```
   - **Chocolatey (Windows):**
     ```
     choco upgrade vcf-cli
     ```
     The **vcf-cli** package is part of the main Chocolatey Community Repository. When a new **vcf-cli** version is released, it may not be available immediately. If the above command fails, install unapproved version with below command:
     ```
     choco upgrade vcf-cli --version <VCF-CLI-VERSION>
     ```
     Where **VCF-CLI-VERSION** is the version you want to update the CLI to.

     For example :
     ```
     choco upgrade vcf-cli --version 9.0.0
     ```
2. Check that the correct version of CLI is properly installed.
   ```
   vcf version
   version: v9.0.0
   ...
   ```

Update using a binary release

To update the VCF CLI version using a binary release:

- If you are updating the CLI version in an internet-connected environment, follow the instructions in Installing the VCF CLI in Internet Connected Environments above.
- If you are updating the CLI version in an internet-restricted environment, follow the instructions in Installing the VCF CLI in Internet Restricted Environments above.

Installing the VCF CLI in Internet Restricted Environments

You can install and use the VCF CLI in an internet-restricted environment, which includes the following steps

1. Download and Install the VCF CLI Binary
2. Download VCF CLI Plugins
3. Move the VCF CLI and CLI Plugins to Your Internet-Restricted Environment

Download and Install the VCF CLI Binary

To download and install the VCF CLI binary:

1. While connected to the internet, download the VCF CLI binary to a storage device, such as your computer, and then install it. To download and install the VCF CLI binary, follow the instructions from the page Install VCF CLI From Binary Release
2. Proceed to Download VCF CLI Plugins below.

Download VCF CLI Plugins

The VCF CLI allows you to install and run CLI plugins in internet-restricted environments with no direct connection to the internet. For information about VCF CLI plugins, see Installing VCF CLI Plugins above.

To enable the VCF CLI to install and update plugins in an internet-restricted environment, you must set up a private Docker-compatible container registry such as Harbor, Docker, or Artifactory.

Note: The VCF CLI supports uploading plugins to private registries that require authentication. However, your private registry must allow the CLI to pull the resulting images without authentication.

After the private registry is set up, the operator of the private registry can download the VCF CLI plugins required from the publicly available registry and move them to the private registry as follows:

1. Using the computer that has the VCF CLI installed, download the desired CLI plugins as a **tar.gz** file to your storage device.

   To discover which plugin groups and plugins are available for download, use the **vcf plugin group search** and **vcf plugin search** commands. For information about using these commands, see vcf plugin. After you identify the desired CLI plugins, you can use any of the options below to download them:
   - Download all plugins available in the default central repository:
     ```
     vcf plugin download-bundle --to-tar /tmp/FILE-NAME.tar.gz
     ```
   - Download all plugins included in the latest version of a specific plugin group. The command below uses the **vmware-vcfcli/essentials** plugin group as an example:
     ```
     vcf plugin download-bundle --group vmware-vcfcli/essentials --to-tar /tmp/FILE-NAME.tar.gz
     ```
   - Download all plugins included in a specific version of a plugin group, for example, **v9.0.0** of **vmware-vcfcli/essentials**:
     ```
     vcf plugin download-bundle --group vmware-vcfcli/essentials:v9.0.0 --to-tar /tmp/FILE-NAME.tar.gz
     ```
     You can also specify more than one plugin group version. For example:
     ```
     vcf plugin download-bundle --group vcfcli/essentials:v4.0.0,vcfcli/essentials:v9.0.0 --to-tar /tmp/FILE-NAME.tar.gz
     ```
     For more information about **vcf plugin download-bundle** and its options, see vcf plugin download-bundle.
2. Proceed to Move the VCF CLI and CLI Plugins to Your Internet-Restricted Environment below.

Move the VCF CLI and CLI Plugins to Your Internet-Restricted Environment

To configure the VCF CLI in your internet-restricted environment:

1. Move the storage device and if separate, the computer with the VCF CLI installed to your internet-restricted environment.
2. If the private registry requires authentication to upload images, run **docker login** or **crane auth login** to set up authentication with the registry.
3. If the private registry is configured with a self-signed CA certificate, add your certificate verification preferences by following the instructions in Adding Certificate Configuration for the Custom Registry below.

4. Upload the CLI plugins to the registry in the internet-restricted environment using the **vcf plugin upload-bundle** command. For example:
   ```
   vcf plugin upload-bundle --tar /tmp/FILE-NAME.tar.gz --to-repo registry.example.com/vcf_cli/plugins
   ```
   The **vcf plugin upload-bundle** command uploads the plugin bundle to the specified private repository and publishes the plugin inventory image under **plugin-inventory:latest**. In the example above, the plugin inventory image will be published to **registry.example.com/vcf_cli/plugins/plugin-inventory:latest**. If you already uploaded any plugins to the specified private repository, the command will keep them and append your new plugins from the plugin bundle to the existing ones.
5. Update the VCF CLI to point to the new plugin source:
   ```
   vcf plugin source update default --uri registry.example.com/vcf_cli/plugins/plugin-inventory:latest
   ```
   Note: Each VCF CLI user working within your organization will need to execute the **vcf plugin source update** command above in order to access and install or update the plugins that you make available in your internal registry.
6. Verify that the plugins are discoverable by running the **vcf plugin search** and **vcf plugin group search** commands.
7. Remove all pre-existing plugins before the final validation step:
   ```
   vcf plugin clean && vcf plugin list
   ```
8. Run **vcf plugin install --group <PLUGIN-GROUP-NAME>:<PLUGIN-GROUP-VERSION>** to test plugin installation from your new plugin source.
9. Run **vcf plugin list** to confirm the expected plugins have been installed.

Adding Certificate Configuration for the Custom Registry

If your custom registry is configured with a self-signed CA certificate, you must do one of the following:

- Provide the registry CA certificate to the VCF CLI
- Configure the VCF CLI to use HTTPS connections and skip server certificate verification by activating the **skip-cert-verify** option
- Allow HTTP connections to the registry and skip server certificate verification by activating the **--insecure** option

If your registry CA certificate is expired, use the **--skip-cert-verify** or **--insecure** option.

To configure your certificate verification preferences for the registry CA certificate:

- To add the registry CA certificate to the VCF CLI, run the **vcf config cert add** command with the **--ca-certificate** option. For example:
  ```
  vcf config cert add --host test.registry.com --ca-certificate PATH-TO-CA-CERT
  ```
  If the registry is serving on a non-default port, add the port number. For example:
  ```
  vcf config cert add --host test.registry.com:8443 --ca-certificate PATH-TO-CA-CERT
  ```
- To skip certificate verification, run the **vcf config cert add** command with the **skip-cert-verify** option. For example:
  ```
  vcf config cert add --host test.registry.com --skip-cert-verify true
  ```
- To allow HTTP connections to the registry and skip certificate verification, run the **vcf config cert add** command with the **--insecure** option. For example:
  ```
  vcf config cert add --host test.registry.com  --insecure true
  ```

You can update or delete your certificate configuration using the **vcf config cert update** and **vcf config cert delete** commands. For more information, see vcf config cert.

Adding a Proxy CA certificate

If you configured a proxy between the VCF CLI and the central repository and the proxy uses a self-signed CA certificate, set the **PROXY_CA_CERT** environment variable to the Base64 value of the proxy CA certificate.

VCF CLI Context, Architecture, and Configuration

This topic explains the VCF CLI context, its architecture and how to update your VCF CLI configuration.

VCF CLI Context

What is a Context in VCF CLI?

- **A Configuration Set:** A context in VCF CLI essentially stores a specific set of configurations related to a particular VCF environment. This environment could be a single Kubernetes cluster, vSphere Supervisor Cluster, or the VCF Automation.
- **Key Elements:** A context typically includes things like:
  - **Endpoint:** The URL or IP address of the Kubernetes API server or the VCF Automation you connect to.
  - **Context Type:** The nature of the endpoint. It can be Kubernetes or Cloud-Consumption-Interface.
  - **PATH**: Path to the Kubeconfig file.
  - **Credentials:** Authentication information (username, token, and so on) to access the endpoint.

Why Use Contexts?

- **Multi-Environment Work:** Let's say you're managing multiple vSphere Supervisor clusters for different development teams or projects. You could set up a separate context for each cluster, making it easy to switch between them without having to manually change your configuration each time.
- **Simplified Development:** You might have a context for your local development cluster and another for your staging or production environment. Contexts let you seamlessly move between these environments without needing to constantly update your configuration.

How to Manage Contexts:

- **Creating Contexts:** You can create new contexts using the **vcf config context create** command. You'll need to provide the necessary configuration details (cluster endpoint, credentials, etc.).
- **Listing Contexts:** The **vcf config context list** command will show you the available contexts.
- **Switching Contexts:** Use **vcf config context use [context-name]** to switch to a specific context.
- **Deleting Contexts:** You can delete a context with **vcf config context delete [context-name]**.

For more details on how to create contexts , see Create VCF Automation Context and Create vSphere Supervisor Context.

Create VCF Automation Context

You can create VCF CLI contexts with VCF Automation endpoint to manage VCF Automation resources (regions, zones,etc.) or Kubernetes cluster resources (pods,secrets,etc.).

Attention:

The commands use "CCI" to represent the VCFA context type

```
vcf context create <context_name> --endpoint <VCFA_ENDPOINT> --api-token <API-TOKEN> --type cci --ca-certifi-
cate <VCFA_CERT_PATH>
```

Note:

When you create context with a VCFA endpoint, a primary VCFA context is created. However, if the user has permissions to access namespaces within VCF, additional contexts are created for each of those namespaces.

For example , below contexts will be created if a user has created a VCFA context with name 'vcfa_ctx_name' and has access to 'namespace1' namespace.

```
vcfa_ctx_name
vcfa_ctx_name:namespace1:project1
```

Context 'vcfa_ctx_name' is a VCFA context and can be used to manage VCFA resources like projects , regions etc.

Context 'vcfa_ctx_name:namespace1:project1' is a namespace context and can be used to manage resources like virtual machine etc.

Create context for Kubernetes Cluster :

In this section we will see how to create VCF CLI context for a kubernetes cluster deployed in a namespace in VCF Automation.

1. Use VCF Automation Namespace Context created above.
   ```
   vcf context use vcfa_ctx_name:namesapce1:project1
   ```
2. Get Kubernetes cluster name using kubectl.
   ```
   kubectl get cluster
   ```
3. Install cluster plugin
   ```
   vcf plugin install cluster
   ```
4. Register a VCFA JWT authenticator using the vcf cluster plugin before creating your context. This is a one-time setup process.
   ```
   vcf cluster register-vcfa-jwt-authenticator $KUBERNETES_CLUSTER_NAME
   ```
5. Get Kubeconfig for the cluster using below command.
   ```
   vcf cluster kubeconfig get $KUBERNETES_CLUSTER_NAME --export-file <KUBECONFIG_FILE>
   ```
6. Create CLI context using the above generated kubeconfig
   ```
   vcf context create <CLI_CONTEXT_NAME> --kubeconfig <KUBECONFIG_FILE> --kubecontext <KUBE_CONTEXT_NAME>
   ```
7. Execute **kubectl** commands by using the **--kubeconfig** flag followed by the path to **KUBECONFIG_FILE**
   ```
   kubectl --kubeconfig=<KUBECONFIG_FILE> get ns
   ```

Create vSphere Supervisor Context

Create vSphere Supervisor Context:

You can create VCF CLI context for vSphere Supervisor using username/password or OIDC. In order to use OIDC based authentication , an external Identity provider must be configured in the Supervisor.

```
vcf context create <context_name> --endpoint <SUPERVISOR_ENDPOINT> --insecure-skip-tls-verify --type k8s
```

Note:

When you create context with a vSphere Supervisor Endpoint, a primary supervisor context is created. However, if the user has permissions to access to Supervisor namespaces , additional contexts are created for each of those namespaces.

For example , below contexts will be created if a user has created a vSphere Supervisor context with name 'vc_ctx_name' and has access to 'namespace1' namespace.

```
vc_ctx_name
vc_ctx_name:namespace1
```

These contexts can be used to manage resources like virtual machine, cluster etc.

Create Context for Kubernetes Cluster:

You can create CLI context for Kubernetes cluster using username/password or by using 'cluster' plugin as described below:

- **With username/Password**
  ```
  vcf context create <context_name> --endpoint <SUPERVISOR_ENDPOINT> --workload-cluster-name <WORK-
  LOAD_CLUSTER_NAME> --workload-cluster-namespace <WORKLOAD_CLUSTER_NAMESPACE> -u <USER> --insecure-skip-tls-
  verify
  ```
  Note:

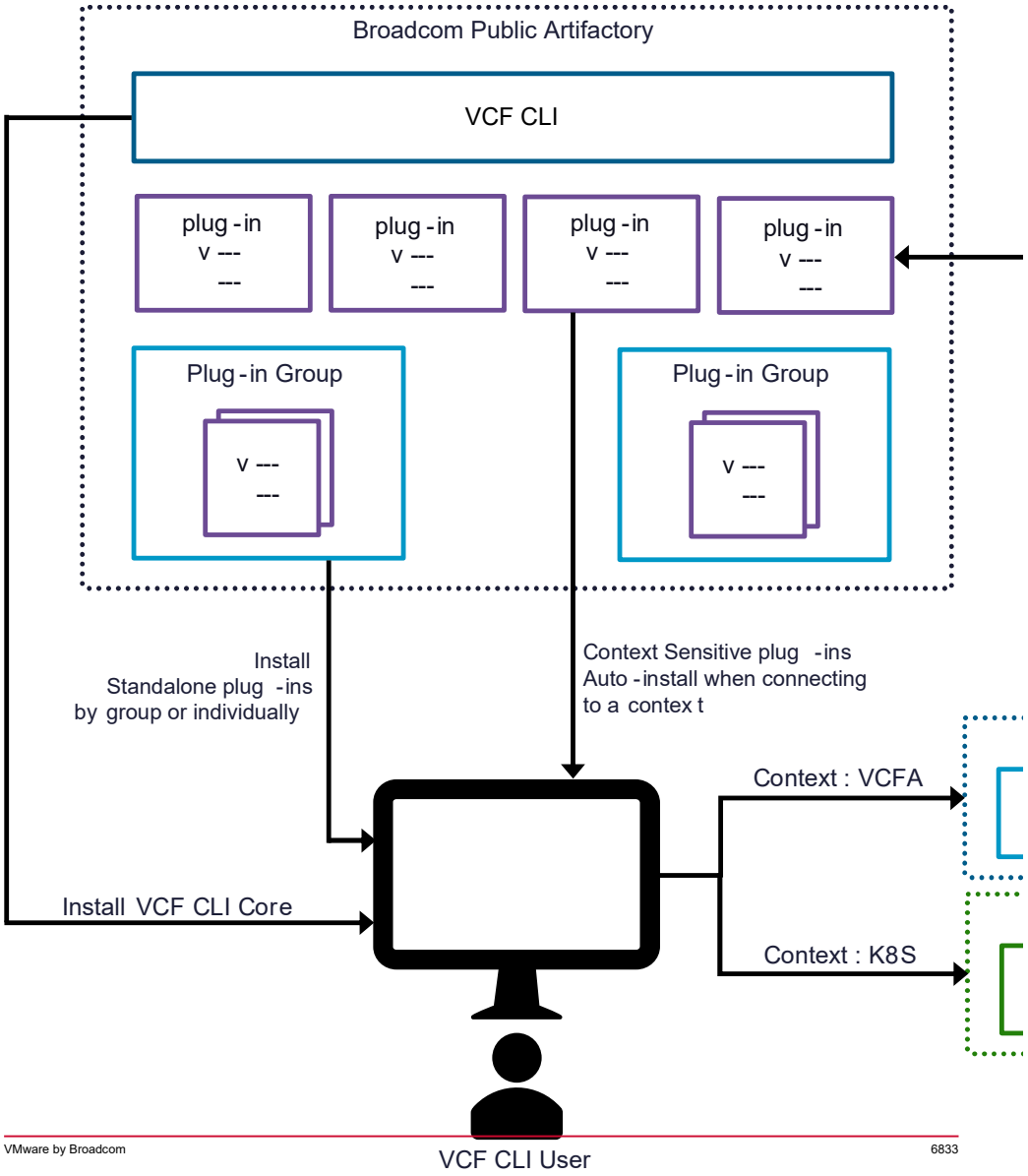
  The username and password must be those for the vCenter SSO user.
- **With VCF cluster plugin**
  This method requires that the environment be IDP enabled. The user must also be granted the appropriate permissions to the target namespace. See Configure vSphere Namespace Permissions for External Identity Provider Users and Groups.
  a. Export the workload cluster kubeconfig
     ```
     vcf cluster kubeconfig get <CLUSTER_NAME> [flags]
     ```
  b. Create the context with the exported kubeconfig.
     ```
     vcf context create vks --kubeconfig <VKSKUBECONFIG> --kubecontext <CONTEXT_NAME> --type k8s
     ```

For more details on cluster plugin , refer the vcf cluster command reference page.

VCF CLI Architecture

The VCF CLI uses a context-scoped, pluggable architecture that lets you install plugins that provide different sets of commands to use with different server endpoints.

Figure 250:



CLI Core, Plugins, and Plugin Installation

The core CLI has some command groups built in, and CLI *plugins* extend the CLI with additional command groups that are useful for particular products. Each plugin is an executable binary that packages a group of CLI commands.

Most plugins need to be installed with the **vcf plugin install** command, either in groups or individually:

- **Plugin groups**: To install all the plugins typically needed for a facet of VCF with a single command, include the **--group** option.
  - Run **vcf plugin group search** to see available groups, or see Plugins and Plugin Groups below.
- **Individual plugins**: Without the **--groups** option, **vcf plugin install** installs CLI plugins individually.

Context-scoped plugins install themselves automatically when you switch the CLI to use a context that they apply to. Many plugins for vSphere Supervisor, the **K8S** context type, are context-scoped, and so their commands are available without you having to run **vcf plugin install**.

Context Types

The VCF CLI connects to Kubernetes clusters and VCF Automation. Depending on what the CLI is connected to as a client, it changes the commands that it supports and what those commands do.

VCF CLI uses **context** to represent the server side it connects to.

- **Context** identifies the specific server that the VCF CLI connects to. You create these with **vcf context create** and log in to them with **vcf context use**.
  - Each context has one of the following context types:
    - **kubernetes** or **k8s** for Kubernetes clusters
    - **cci** for VCF Automation
- You pass the context type to the **-t** or **--type** flag of **vcf context create** when you create a context.

Plugins and Plugin Groups

Command functionality comes from either the core CLI or from CLI plugins. Each plugin is associated with a command group, which is a set of available commands that all start with the same words, for example **vcf imgpkg**.

- **Source**: Where the commands come from, whether core CLI or a CLI plugin.
- **Context type**: The type of server that the plugin commands typically apply to: **k8s**, or **cci**.
  - You pass this to the **-t** or **--type** flag of **vcf context create** when you create a context.
- **Plugin groups (or context-scoped)**: The plugin groups, if any, that install the plugin, or else “context-scoped” for plugins that install automatically.
  - You pass a plugin group to the **--group** flag of **vcf plugin install** when you install a group of plugins at once. VCF CLI currently supports only one plugin group listed below:
    - **vmware-vcfcli/essentials**

VCF CLI Configuration

The VCF CLI configuration file **$USER_HOME/.config/vcf/config.yaml** contains your VCF CLI configuration, including

- Sources for CLI plugin discovery
- Global and plugin-specific configuration options, or *features*
- Names, contexts, and **kubeconfig** locations for the servers that the CLI knows about, and which is the current one

You can use the **vcf config set PATH VALUE** and **vcf config unset PATH** commands to customize your CLI configuration, as described in the table below. Running these commands updates the **~/.config/vcf/config.yaml** file.

Note:

For Windows users , the VCF CLI configuration file is located at `C:\Users\<USER>\.config\vcf\config.yaml`.

Path	Value	Description
env.VARIABLE	Your variable value; for example, **VCF_CLI_LOG_LEVEL**	This path sets or unsets global environment variables for the VCF CLI. For example, **vcf config set env.VCF_CLI_LOG_LEVEL Yes**. Variables set by running **vcf config set** persist until you unset them with **vcf config unset**.
features.PLUGIN.FEATURE	true or false	This path activates or deactivates plugin-specific features in your CLI configuration. Use only if you want to change or restore the defaults; some of these features are experimental and intended for evaluation and test purposes only. For example, running **vcf config set features.cluster.dual-stack-ipv4-primary true** sets the dual-stack-ipv4-primary feature of the cluster CLI plugin to **true**. By default, only production-ready plugin features are set to **true** in the CLI.

Activating and Deactivating Features

To activate a plugin feature, run

```
vcf config set features.PLUGIN.FEATURE true
```

Where:

- **PLUGIN** is the name of the CLI plugin. For example, **cluster** or **package**.
- **FEATURE** is the name of the feature that you want to activate.

To deactivate a plugin feature, run:

```
vcf config set features.PLUGIN.FEATURE false
```

Where:

- **PLUGIN** is the name of the CLI plugin. For example, **cluster** or **package**.
- **FEATURE** is the name of the feature that you want to deactivate.

Environment Variable

Some options affecting the CLI are only available through the use of environment variables. These variables can be set in the shell or using **vcf config set env.<VAR> <VALUE>**. Below table highlights the list of such environment variables.

Variable	Description
ALLOWED_REGISTRY	If set, these plugin registries will be considered as trusted registries
VCF_ACTIVE_HELP	ActiveHelp are messages printed through auto-completions as the program is being used. Once auto-completion has been set up,the VCF CLI automatically provides ActiveHelp messages to improve the user-experience when the user presses `TAB`. To deactivate all ActiveHelp messages you can set the `VCF_ACTIVE_HELP` environment variable to `0`.

Variable	Description
VCF_CLI_AUTHENTICATED_REGISTRY	Provides a comma separated list of registry hosts that requires authentication// to pull images. VCF CLI will use default docker auth to communicate to these registries
VCF_CLI_ESSENTIALS_PLUGIN_GROUP_NAME	Used to override and customize the default essentials plugin group name. The "essentials" plugin group provides core functionality for the CLI and is automatically installed when you initialize the VCF CLI for the first time.
VCF_CLI_ESSENTIALS_PLUGIN_GROUP_VERSION	Used to override and customize what version of essentials plugin group must be installed
VCF_CLI_LOG_LEVEL	Used to increase the amount of logging during troubleshooting. This variable is only respected by the CLI core commands as of now. This can take values as (1, 2, 3, ..., 9) with 9 being the maximum LOG LEVEL.
VCF_CLI_PLUGIN_DB_CACHE_REFRESH_THRESHOLD_SECONDS	Overrides the default threshold at which point the plugin inventory will be automatically refreshed. Default: 24 hours. For testing , you can modify the default behavior and set it to a custom value.
VCF_CLI_PLUGIN_DB_CACHE_TTL_SECONDS	Overrides the default 30 minute delay in which the plugin inventory cache is used without checking if it must be refreshed. For testing , you can modify the default behavior and set it to a custom value.
VCF_CLI_PLUGIN_DISCOVERY_IMAGE_SIGNATURE_PUBLIC_KEY_PATH	A custom public key path can be provided to configure the cosign verifier. This overrides the plugin inventory's verification key, but must rarely be necessary. It's primarily used in the event of a change in signature keys, which will be clearly documented.
VCF_CLI_PLUGIN_DISCOVERY_IMAGE_SIGNATURE_VERIFICATION_SKIP_LIST	Comma separated list of discovery image urls whose authentication needs to be skipped
VCF_CLI_PRIVATE_PLUGIN_DISCOVERY_IMAGES	Enables use of private plugin repositories in addition to the official central plugin inventory repository. You can set the value as shown below. export VCF_CLI_PRIVATE_PLUGIN_DISCOVERY_IMAGES="registry2.private.bro vcf-cli/plugins/plugin-inventory:latest"
VCF_CLI_RECOMMEND_VERSION_DELAY_DAYS	Change the default delay (24 hours) between notifications that a new CLI version is available for upgrade
VCF_CLI_SHOW_PLUGIN_INSTALLATION_LOGS	Enable or disable the logs for plugin group installation process. Possible values are "True" or "False". Set to "True" to enable the logs.
VCF_CLI_SHOW_TELEMETRY_CONSOLE_LOGS	Enable or disable the logs for telemetry collection. Set to "True" to enable the logs and "False" to disable.
VCF_CLI_SKIP_CONTEXT_RECOMMENDED_PLUGIN_INSTALLATION	To skip the auto-installation of the context recommended plugins
VCF_CLI_SUPPRESS_SKIP_SIGNATURE_VERIFICATION_WARNING	Variable to suppress logging when plugin inventory image verification is skipped
VCF_CLI_VCFA_API_TOKEN	Allows users to set the VCFA api token as environment variable
VCF_CLI_VSPHERE_PASSWORD	Allows users to set the vSphere password as environment variable
HTTPS_PROXY	This environment variable specifies a proxy server that should handle outgoing HTTPS requests. If set, the http.Client will use the provided proxy server unless explicitly instructed otherwise.

Variable	Description
HTTP_PROXY	This environment variable specifies a proxy server that should handle outgoing HTTP requests. If set, the http.Client will use the provided proxy server unless explicitly instructed otherwise.
NO_PROXY	This environment variable defines a list of domains, IP addresses, or subnets for which the proxy should be bypassed. This is useful for ensuring specific requests don't go through the proxy, potentially for performance, security, or other reasons.

Command Reference

This VCF CLI command reference is intended for the VCF CLI v9.0.x.

VCF CLI provides below commands through built-in system & plugin based commands

- **System commands**
  - completion
  - config
  - context
  - plugin
  - version
- **Plugin based commands**
  - cluster
  - imgpkg
  - kubernetes-release
  - namespaces
  - package
  - pais
  - registry-secret
  - secret
  - telemetry
  - vm

System Commands

The VCF CLI provides the following system commands. These commands are installed by default with the VCF CLI.

- completion
- config
- context
- plugin
- version

completion

Output shell completion code for the specified shell (bash, zsh, fish, powershell).

Usage

System command: `completion` | **Primarily used for:** VCF CLI

Examples

To enable auto-completion for Bash:

Note: Install the **bash-completion** OS package before following the steps below.

- Load only for the current session:
  ```
  source <(vcf completion bash)
  ```
- Load for all new sessions:
  ```
  vcf completion bash > $HOME/.config/vcf/completion.bash.inc printf "\n# VCF CLI shell completion\nsource
   '$HOME/.config/vcf/completion.bash.inc'\n" >> $HOME/.bashrc
  ```

To enable auto-completion for Zsh:

- Load only for the current session:
  ```
  autoload -U compinit; compinit
  source <(vcf completion zsh)
  ```
- Load for all new sessions:
  ```
  echo "autoload -U compinit; compinit" >> $HOME/.zshrc
  vcf completion zsh > "${fpath[1]}/_vcf"
  ```

To enable auto-completion for Fish:

- Load only for the current session:
  ```
  vcf completion fish | source
  ```
- Load for all new sessions:
  ```
  vcf completion fish > $HOME/.config/fish/completions/vcf.fish
  ```

To enable auto-completion for Powershell:

- Load only for the current session:
  ```
  vcf completion powershell | Out-String | Invoke-Expression
  ```
- Load for all new sessions:
  ```
  printf "\n# VCF CLI shell completion\nvcf completion powershell | Out-String | Invoke-Expression" >>
   $PROFILE
  ```

Flags

`-h, --help`

Help text

Global Flags

`--verbose int32`

Sets the log level between 0 to 9 for this command invocation

Active Help

The VCF CLI prints ActiveHelp messages for auto-completions when you press **TAB**. This feature is currently supported only for **bash** and **zsh**.

To deactivate all ActiveHelp messages, you can set the **VCF_ACTIVE_HELP** environment variable to **0**

config

Configures VCF CLI

Usage

System command: `config` | **Primarily used for:** VCF CLI

Syntax

```
vcf config [COMMAND]
```

Flags

`-h, --help`

Help text

Global Flags

`--verbose int32`

Sets the log level between 0 to 9 for this command invocation with 9 being the most verbose.

vcf config cert

Manages custom certificate configuration for a given host. The host can be VCFA endpoint or vSphere Supervisor endpoint.

Available commands

- vcf config cert add
- vcf config cert delete
- vcf config cert list
- vcf config cert update

vcf config cert add

Configures the CLI to add a custom certificate to the list of the CLI trusted certificates for a given host.

Usage

```
vcf config cert add [FLAGS]
```

Examples

To add a CA certificate for test.broadcom.com

```
vcf config cert add test.broadcom.com --ca-cert PATH-TO-CERTIFICATE
```

To add a CA certificate for test.broadcom.com:8443

```
vcf config cert add --host test.broadcom.com:8443 --ca-cert PATH-TO-CERTIFICATE
```

To skip certificate verification for test.broadcom.com

```
vcf config cert add --host test.broadcom.com --skip-cert-verify true
```

Flags

`--ca-cert`

The path to the CA certificate

`-h, --help`

Help text

`--host`

The host address or the host address and port number. Its a mandatory parameter.

`--insecure`

Allows the use of HTTP when communicating with the host. Defaults to false

`--skip-cert-verify`

Skips the server's TLS certificate verification. Defaults to false

vcf config cert delete

Deletes a custom certificate configuration

Usage

```
vcf config cert delete [HOST] [FLAGS]
```

Example

To delete the custom certificate for test.broadcom.com:

```
vcf config cert delete test.broadcom.com
```

Flags

`-h, --help`

Help text

vcf config cert list

Lists available certificate configurations

Usage

```
vcf config cert list [FLAGS]
```

Example

To list available certificate configurations in yaml format

```
vcf config cert list -o yaml
```

Flags

`-h, --help`

Help text

`-o, --output`

Output format. Supported values are yaml, json, and table

vcf config cert update

Updates a custom certificate configuration

Usage

```
vcf config update [HOST] [FLAGS]
```

Examples

To update the CA certificate for test.broadcom.com

```
vcf config cert update test.broadcom.com --ca-cert PATH-TO-CERTIFICATE
```

To update the CA certificate for test.broadcom.com:5443

```
vcf config cert update test.broadcom.com:5443 --ca-cert PATH-TO-CERTIFICATE
```

To skip certificate verification for test.broadcom.com

```
vcf config cert update test.broadcom.com --skip-cert-verify true
```

Flags

--ca-cert

The path to the CA certificate

-h, --help

Help text

--insecure

Allows the use of HTTP when communicating with the host. Supported values are true and false

--skip-cert-verify

Skips the server's TLS certificate verification. Supported values are true and false

vcf config get

Get the current configuration

Usage

```
vcf config get [FLAGS]
```

Example

To get the current VCF CLI configuration

```
vcf config get
```

Flags

-h, --help

Help text

vcf config init

Initializes VCF CLI configuration files when run for the first time on new installation. Initializes config with defaults including plugin specific defaults such as default feature flags for all active and installed plugins.

Usage

```
vcf config init [FLAGS]
```

Example

To initialize config with defaults

```
vcf config init
```

Flags

-h, --help

Help text

vcf config set

Sets the value for the specified path. Supported paths are **features.global.FEATURE**, **features.PLUGIN.FEATURE**, and **env.VARIABLE**. For more information, see VCF CLI Architecture and Configuration

Usage

```
vcf config set PATH VALUE [FLAGS]
```

Note:

The command allows setting **VALUE** to an empty string.

Enables a specific plugin feature

```
vcf config set features.<PLUGIN_NAME>.<PLUGIN_FEATURE> <true|false>
```

Where PLUGIN_NAME is the name of the plugin and PLUGIN_FEATURE is the name of the plugin feature.

Example

Sets a custom CA cert for a proxy that requires it

```
vcf config set env.PROXY_CA_CERT b329baa034afn3.....
```

Enables a general CLI feature

```
vcf config set features.global.abcd true
```

Flags

-h, --help

Help text

vcf config unset

Unsets configuration values at the specified path. Supported paths are **features.global.FEATURE**, **features.PLUGIN.FEATURE**, and **env.VARIABLE**

Usage

```
vcf config unset PATH [FLAGS]
```

Examples

To unset **PROXY_CA_CERT** in the configuration of the CLI

```
vcf config unset env.PROXY_CA_CERT
```

Flags

-h, --help

Help text

context

Configure and manage contexts for the VCF CLI. A VCF CLI context is a combination of a user identity and a server identity. Each context is associated with a context type: **kubernetes** or **k8s, cci.** CLI supports various context creation workflows which can be broadly classified into two categories as described below. For more information about contexts and the different types supported, check VCF CLI Context.

Usage

System command: `context` **| Primarily used for:** VCF CLI

Syntax:

```
vcf context [COMMAND]
```

Flags

-h, --help

Help text

Global Flags

--verbose int32

Sets the log level between 0 to 9 for this command invocation

Available commands

- vcf context create
- vcf context delete
- vcf context get
- vcf context list
- vcf context refresh
- vcf context unset
- vcf context use

vcf context create

Create a VCF CLI context. Users can create contexts for VCFA endpoint or vSphere Supervisor endpoint. For more details on context , check VCF CLI Context.

Syntax:

```
vcf context create CONTEXT_NAME [flags]
```

Examples:

Note:

Users have two options to create a **K8S cluster context**. They can choose the control plane option by providing 'endpoint', or use the kubeconfig for the cluster by providing 'kubeconfig' and 'kubecontext'. If only '--kubecontext' is set and '--kubeconfig' is not, the $KUBECONFIG env variable will be used and, if the $KUBECONFIG env is also not set, the default kubeconfig file ($HOME/.kube/config) will be used

Create a vSphere Supervisor context using endpoint and type (--type is not mandatory)

```
vcf context create mgmt-cluster --endpoint https://k8s.example.com[:port] --type k8s
```

Create a vSphere Supervisor context using kubeconfig path and context

```
vcf context create mgmt-cluster --kubeconfig path/to/kubeconfig --kubecontext kubecontext --type k8s
```

Create a VCFA context using the provided CA Bundle for TLS verification. Note: 'cluster kubeconfig get' needs a context with --ca-certificate.

```
vcf context create vcfa-context --endpoint https://vcfa.endpoint[:port] --ca-certificate /path/to/ca/ca-cert
 --type cci --api-token <API-TOKEN>
```


Create a VCFA context by skipping TLS verification, which is insecure

```
vcf context create vcfa-context --endpoint https://vcfa.endpoint[:port] --insecure-skip-tls-verify --type cci
 --api-token <API-TOKEN>
```

Note:

The VCFA ("cci") context type is being released to provide advance support for the development and release of new services (and CLI plugins) which extend and combine features provided by individual cci components.

Flags

--api-token string

API token generated from the VCFA Endpoint. Only applicable for 'cci' context type

-auth-type string

Auth type to be used for context creation. It can be 'oidc' or 'basic' or empty for auto detection

--ca-certificate string

Path to the endpoint public certificate file

-e, --endpoint string

Endpoint URL

-h, --help

Help text

--insecure-skip-tls-verify

Skip TLS certificate verification for endpoint

--kubeconfig string

Path to the kubeconfig file. If not provided, defaults to KUBECONFIG environment variable; valid only if user doesn't choose 'Endpoint' option

--kubecontext string

The context in the kubeconfig to use; valid only if user doesn't choose 'Endpoint' option

--set-current

Set the context as current context and download all required plugins

--skip-create-related-contexts

Skip the context creation for namespaces

--tenant-name string

Tenant Name. Only applicable for 'cci' context type

-t, --type string

Type of context to create (kubernetes[k8s]|cloud-consumption-interface[cci]). This is not a mandatory flag

-u, --username string

Username for login. Not required to be passed when OIDC is configured. Only applicable for 'kubernetes' context type

--workload-cluster-name string

Name of the workload cluster. Only applicable for 'K8S' context type

--workload-cluster-namespace string

Namespace of the workload cluster. Only applicable for 'K8S' context type

vcf context delete

Delete a context from the config

Syntax:

```
vcf context delete CONTEXT_NAME [flags]
```

Flags

-h, --help

Help text

-skip-delete-kubeconfig-context

Delete the VCF CLI context but do not delete the kube-context from the kubeconfig file

-y, --yes

Delete the context entry and any associated supervisor namespace or workload cluster context entry without confirmation

vcf context get

Display a context from the config

Syntax:

```
vcf context get CONTEXT_NAME [flags]
```

Flags

-h, --help

Help text

o, --output string

output format: yaml|json (default "yaml")

vcf context list

List contexts

Syntax:

```
vcf context list [flags]
```

Flags

--current

list only current active context

-h, --help

Help text

o, --output string

output format: yaml|json (default "yaml")

-t, --type string

list only contexts associated with the specified context-type (kubernetes[k8s]|cloud-consumption-interface[cci])

--wide

display additional columns for the contexts

vcf context refresh

Re-authenticate the user for the given context, if the existing token/credentials is expired

Syntax:

```
vcf context refresh CONTEXT_NAME [flags]
```

Flags

--ca-certificate string

Path to the endpoint certificate

-h, --help

Help text

vcf context unset

Unset the given context, only if it is active, so that it is not used by default

Syntax:

```
vcf context unset CONTEXT_NAME [flags]
```

Flags

-h, --help

Help text

vcf context use

Set the context to be used by default

Syntax:

```
vcf context use CONTEXT_NAME [flags]
```

Flags

--ca-certificate string

Path to the endpoint public certificate

-h, --help

Help text

--insecure-skip-tls-verify

Skip TLS certificate verification for endpoint

plugin

Manages VCF CLI plugins

Usage

System command: `plugin` **| Primarily used for:** VCF CLI

Syntax

```
vcf plugin [command]
```

Flags

-h, --help

Help text

Global Flags

--verbose int32

Sets the log level between 0 to 9 for this command invocation.

vcf plugin clean

Usage

```
vcf plugin clean [FLAGS]
```

Example

To remove existing CLI plugins from your machine:

vcf plugin clean

Flags

-h, --help

Help text

vcf plugin describe

Describes the specified plugin

Usage

```
vcf plugin decribe <PLUGIN-NAME> [FLAGS]
```

Example

- To describe *my-plugin*
  vcf plugin describe my-plugin

Flags

-h, --help

Help text

-o, --output string

Output format (yaml|json|table)

vcf plugin download-bundle

Downloads a plugin bundle to a local **tar** file, including the standalone plugins in the specified group along with all context-sensitive plugins that are used for the same product

Usage

```
vcf plugin download-bundle [FLAGS]
```

Example

To download all plugins associated with **v1.0.0** of a plugin group named **vmware-vcfcli** from the default plugin discovery source

```
vcf plugin download-bundle --group vmware-vcfcli/default:v1.0.0 --to-tar /tmp/bundle_vmware_vcf-cli_v1.0.0.tar.gz
```

To download the latest version of a plugin repository from a custom plugin discovery source

```
vcf plugin download-bundle --image custom.registry.broadcom.com/vcf/-cli/plugins/plugin-inventory:latest --to-tar /tmp/bundle_complete.tar.gz
```

Flags

-h, --help

help for download-bundle

--group

The plugin group and plugin group version to download the plugin bundle from

--image

The URI of the plugin discovery source. Defaults to *projects.packages.broadcom.com/vcf-cli/plugins/plugin-inventory:latest*

--plugin

Only download plugins matching specified pluginID. Format: name/name:version. Can specify multiple plugins

--refresh-configuration-only

Only refresh the central configuration data

--strict-version-check

only download plugins matching the exact specified name and version in the 'plugin' flags

--to-tar

The local path to which you want to download the plugin bundle

vcf plugin group

Manages plugin groups

Available commands

- vcf plugin group get
- vcf plugin group search

vcf plugin group get

Gets the list of plugins included in the specified plugin group

Usage

```
vcf plugin group get GROUP-NAME [FLAGS]
```

Example

Note:

You can specify the plugin group version in any of the following formats: **vMAJOR**, **vMAJOR.MINOR**, or **vMAJOR.MINOR.PATCH**. For example, **vmware-vcfcli/default:v1**, **vmware-vcfcli/default:v1.0**, or **vmware-vcfcli/default:v1.0.0**

- To list plugins in *vmware-vcfcli/default:v1.0.0*
  vcf plugin group get vmware-vcfcli/default:v1.0.0

Flags

--all

Lists all plugins in the plugin group, including context-scoped ones

-h, --help

Help text

-o, --output

Output format. Supported values are **yaml**, **json**, and **table**

vcf plugin group search

Lists available plugin groups from the plugin source.

Usage

```
vcf plugin group search [FLAGS]
```

Example

Note:

You can specify the plugin group version in any of the following formats: **vMAJOR**, **vMAJOR.MINOR**, or **vMAJOR.MINOR.PATCH**. For example, **vmware-vcfcli/default:v1**, **vmware-vcfcli/default:v1.0**, or **vmware-vcfcli/default:v1.0.0**

To list all available plugin groups

```
vcf plugin group search
```

Flags

-h, --help

Help text

-n, --name

Limits the search to the plugin group that you specify with the **-n** flag

-o, --output

Output format. Supported values are **yaml**, **json**, and **table**

--show-details

Shows the details of the specified plugin group, including all available versions

vcf plugin install

Installs the specified plugin or plugins. For more information, see VCF CLI Architecture and Configuration

Usage

```
vcf plugin install <PLUGIN-NAME> [FLAGS]
```

Example

To install the latest version of *my-plugin*

```
vcf plugin install my-plugin
```

To install all plugins included in the latest version of the *vmware-vcfcli/default* plugin group

```
vcf plugin install --group vmware-vcfcli/default
```

To install all plugins included in *v2.3.0* of the *vmware-vcfcli/default* plugin group

```
vcf plugin install --group vmware-vcfcli/default:v2.3.0
```

Flags

--group

The plugin group that you are targeting. You can specify the plugin group version in any of the following formats: **vMAJOR**, **vMAJOR.MINOR**, or **vMAJOR.MINOR.PATCH**. For example, **vmware-vcfcli/default:v1**, **vmware-vcfcli/default:v1.0**, or **vmware-vcfcli/default:v1.0.0**.

-h, --help

Help text

-v, --version

The version of the plugin. Defaults to **latest**. You can specify the plugin version in any of the following formats: **vMAJOR**, **vMAJOR.MINOR**, or **vMAJOR.MINOR.PATCH**. For example, **v1**, **v1.9**, or **v1.9.1**

vcf plugin list

Lists installed plugins. If a plugin required by an existing context is not installed on your machine, the command also prints the name of the missing plugin

Usage

```
vcf plugin list [FLAGS]
```

Example

To list installed plugins

vcf plugin list

Flags

-h, --help

Help text

-o, --output

Output format. Supported values are **yaml**, **json**, and **table**

vcf plugin search

List all the plugins from plugin inventory.

Usage

```
vcf plugin search [FLAGS]
```

Example

To list all available plugins

```
vcf plugin search
```

To list a specific plugin

```
vcf plugin search -n cluster
```

Flags

-h, --help

Help text

-n, --name

Limits the search to the plugin that you specify with the **-n** flag

-o, --output

Output format. Supported values are **yaml**, **json**, and **table**

--show-details

Shows the details of the specified plugin, including all available versions

vcf plugin source

Manages plugin discovery sources. A discovery source provides metadata about available plugins, their supported versions, and how to download them.

Available commands

- vcf plugin source init
- vcf plugin source list
- vcf plugin source update

vcf plugin source init

Initializes the plugin discovery source to its default value

Usage

```
vcf plugin source init [FLAGS]
```

Example

To initialize the plugin discovery source to its default value

```
vcf plugin source init
```

Flags

-h, --help

Help text

vcf plugin source list

Lists available discovery sources

Usage

```
vcf plugin source list [FLAGS]
```

Example

To list available discovery sources

```
vcf plugin source list
```

Flags

-h, --help

Help text

-o, --output

Output format. Supported values are **yaml**, **json**, and **table**

vcf plugin source update

Updates the specified discovery source

Usage

```
vcf plugin source update SOURCE-NAME [FLAGS]
```

Example

To update a discovery source, for example, **my-oci**

```
vcf plugin source update default --uri registry.broadcom.com/vcf/plugins/plugin-inventory:latest
```

Flags

-h, --help

Help text

-u, --uri

The URI of the discovery source. The URI must be an OCI image.

vcf plugin sync

Synchronizes VCF CLI plugins. For each active CLI context, the command:

- Checks if all of the plugins required by the context are installed and if not, installs them.
- Checks if the version of each installed plugin is supported and if not, updates the plugin to the recommended

Usage

```
vcf plugin sync [FLAGS]
```

Example

To synchronize VCF CLI plugins

```
vcf plugin sync
```

Flags

-h, --help

Help text

vcf plugin uninstall

Uninstalls the specified plugin

Usage

```
vcf plugin uninstall <PLUGIN-NAME> [FLAGS]
```

Example

To uninstall *my-plugin*

```
vcf plugin uninstall my-plugin
```

Flags

-h, --help

Help text

-y, --yes

When **--yes** is specified, the command skips the confirmation step

vcf plugin upgrade

Upgrades the specified VCF CLI plugin

Usage

```
vcf plugin upgrade <PLUGIN-NAME> [FLAGS]
```

Example

To upgrade *my-plugin*

```
vcf plugin upgrade my-plugin
```

Flags

-h, --help

Help text

vcf plugin upload-bundle

Uploads a plugin bundle to a repository

Usage

```
vcf plugin upload-bundle [FLAGS]
```

Example

To upload a plugin bundle named *bundle_vmware_vcfcli_v1.0.0.tar.gz* to *custom.registry.broadcom.com/vcf/plugins/*

```
vcf plugin plugin upload-bundle --tar /tmp/bundle_vmware_vcfcli_v1.0.0.tar.gz --to-repo custom.registry.broad-
com.com/vcf/plugins/
```

Flags

-h, --help

Help text

--tar

The plugin bundle you want to upload

--to-repo

The destination repository

version

Version information of CLI

Usage

System command: `version` **| Primarily used for:** VCF CLI

Syntax

```
vcf version [flags]
```

Flags

-h, --help

Help text

Global Flags

--verbose int32

Sets the log level between 0 to 9 for this command invocation.

addon

Plugin to manage add-ons. For more information, see Managing Add-ons in VKS Clusters.

Usage

CLI plugin: `addon` **| Primarily used for**: VCF CLI

Syntax

```
vcf addon [command]
```

Flags

-h, --help

Help text

--log-file string

Log file path

--verbose int32

Number for the log level verbosity(0-9)

vcf addon repository

Get repository information

Syntax

```
vcf addon repository [command]
```

Flags

-h, --help

Help text

Available commands

- vcf addon repository delete
- vcf addon repository list
- vcf addon repository get
- vcf addon repository register

vcf addon repository delete

Delete an add-on repository.

Syntax

```
vcf addon repository delete ADDON_REPOSITORY_NAME [flags]
```

Flags

-h, --help

Help text

-y, --yes

Assume 'yes' for any prompt

vcf addon repository list

List all add-on repositories in the supervisor cluster

Syntax

```
vcf addon repository list
```

Flags

-h, --help

Help text

-o,--output string

Output format (yaml|json|table), optional

vcf addon repository get

Get the details for the add-on repository in the supervisor cluster.

Syntax

```
vcf addon repository get AddonRepository_NAME
```

Flags

-h, --help

Help text

vcf addon repository register

Register an add-on repository.

Syntax

```
vcf addon repository register [flags]
```

Flags

`-f, --file`

AddonRepository objects from which to create an add-on repository

`-h, --help` help text

`-y, --yes`

Register an add-on repository without asking for confirmation

vcf addon repository-install

Get information about RepositoryInstalls in the Supervisor cluster.

Syntax

```
vcf addon repository-install [command]
```

Flags

-h, --help

Help text

Available commands

- vcf addon repository-install list
- vcf addon repository-install get

vcf addon repository-install list

List all AddonRepositoryInstalls in the Supervisor cluster.

Syntax

```
vcf addon repository-install list
```

Flags

-h, --help

Help text

-o,--output string

Output format (yaml|json|table), optional

vcf addon repository-install get

Get the details for an AddonRepositoryInstall in the Supervisor cluster

Syntax

```
vcf addon repository-install get AddonRepositoryInstall_NAME
```

vcf addon repository-install reset

Reset an installed add-on repository.

Syntax

```
vcf addon repository-install reset AddonRepositoryInstallName [flags]
```

Flags

`-h, --help`

Help text

`-y, --yes`

Reset an AddonRepositoryInstall without asking for confirmation

vcf addon repository-install update

Syntax

```
vcf addon repository-install update AddonRepositoryInstallName [flags]
```

Flags

`--addon-repository-name`

Set AddonRepository name

`-h, --help`

Help text

`-y, --yes`

Update an AddonRepositoryInstall without asking for confirmation

vcf addon available-releases

Get information about available add-on releases in the Supervisor cluster.

Syntax

```
vcf addon available-releases [command]
```

Flags

`-h, --help`

Help text

Available commands

- vcf addon available-releases list
- vcf addon available-releases get

vcf addon available-releases list

List all AddonReleases in the Supervisor cluster.

Syntax

```
vcf addon available-releases list
```

Flags

`-h, --help`

Help text

`-o,--output string`

Output format (yaml|json|table), optional

vcf addon available-releases get

Get details of an AddonRelease in the Supervisor cluster.

Syntax

```
vcf addon available-releases get ADDON_RELEASE_NAME
```

Flags

`-h, --help`

Help text

`-o, --values-config-file-output`

File path for default configuration values file for the add-on

vcf addon available

Get information about available add-ons in the Supervisor cluster.

Syntax

```
vcf addon available [command]
```

Flags

`-h, --help`

Help text

Available commands

- vcf addon available list
- vcf addon available get

vcf addon available list

List add-ons available in the Supervisor cluster. If ADDON_NAME is specified, list all available versions for that add-on.

Syntax

```
vcf addon available list or list ADDON_NAME
```

Flags

`-h, --help`

Help text

vcf addon available get

Get the details for an available add-on in the Supervisor cluster.

Syntax

```
vcf addon available get ADDON_NAME
```

Flags

-h, --help

Help text

-v, --version

Version of the add-on

-o, --values-config-file-output

File path for default configuration values file for the add-on.

vcf addon install

Get information and perform operations related to add-on installation.

Syntax

```
vcf addon install [command]
```

Flags

-h, --help

Help text

Available commands

- vcf addon install list
- vcf addon install get
- vcf addon install update
- vcf addon install delete
- vcf addon install create
- vcf addon install migrate

vcf addon install list

List all installed add-ons for a particular workload cluster.

Syntax

```
vcf addon install list --cluster-name <workload-cluster-name>
```

Flags

-h, --help

Help text

--cluster-name

Name for the workload cluster. Required.

-n, --namespace

Namespace for the workload cluster

vcf addon install get

Get the details for a particular installed add-on.

Syntax

```
vcf addon install get INSTALLED_ADDON_NAME --cluster-name <workload-cluster-name>
```

Flags

-h, --help

Help text

--cluster-name

Name for the workload cluster. Required.

-n, --namespace

Namespace for the workload cluster

-o, --values-config-file-output

File path for default configuration values file for the add-on.

vcf addon install update

Update an installed add-on. Note that you cannot update the add-on version or config at the same time that you update the add-on.

Syntax

```
vcf addon install update INSTALLED_ADDON_NAME [flags]
```

Flags

-h, --help

Help text

--cluster-name

Name for the workload cluster. Required.

-n, --namespace

Namespace for the workload cluster

-y, --yes

Assume 'yes' for any prompt

-f, --values-config-file

File path for default configuration values file for the add-on.

-v, --version

Version of the add-on

-addon-release-name

Set add-on release name

vcf addon install delete

Delete an installed add-on.

Syntax

```
vcf addon install delete INSTALLED_ADDON_NAME --cluster-name <workload-cluster-name>
```

Flags

-h, --help

Help text

--cluster-name

Name for the workload cluster. Required.

-n, --namespace

Namespace for the workload cluster

-y, --yes

Assume 'yes' for any prompt

vcf addon install create

Install an add-on in a workload cluster.

Syntax

```
vcf addon install create ADDON_NAME [flags]
```

Flags

-h, --help

Help text

--cluster-name

Name for the workload cluster. Required.

-n, --namespace

Namespace for the workload cluster

-y, --yes

Assume 'yes' for any prompt

-f, --values-config-file

File path for default configuration values file for the add-on. Optional.

-v, --version

Version of the add-on

--addon-release-name

Set add-on release name

vcf addon install migrate

Migrate a legacy package install to the new Add-on framework. For a detailed migration workflow see Migrate a Legacy Package Add-on.

Syntax

```
vcf addon install migrate PACKAGE_INSTALL_NAME --cluster-name <workload-cluster-name> --addon-name ADDON_NAME
 [flags]
```


Flags

-h, --help

Help text

--cluster-name

Name for the workload cluster. Required.

-n, --namespace

Namespace for the workload cluster

-y, --yes

Assume 'yes' for any prompt

--addon-name

Set the add-on name. Required.

--addon-release-name

Set add-on release name

-v, --version

Version of the add-on

--existing-package-install-namespace

Namespace for the installed standard package to be migrated. Optional.

-f, --values-config-file

The path to the package values configuration file, optional

cluster

Plugin to manage workload clusters.

Note:

Install kubectl on the CLI machine and add it to the PATH environment variable.

Usage

CLI plugin: `cluster` | **Primarily used for**: VCF CLI

Important:

For VCFA contexts, we need to switch to the corresponding namespace context before using the cluster plugin.

Example:

```
vcf context use vcfa-context:namespace1:project1
```

Syntax

```
vcf cluster [command]
```

Flags

-h, --help

Help text

--log-file string

Log file path

--verbose int32

Number for the log level verbosity(0-9)

available-upgrades

Get upgrade information for a workload cluster

Syntax

```
vcf cluster available-upgrades [command]
```

Flags

-h, --help

Help text

Available commands

- vcf cluster available-upgrades get

vcf cluster available-upgrades get

Get all available upgrades for a workload cluster

Syntax

```
vcf cluster available-upgrades get <CLUSTER_NAME> [flags]
```

Flags

-h, --help

Help text

-n, --namespace string

The namespace where the workload cluster was created. In VCFA context, this parameter is not required.

vcf cluster create

Create a workload cluster

Syntax

```
vcf cluster create [flags]
```

Flags

-f, --file string

Configuration file or Cluster objects from which to create a cluster

-h, --help

Help text

-y, --yes

Create workload cluster without asking for confirmation

vcf cluster delete

Delete a workload cluster

Syntax

```
vcf cluster delete <CLUSTER_NAME> [flags]
```

Flags

-h, --help

Help text

-n, --namespace string

The namespace where the workload cluster was created. In VCFA context, this parameter is not required.

-y, --yes

Delete workload cluster without asking for confirmation

vcf cluster get

Get details from a workload cluster

Syntax

```
vcf cluster get <CLUSTER_NAME> [flags]
```

Flags

-h, --help

Help text

-n, --namespace string

The namespace from which to get workload clusters. In VCFA context, this parameter is not required.

--show-all-conditions string

List of comma separated kind or kind/name for which we must show all the object's conditions (all to show conditions for all the objects)

--show-details

Show details of MachineInfrastructure and BootstrapConfig when ready condition is true or it has the Status, Severity and Reason of the machine's object

--show-group-members

Expand machine groups whose ready condition has the same Status, Severity and Reason

--show-v1beta2

Construct tree to use V1Beta2 conditions. Only works with --show-all-conditions

vcf cluster kubeconfig

Cluster kubeconfig operations

Syntax

```
vcf cluster kubeconfig [command]
```

Flags

-h, --help

Help text

Available commands

- vcf cluster kubeconfig get

vcf cluster kubeconfig get

Get kubeconfig of a workload cluster and merge the context into the default kubeconfig file

Prerequisites

- **Supervisor**
  - Register IDP on supervisor. See Configure an External Identity Provider for VKS Clusters.
  - Add permissions in Supervisor for the external identity provider(IDP) user and group. See Configure vSphere Namespace Permissions for External Identity Provider Users and Groups.
- **VCFA**
  - Create a vcfa context with the ---ca-certificate parameter
  - Switch to the namespace context
  - Run jwt authenticator command: vcf cluster register-vcfa-jwt-authenticator <CLUSTER_NAME>

Syntax

```
vcf cluster kubeconfig get <CLUSTER_NAME> [flags]
```

Examples

Get workload cluster kubeconfig

```
vcf cluster kubeconfig get <CLUSTER_NAME>
```

--export-file string

File path to export a standalone kubeconfig for workload cluster

-h, --help

Help text

-n, --namespace string

The namespace where the workload cluster was created. In VCFA context, this parameter is not required.

vcf cluster list

List workload clusters

Syntax

```
vcf cluster list [flags]
```

Flags

-A, --all-namespaces

If present, list the cluster(s) across all namespaces. Namespace in current context is ignored even if specified with --namespace

-h, --help

Help text

-n, --namespace string

The namespace from which to list workload clusters. In VCFA context, this parameter is not required.

-o, --output string

Output format (yaml|json|table)

vcf cluster machinehealthcheck

Manage lifecycle of MachineHealthCheck object for a workload cluster

Syntax

```
vcf cluster machinehealthcheck [command]
```

Flags

-h, --help

Help text

Available commands

- vcf cluster machinehealthcheck control-plane
- vcf cluster machinehealthcheck node

vcf cluster machinehealthcheck control-plane

Manage lifecycle of MachineHealthCheck object for the control plane of a workload cluster

Syntax

```
vcf cluster machinehealthcheck control-plane [command]
```

Flags

-h, --help

Help text

Available commands

- vcf cluster machinehealthcheck control-plane delete
- vcf cluster machinehealthcheck control-plane get
- vcf cluster machinehealthcheck control-plane set

vcf cluster machinehealthcheck control-plane delete

Delete a MachineHealthCheck object of the control plane of a workload cluster

Syntax

```
vcf cluster machinehealthcheck control-plane delete <CLUSTER_NAME> [flags]
```

Flags

-h, --help

Help text

-m, --mhc-name string

Name of the MachineHealthCheck object (ignored for Classy Clusters)

-n, --namespace string

The namespace where the MachineHealthCheck object was created. In VCFA context, this parameter is not required.

-y, --yes

Delete the MachineHealthCheck object without asking for confirmation

vcf cluster machinehealthcheck control-plane get

Get a MachineHealthCheck object of the control plane for the given workload cluster

Syntax

```
vcf cluster machinehealthcheck control-plane get <CLUSTER_NAME> [flags]
```

Flags

-h, --help

Help text

-m, --mhc-name string

Name of the MachineHealthCheck object (ignored for Classy Clusters)

-n, --namespace string

The namespace where the MachineHealthCheck object was created. In VCFA context, this parameter is not required.

vcf cluster machinehealthcheck control-plane set

Create or update a MachineHealthCheck for the control plane of a workload cluster

Syntax

```
vcf cluster machinehealthcheck control-plane set <CLUSTER_NAME> [flags]
```

Flags

-h, --help

Help text

--match-labels string

Label selector to match machines whose health will be exercised

--max-unhealthy string

Remediation will not be performed when the number of unhealthy Machines exceeds the value. It must be either an absolute number or a percentage.

-m, --mhc-name string

Name of the MachineHealthCheck object (ignored for Classy Clusters)

-n, --namespace string

Namespace of the cluster, In VCFA context, this parameter is not required.

--node-startup-timeout string

Any machine being created that takes longer than this duration to join the cluster is considered to have failed and will be remediated.

--unhealthy-conditions string

A list of the conditions that determine whether a contol-plane node is considered unhealthy. Available condition types: [Ready, MemoryPressure,DiskPressure,PIDPressure, NetworkUnavailable], Available condition status: [True, False, Unknown]

vcf cluster machinehealthcheck node

Manage lifecycle of MachineHealthCheck object for the nodes of a workloadcluster

Syntax

```
vcf cluster machinehealthcheck node [command]
```

Flags

-h, --help

Help text

Available commands

- vcf cluster machinehealthcheck node delete
- vcf cluster machinehealthcheck node get
- vcf cluster machinehealthcheck node set

vcf cluster machinehealthcheck node delete

Delete a MachineHealthCheck object of the nodes for the given workload cluster

Syntax

```
vcf cluster machinehealthcheck node delete <CLUSTER_NAME> [flags]
```

Flags

-h, --help

Help text

--machine-deployment string

Name of the Machine Deployment (only for Classy Clusters)

-m, --mhc-name string

Name of the MachineHealthCheck object (ignored for Classy Clusters)

-n, --namespace string

The namespace where the MachineHealthCheck object was created. In VCFA context, this parameter is not required.

-y, --yes

Delete the MachineHealthCheck object without asking for confirmation

vcf cluster machinehealthcheck node get

Get a MachineHealthCheck object of the nodes for the given workload cluster

Syntax

```
vcf cluster machinehealthcheck node get <CLUSTER_NAME> [flags]
```

Flags

-h, --help

Help text

-m, --mhc-name string

Name of the MachineHealthCheck object

-n, --namespace string

The namespace where the MachineHealthCheck object was created. In VCFA context, this parameter is not required.

vcf cluster machinehealthcheck node set

Create or update a MachineHealthCheck for the worker nodes of a workload cluster

Syntax

```
vcf cluster machinehealthcheck node set <CLUSTER_NAME> [flags]
```

Flags

-h, --help

Help text

--machine-deployment string

Name of the Machine Deployment (only for Classy Clusters)

--match-labels string

Label selector to match machines whose health will be exercised. It must be in key:value format

--max-unhealthy string

Remediation will not be performed when the number of unhealthy Machines exceeds the value. Either an absolute number or a percentage of the total Machines.

-m, --mhc-name string

Name of the MachineHealthCheck object (ignored for Classy Clusters)

-n, --namespace string

Namespace of the cluster. In VCFA context, this parameter is not required.

--node-startup-timeout string

Any machine being created that takes longer than this duration to join the cluster is considered to have failed and will be re-mediated

--unhealthy-conditions string

A list of the conditions that determine whether a node is considered unhealthy. Available condition types: [Ready, MemoryPressure,DiskPressure,PIDPressure, NetworkUnavailable], Available condition status: [True, False, Unknown]

vcf cluster node-pool

Cluster node-pool operations

Syntax

```
vcf cluster node-pool [command]
```

Flags

-h, --help

Help text

Available commands

- vcf cluster node-pool delete
- vcf cluster node-pool delete
- vcf cluster node-pool set

vcf cluster node-pool delete

Delete a NodePool object of a workload cluster

Syntax

```
vcf cluster node-pool delete <CLUSTER_NAME> [flags]
```

Flags

-h, --help

Help text

-n, --name string

Name of the NodePool object

--namespace string

The namespace the cluster is found in. In VCFA context, this parameter is not required.

vcf cluster node-pool list

List node pools of a workload cluster

Syntax

```
vcf cluster node-pool list [<CLUSTER_NAME>] [flags]
```

Flags

-h, --help

Help text

--namespace string

The namespace from which to list workload clusters. (default "default")

-o, --output string

Output format (yaml|json|table)

vcf cluster node-pool set

Set node pool for a workload cluster

Syntax

```
vcf cluster node-pool set <CLUSTER_NAME> [flags]
```

Flags

--base-machine-deployment string

The machine deployment to use as a base for creating a new node pool

-f, --file string

The file describing the node pool (required)

-h, --help

Help text

--namespace string

The namespace the cluster is found in. In VCFA context, this parameter is not required.

vcf cluster register-vcfa-jwt-authenticator

Inject the VCFA JWTAuthenticator into a workload cluster

Syntax

```
vcf cluster register-vcfa-jwt-authenticator <CLUSTER_NAME> [flags]
```

Flags

-h, --help

vcf cluster scale

Scale a workload cluster

Syntax

```
vcf cluster scale <CLUSTER_NAME> [flags]
```

Flags

-c, --controlplane-machine-count int32

The number of control plane nodes to scale to. Assumes unchanged if not specified

-h, --help

Help text

-n, --namespace string

The namespace where the workload cluster was created. In VCFA context, this parameter is not required.

-p, --node-pool-name string

The name of the node-pool to scale

-w, --worker-machine-count int32

The number of worker nodes to scale to. Assumes unchanged if not specified

vcf cluster upgrade

Upgrade a workload cluster

Syntax

```
vcf cluster upgrade <CLUSTER_NAME> [flags]
```

Examples

Upgrade a workload cluster


```
vcf cluster upgrade wc-1
```

Upgrade a workload cluster with specific tkr v1.20.1---vmware.1-tkg.2

```
vcf cluster upgrade wc-1 --kr v1.20.1---vmware.1-tkg.2
```

Upgrade a workload cluster with tkr prefix v1.20.1

```
vcf cluster upgrade wc-1 --kr v1.20.1
```

Flags

--force

Upgrade workload cluster even it is not in running status

-h, --help

Help text

--kr string

KubernetesRelease to upgrade to. If KubernetesRelease name prefix is provided, the latest compatible KubernetesRelease matching the KubernetesRelease name prefix would be used

-n, --namespace string

The namespace where the workload cluster was created. In VCFA context, this parameter is not required.

-t, --timeout duration

Time duration to wait for an operation before timeout. Timeout duration in hours(h)/minutes(m)/seconds(s) units or as some combination of them (e.g. 2h, 30m, 2h30m10s) (default 30m0s)

-y, --yes

Upgrade workload cluster without asking for confirmation

vcf cluster support-bundler create

Create a support bundle for a workload cluster.

Syntax

```
vcf cluster support-bundler create [flags]
```

Flags

-b, --batch-size int

Number of nodes on which parallel collection is triggered. Default: 5.

--ca-certificate string

Path to the endpoint public certificate file

--channel string

Communication protocol to use for support bundle collection: `guestops` or `ssh` . When choosing the `ssh` channel to collect logs, you are required to run this command in the same subnets as guest clusters. When using the `guestops` channel, you must supply the vCenter username and password. The username and password are not required when using the `ssh` channel.

-c, --cluster string

Workload cluster to collect support-bundle for

--controlplane-node-only

To collect support bundle only from control plane nodes

-h, --help

Help text

-i, --insecure

Creates an insecure connection to the VC

-k, --kubeconfig string

Absolute path to the Supervisor cluster `kubeconfig` . If this flag is not set, `kubeconfig` will be picked up from KUBECONFIG environment variable. Default `HOME/.kube/config` .

-l, --log-ns string

Comma separated namespaces list whose logs should be included

-n, --namespace string

Supervisor cluster namespace where the workload cluster resides

-s, --node-stats

To include the node stats in the support bundle

-o, --output string

Absolute path to the directory where the support-bundle will be stored, for example, */home/myuser/mybundle*

-p, --progress-bar

To progress-bar for support-bundle collection per node

-t, --resource-types string

Comma-separated list of Kubernetes resource types, such as pvc or pv

--skip-create-user

Skip creation of a Guest Operations user. When you use this option, *-u, --user string* must specify a vCenter user that meets these requirements:

- Belongs to the ServiceProviderUsers group.
- Has the following privileges:
  - VirtualMachine.GuestOperations.Execute
  - VirtualMachine.GuestOperations.Query
  - VirtualMachine.GuestOperations.Modify

-u, --user string

VC user name

-v, --vc string

VC IP or FQDN with optional port. Default: 443 for HTTPS.

-V, --verbose

Collect additional logs to help debug support-bundle collection failures and print to stderr

-w, --windows-admin-username string

Windows vm admin username

-e, --windows-event-hours-ago string

Specify the collection of windows events within a few hours. Default: 12.

Example

Using `ssh` channel:

```
vcf cluster support-bundler create \
   -c vcf-cli-vcf-cli-7a2vss-5ex1 \
   -n vcf-cli-7a2vss \
   -o test_logs \
   --channel ssh
```

Using `guestops` channel:

```
vcf cluster support-bundler create \
  -v 10.167.xx.xxx \
  -u administrator@vsphere.local \
  -k /root/.kube/wcp-config \
  -c vcf-cli-vcf-cli-7a2vss-5ex1 \
  -n vcf-cli-7a2vss \
  -o test_logs \
  -i true
```

imgpkg

Plugin to store configuration and refer images as OCI artifacts

Usage

CLI plugin: `imgpkg` **| Primarily used for**: VCF CLI | Release Notes

Syntax

```
vcf imgpkg [flags]
vcf imgpkg [command]
```

Flags

-h, --help

Help text

--color

Set color output (default true)

--column string

Filter to show only given columns

--debug

Enables debugging

--json

Output as JSON

--tty

Force TTY-like output

-y, --yes

Assume yes for any prompt

vcf imgpkg copy

Copy a bundle from one location to another

Syntax

```
vcf imgpkg copy [flags]
```

Examples

Copy bundle dkalinin/app1-bundle to local tarball at /Volumes/app1-bundle.tar

```
vcf imgpkg copy -b dkalinin/app1-bundle --to-tar /Volumes/app1-bundle.tar
```

Copy bundle dkalinin/app1-bundle to another registry (or repository)

```
vcf imgpkg copy -b dkalinin/app1-bundle --to-repo internal-registry/app1-bundle
```

Copy image dkalinin/app1-image to another registry (or repository)

```
vcf imgpkg copy -i dkalinin/app1-image --to-repo internal-registry/app1-image
```

Note:

If ~/.docker.config is not used for authn, use environment variables as described https://carvel.dev/imgpkg/docs/v0.24.0/auth/#via-environment-variables

Copy using image --repo-based-tags flag

```
vcf copy -i registry.foo.bar/some/application/app --to-repo other-reg.faz.baz/my-app --repo-based-tags
```

Note:

If the above source repo has a tag sha256:669e010b58baf5beb2836b253c1fd5768333f0d1dbcb834f7c07a4dc93f474be, a new tag some-application-app-sha256-669e010b58baf5beb2836b253c1fd5768333f0d1dbcb834f7c07a4dc93f474be.imgpkg will be created in the destination repo.

Also note that the part of the new tag preceeding '-sha256' will be truncated to the last 49 charachters

Flags

-b, --bundle string

Bundle reference for copying (happens thickly, i.e. bundle image + all referenced images)

--concurrency int

Concurrency (default 5)

--cosign-signatures

Find and copy cosign signatures for images

-h, --help

Help text

-i, --image string

Image reference for copying a generic image (example: docker.io/dkalinin/test-content)

--include-non-distributable-layers

Include non-distributable layers when copying an image/bundle

--lock string

Lock file with asset references to copy to destination

--lock-output string

Location to output the generated lockfile. Option only available when using --bundle or --lock flags

--registry-anon

Set anonymous auth ($IMGPKG_ANON)

--registry-ca-cert-path string

Add CA certificates for registry API (format: /tmp/foo) (can be specified multiple times)

--registry-insecure

Allow the use of http when interacting with registries

--registry-password string

Set password for auth ($IMGPKG_PASSWORD)

--registry-response-header-timeout duration

Maximum time to allow a request to wait for a server's response headers from the registry (ms|s|m|h) (default 30s)

--registry-retry-count int

Set the number of times imgpkg retries to send requests to the registry in case of an error (default 5)

--registry-token string

Set token for auth ($IMGPKG_TOKEN)

--registry-username string

Set username for auth ($IMGPKG_USERNAME)

--registry-verify-certs

Set whether to verify server's certificate chain and host name (default true)

--repo-based-tags

Allow imgpkg to use repository-based tags for convenience

--resume

Resume the copy to tar. When set to true will try to read the tar and only download the missing blobs

--tar string

Path to tar file which contains assets to be copied to a registry

--to-repo string

Location to upload assets

--to-tar string

Location to write a tar file containing assets

vcf imgpkg describe

Describe the images and bundles associated with a give bundle

Syntax

```
vcf imgpkg describe [flags]
```

Examples

Describe a bundle

```
vcf imgpkg describe -b carvel.dev/app1-bundle
```

Flags

-b, --bundle string

Bundle reference for copying (happens thickly, i.e. bundle image + all referenced images)

--concurrency int

Concurrency (default 5)

--cosign-signatures

Find and copy cosign signatures for images

-h, --help

Help text

--layers

Retrieve image layers info (Default: false) (default true)

-o, --output-type string

Type of output possible values: [text, yaml] (default "text")

--registry-anon

Set anonymous auth ($IMGPKG_ANON)

--registry-ca-cert-path string

Add CA certificates for registry API (format: /tmp/foo) (can be specified multiple times)

--registry-insecure

Allow the use of http when interacting with registries

--registry-password string

Set password for auth ($IMGPKG_PASSWORD)

--registry-response-header-timeout duration

Maximum time to allow a request to wait for a server's response headers from the registry (ms|s|m|h) (default 30s)

--registry-retry-count int

Set the number of times imgpkg retries to send requests to the registry in case of an error (default 5)

--registry-token string

Set token for auth ($IMGPKG_TOKEN)

--registry-username string

Set username for auth ($IMGPKG_USERNAME)

--registry-verify-certs

Set whether to verify server's certificate chain and host name (default true)

vcf imgpkg pull

Pull files from bundle, image, or bundle lock file

Syntax

```
vcf imgpkg pull [flags]
```

Examples

Pull bundle repo/app1-bundle and extract into /tmp/app1-bundle

```
vcf imgpkg pull -b repo/app1-bundle -o /tmp/app1-bundle
```

Pull image repo/app1-image and extract into /tmp/app1-image

```
vcf imgpkg pull -i repo/app1-image -o /tmp/app1-image
```

Flags

-b, --bundle string

Bundle reference for copying (happens thickly, i.e. bundle image + all referenced images)

-h, --help

Help text

-i, --image string

Image reference for copying a generic image (example: docker.io/dkalinin/test-content)

--image-is-bundle-check

Error when image is a bundle (disable pulling bundles via -i) (default true)

--lock string

Lock file with asset references to copy to destination

-o, --output-type string

Type of output possible values: [text, yaml] (default "text")

-r, --recursive

Recursively iterate and fetch content of every bundle

--registry-anon

Set anonymous auth ($IMGPKG_ANON)

--registry-ca-cert-path string

Add CA certificates for registry API (format: /tmp/foo) (can be specified multiple times)

--registry-insecure

Allow the use of http when interacting with registries

--registry-password string

Set password for auth ($IMGPKG_PASSWORD)

--registry-response-header-timeout duration

Maximum time to allow a request to wait for a server's response headers from the registry (ms|s|m|h) (default 30s)

--registry-retry-count int

Set the number of times imgpkg retries to send requests to the registry in case of an error (default 5)

--registry-token string

Set token for auth ($IMGPKG_TOKEN)

--registry-username string

Set username for auth ($IMGPKG_USERNAME)

--registry-verify-certs

Set whether to verify server's certificate chain and host name (default true)

vcf imgpkg push

Push files as image

Syntax

```
vcf imgpkg push [flags]
```

Examples

Push bundle repo/app1-config with contents of config/ directory

```
vcf imgpkg push -b repo/app1-config -f config/
```

Push image repo/app1-config with contents from multiple locations

```
vcf imgpkg push -i repo/app1-config -f config/ -f additional-config.yml
```

Flags

-b, --bundle string

Bundle reference for copying (happens thickly, i.e. bundle image + all referenced images)

-f, --file string

Set file (format: /tmp/foo) (can be specified multiple times)

--file-exclusion string

Exclude file whose path, relative to the bundle root, matches (format: bar.yaml, nested-dir/baz.txt) (can be specified multiple times) (default [.git])

-h, --help

Help text

-i, --image string

Image reference for copying a generic image (example: docker.io/dkalinin/test-content)

-l, --labels string

ToString Set labels on image (default [])

--lock-output string

Location to output the generated lockfile. Option only available when using --bundle or --lock flags

--preserve-permissions

Preserve the group and all permissions of all the files and folders

--registry-anon

Set anonymous auth ($IMGPKG_ANON)

--registry-ca-cert-path string

Add CA certificates for registry API (format: /tmp/foo) (can be specified multiple times)

--registry-insecure

Allow the use of http when interacting with registries

--registry-password string

Set password for auth ($IMGPKG_PASSWORD)

--registry-response-header-timeout duration

Maximum time to allow a request to wait for a server's response headers from the registry (ms|s|m|h) (default 30s)

--registry-retry-count int

Set the number of times imgpkg retries to send requests to the registry in case of an error (default 5)

--registry-token string

Set token for auth ($IMGPKG_TOKEN)

--registry-username string

Set username for auth ($IMGPKG_USERNAME)

--registry-verify-certs

Set whether to verify server's certificate chain and host name (default true)

vcf imgpkg tag

List or resolve image tag

Syntax

```
vcf imgpkg tag [flags]
vcf imgpkg tag [command]
```

Flags

-h, --help

Help text

Available commands

- vcf imgpkg tag list
- vcf imgpkg tag resolve

vcf imgpkg tag list

List tags for image

Syntax

```
vcf imgpkg tag list [flags]
```

Flags

--digests

Include digests

-h, --help

Help text

-i, --image string

Set image (example: docker.io/dkalinin/test-content)

--imgpkg-internal-tags

Include internal .imgpkg tags

--registry-anon

Set anonymous auth ($IMGPKG_ANON)

--registry-ca-cert-path string

Add CA certificates for registry API (format: /tmp/foo) (can be specified multiple times)

--registry-insecure

Allow the use of http when interacting with registries

--registry-password string

Set password for auth ($IMGPKG_PASSWORD)

--registry-response-header-timeout duration

Maximum time to allow a request to wait for a server's response headers from the registry (ms|s|m|h) (default 30s)

--registry-retry-count int

Set the number of times imgpkg retries to send requests to the registry in case of an error (default 5)

--registry-token string

Set token for auth ($IMGPKG_TOKEN)

--registry-username string

Set username for auth ($IMGPKG_USERNAME)

--registry-verify-certs

Set whether to verify server's certificate chain and host name (default true)

vcf imgpkg tag resolve

Resolve tag to digest for image

Syntax

```
vcf imgpkg tag resolve [flags]
```

Flags

-h, --help

Help text

-i, --image string

Set image (example: docker.io/dkalinin/test-content)

--registry-anon

Set anonymous auth ($IMGPKG_ANON)

--registry-ca-cert-path string

Add CA certificates for registry API (format: /tmp/foo) (can be specified multiple times)

--registry-insecure

Allow the use of http when interacting with registries

--registry-password string

Set password for auth ($IMGPKG_PASSWORD)

--registry-response-header-timeout duration

Maximum time to allow a request to wait for a server's response headers from the registry (ms|s|m|h) (default 30s)

--registry-retry-count int

Set the number of times imgpkg retries to send requests to the registry in case of an error (default 5)

--registry-token string

Set token for auth ($IMGPKG_TOKEN)

--registry-username string

Set username for auth ($IMGPKG_USERNAME)

--registry-verify-certs

Set whether to verify server's certificate chain and host name (default true)

kubernetes-release

Plugin for Kubernetes release operations

Usage

CLI plugin: `kubernetes-release` **| Primarily used for**: VCF CLI

Note:

For VCFA contexts, we need to switch to the corresponding namespace context before using the kubernetes-release plugin.

Example:

```
vcf context use vcfacontext:namespace1:context
```

Syntax

```
vcf kubernetes-release [command]
```

Flags

-h, --help

Help text

--log-file string

Log file path

--verbose int32

Number for the log level verbosity(0-9)

vcf kubernetes-release get

Get available Kubernetes releases

Syntax

```
vcf kubernetes-release get KUBERNETES_RELEASE_NAME [flags]
```

Flags

-a, --all

List all the available Kubernetes Releases including incompatible and deactivated ones

-h, --help

Help text

-o, --output string

Output format (yaml|json|table)

vcf kubernetes-release os

Get the OS information for a Kubernetes Release

Syntax

```
vcf kubernetes-release os [commands]
```

Flags

-h, --help

Help text

Available commands

- vcf kubernetes-release os get

vcf kubernetes-release os get

Get the OSes that are available for a Kubernetes Release

Usage

```
vcf kubernetes-release os get KUBERNETES_RELEASE_NAME [flags]
```

Flags

-h, --help

Help text

-o, --output string

Output format (yaml|json|table)

namespaces

Plugin to discover vSphere supervisor namespaces on which user has access

Usage

CLI plugin: `namespaces` **| Primarily used for**: VCF CLI | Release Notes

Syntax

```
vcf namespace [command]
```

Flags

-h, --help

Help text

vcf namespaces get

Get namespaces for a vSphere Supervisor

Syntax

```
vcf namespaces get [flags]
```

Examples

Get vSphere Supervisor namespaces which user has access

vcf namespaces get

Flags

-h, --help

Help text

-o, --output string

Output format (yaml|json|table)

-v, --verbose int32

Number for the log level verbosity(0-9) (default 3)

package

Plugin to manage standard packages in a workload cluster.

Usage

CLI plugin: `package` **| Primarily used for:** VCF CLI

This plugin works on a workload cluster.

For k8s contexts, create context with --workload-cluster-name and --workload-cluster-namespace and use the context with a format of 'ContextName:WorkloadClusterName'

For VCFA contexts, we need to create a context using the 'cluster kubeconfig' command before using it.

Example:

```
vcf cluster kubeconfig get cluster1
vcf context create your-namespace1-cluster1 --kubeconfig  ~/.kube/config --kubecontext vcf-cli-name-
space1-cluster1@namespace1-cluster1 --type k8s
vcf context use your-namespace1-cluster1
```

Syntax

```
vcf package [flags]
vcf package [command]
```

Flags

--column string

Filter to show only given columns

--debug

Include debug output

-h, --help

Help text

--kubeconfig string

Path to the kubeconfig file ($VCF_KUBECONFIG)

--kubeconfig-context string

Kubeconfig context override ($VCF_KUBECONFIG_CONTEXT)

--kubeconfig-yaml string

Kubeconfig contents as YAML ($VCF_KUBECONFIG_YAML)

-y, --yes

Assume yes for any prompt

vcf package available

Manage available packages

Syntax

```
vcf package available [flags]
vcf package available [command]
```

Flags

-h, --help

Help text

Available commands

- vcf package available get
- vcf package available list

vcf package available get

Get details for an available package or the openAPI schema of a package with a specific version

Usage


```
vcf package available get PACKAGE_NAME or PACKAGE_NAME/VERSION [flags]
```

Examples

Get details about an available package

```
vcf package available get package.corp.com
```

Get the values schema for a particular version of the package

```
vcf package available get package.corp.com/1.0.0 --values-schema
```

Flags

--default-values-file-output string

File path to save default values (optional)

-h, --help

Help text

-n, --namespace string

Specified namespace ($VCF_NAMESPACE or default from kubeconfig)

-o, --output string

Output format (yaml|json|table), optional

-tty

Force TTY-like output

--values-schema

Values schema of the package (optional)

vcf package available list

List available packages in a namespace

Usage

```
vcf package available list or list PACKAGE_NAME [flags]
```

Examples

List packages available on the cluster

```
vcf package available list
```

List packages available on the cluster with their short description

```
vcf package available list --wide
```

List all available package versions with release dates

```
vcf package available list --summary=false
```

List packages available in all namespaces

```
vcf package available list -A
```

List all available versions of a package

```
vcf package available list package.corp.com
```

Flags

-A, --all-namespaces

List available packages in all namespaces

-h, --help

Help text

-n, --namespace string

Specified namespace ($VCFA_NAMESPACE or default from kubeconfig)

-o, --output string

Output format (yaml|json|table), optional

--summary

Show summarized list of packages (default true) -

-tty

Force TTY-like output

--wide

Show additional info

install

Install package

Syntax

```
vcf package install INSTALLED_PACKAGE_NAME --package PACKAGE_NAME --version VERSION [flags]
```

Examples

Install a package

```
vcf package install sample-pkg-install -p package.corp.com --version 1.0.0
```

Install package with values file

```
vcf package install sample-pkg-install -p package.corp.com --version 1.0.0 --values-file values.yml
```

Install package and ask it to use an existing service account

```
vcf package install sample-pkg-install -p package.corp.com --version 1.0.0 --service-account-name existing-sa
```

Flags

--dangerous-allow-use-of-shared-namespace

Allow use of shared namespaces

--dry-run

Print YAML for resources being applied to the cluster without applying them, optional -n,

-h, --help

Help text

--namespace string

Specified namespace ($VCF_NAMESPACE or default from kubeconfig)

-p, --package string

Set package name (required)

--service-account-name string

Name of an existing service account used to install underlying package contents, optional

-tty

Force TTY-like output

--values

Add or keep values supplied to package install, optional (default true)

--values-file string

The path to the configuration values file, optional

-v, --version string

Set package version (required)

--wait

Wait for reconciliation to complete (default true)

--wait-check-interval duration

Amount of time to sleep between checks while waiting (default 1s)

--wait-timeout duration

Maximum amount of time to wait in wait phase (default 30m0s)

--ytt-overlay-file string

Path to ytt overlay file (can also be a directory)

--ytt-overlays

Add or keep ytt overlays (default true)

vcf package installed

Manage installed packages

Syntax

```
vcf package installed [flags]
vcf package installed [command]
```

Flags

-h, --help

Help text

Available commands

- vcf package installed create
- vcf package installed delete
- vcf package installed get
- vcf package installed list

- vcf package installed pause
- vcf package installed status
- vcf package installed update

vcf package installed create

Install package

Usage

```
vcf package installed create INSTALLED_PACKAGE_NAME --package PACKAGE_NAME --version VERSION [flags]
```

Examples

Install a package

```
vcf package installed create sample-pkg-install -p package.corp.com --version 1.0.0
```

Install package with values file

```
vcf package installed create sample-pkg-install -p package.corp.com --version 1.0.0 --values-file values.yml
```

Install package and ask it to use an existing service account

```
vcf package installed create sample-pkg-install -p package.corp.com --version 1.0.0 --service-account-name ex-
isting-sa
```

Flags

--dangerous-allow-use-of-shared-namespace

Allow use of shared namespaces

--dry-run

Print YAML for resources being applied to the cluster without applying them, optional -n,

-h, --help

Help text

--namespace string

Specified namespace ($VCF_NAMESPACE or default from kubeconfig)

-p, --package string

Set package name (required)

--service-account-name string

Name of an existing service account used to install underlying package contents, optional

-tty

Force TTY-like output

--values

Add or keep values supplied to package install, optional (default true)

--values-file string

The path to the configuration values file, optional

-v, --version string

Set package version (required)

--wait

Wait for reconciliation to complete (default true)

--wait-check-interval duration

Amount of time to sleep between checks while waiting (default 1s)

--wait-timeout duration

Maximum amount of time to wait in wait phase (default 30m0s)

--ytt-overlay-file string

Path to ytt overlay file (can also be a directory)

--ytt-overlays

Add or keep ytt overlays (default true)

vcf package installed delete

Uninstall installed package

Usage

```
vcf package installed delete INSTALLED_PACKAGE_NAME [flags]
```

Examples

Delete package install

```
vcf package installed delete sample-pkg-install
```

Flags

-h, --help

Help text

--namespace string

Specified namespace ($VCF_NAMESPACE or default from kubeconfig)

-tty

Force TTY-like output (default true)

--wait-check-interval duration

Amount of time to sleep between checks while waiting (default 1s)

--wait-timeout duration

Maximum amount of time to wait in wait phase (default 5m0s)

vcf package installed get

Get details for installed package

Usage

```
vcf package installed get INSTALLED_PACKAGE_NAME [flags]
```

Examples

Get details for a package install

```
vcf package installed get sample-pkg-install
```

View values being used by package install

```
vcf package installed get sample-pkg-install --values
```

Download values being used by package install

```
vcf package installed get sample-pkg-install --values-file-output values.yml
```

Flags

-h, --help

Help text

--namespace string

Specified namespace ($VCF_NAMESPACE or default from kubeconfig)

-o, --output string

Output format (yaml|json|table), optional

-tty

Force TTY-like output

--values

Get values data for package install

--values-file string

File path for exporting configuration values file

vcf package installed list

List installed packages in a namespace

Usage

```
vcf package installed list [flags]
```

Examples

List installed packages

```
vcf package installed list
```

List installed packages in all namespaces

```
vcf package installed list -A
```

Flags

-A, --all-namespaces

List installed packages in all namespaces

-h, --help

Help text

--namespace string

Specified namespace ($KCTRL_NAMESPACE or default from kubeconfig)

-o, --output string

Output format (yaml|json|table), optional

-tty

Force TTY-like output (default true)

vcf package installed pause

Pause reconciliation of package install

Usage

```
vcf package installed pause INSTALLED_PACKAGE_NAME [flags]
```

Examples

Pause reconciliation of package install

```
vcf package installed pause sample-pkg-install
```

Flags

-h, --help

Help text

--namespace string

Specified namespace ($KCTRL_NAMESPACE or default from kubeconfig)

-tty

Force TTY-like output (default true)

vcf package installed status

View status of app created by package install

Usage

```
package installed status INSTALLED_PACKAGE_NAME [flags]
```

Examples

Check status of package install

```
vcf package installed status sample-pkg-install
```

Flags

-h, --help

Help text

--namespace string

Specified namespace ($KCTRL_NAMESPACE or default from kubeconfig)

-tty

Force TTY-like output (default true)

vcf package installed update

Update package

Usage

```
vcf package installed update INSTALLED_PACKAGE_NAME [flags]
```

Examples

Upgrade package install to a newer version

```
vcf package installed update sample-pkg-install --version 1.0.0
```

Update package install with new values file

```
vcf package installed update sample-pkg-install --values-file values.yml
```

Update package install to stop consuming supplied values

```
vcf package installed update sample-pkg-install --values false
```

Flags

--dangerous-allow-use-of-shared-namespace

Allow use of shared namespaces

-h, --help

Help text

--namespace string

Specified namespace ($VCF_NAMESPACE or default from kubeconfig)

-p, --package string

Name of package install to be updated

-tty

Force TTY-like output

--values

Add or keep values supplied to package install, optional (default true)

--values-file string

The path to the configuration values file, optional

-v, --version string

Set package version (required)

--wait

Wait for reconciliation to complete (default true)

--wait-check-interval duration

Amount of time to sleep between checks while waiting (default 1s)

--wait-timeout duration

Maximum amount of time to wait in wait phase (default 30m0s)

--ytt-overlay-file string

Path to ytt overlay file (can also be a directory)

--ytt-overlays

Add or keep ytt overlays (default true)

vcf package repository

Manage package repositories

Syntax

```
vcf package repository [flags]
```

Flags

-h, --help

Help text

Available commands

- vcf package repository add
- vcf package repository delete
- vcf package repository get
- vcf package repository list
- vcf package repository update

vcf package repository add

Add a package repository

Usage

```
vcf package repository add REPOSITORY_NAME --url REPOSITORY_URL [flags]
```

Examples

Add a package repository

```
vcf package repository add sample-repo --url projects.repo.com/example:1.0.0
```

Flags

--create-namespace

Create the package repository namespace if not present (default false)

--dangerous-allow-use-of-shared-namespace

Allow use of shared namespaces

--dry-run

Print YAML for resources being applied to the cluster without applying them, optional -n,

-h, --help

Help text

--namespace string

Specified namespace ($VCF_NAMESPACE or default from kubeconfig)

-tty

Force TTY-like output

--url string

OCI registry url for package repository bundle (required)

--wait

Wait for reconciliation to complete (default true)

--wait-check-interval duration

Amount of time to sleep between checks while waiting (default 1s)

--wait-timeout duration

Maximum amount of time to wait in wait phase (default 5m0s)

vcf package repository delete

Delete a package repository

Usage

```
vcf repository delete REPOSITORY_NAME [flags]
```

Examples

Delete a package repository

```
vcf package repository delete sample-repo
```

Flags

-h, --help

Help text

--namespace string

Specified namespace ($VCF_NAMESPACE or default from kubeconfig)

-tty

Force TTY-like output

--wait

Wait for reconciliation to complete (default true)

--wait-check-interval duration

Amount of time to sleep between checks while waiting (default 1s)

--wait-timeout duration

Maximum amount of time to wait in wait phase (default 5m0s)

vcf package repository get

Get details for a package repository

Usage

```
vcf package repository get REPOSITORY_NAME [flags]
```

Examples

Get details for a package repository

```
vcf package repository get sample-repo
```

Flags

-h, --help

Help text

--namespace string

Specified namespace ($VCF_NAMESPACE or default from kubeconfig)

-o, --output string

Output format (yaml|json|table), optional

-tty

Force TTY-like output

vcf package repository list

List package repositories in a namespace

Usage

```
vcf package repository list [flags]
```

Examples

List package repositories

```
vcf package repository list
```

List package repositories in all namespaces

```
vcf package repository list -A
```

Flags

-A, --all-namespaces

List repositories in all namespaces

-h, --help

Help text

--namespace string

Specified namespace ($VCF_NAMESPACE or default from kubeconfig)

-o, --output string

Output format (yaml|json|table), optional

-tty

Force TTY-like output

vcf package repository update

Update a package repository

Usage

```
vcf package repository update REPOSITORY_NAME --url REPOSITORY_URL [flags]
```

Examples

Update a package repository with a new URL

```
vcf package repository update sample-repo --url projects.repo.com/example:1.0.0
```

Flags

-dangerous-allow-use-of-shared-namespace

Allow use of shared namespaces

-h, --help

Help text

--namespace string

Specified namespace ($VCF_NAMESPACE or default from kubeconfig)

-tty

Force TTY-like output

--wait

Wait for reconciliation to complete (default true)

--wait-check-interval duration

Amount of time to sleep between checks while waiting (default 1s)

--wait-timeout duration

Maximum amount of time to wait in wait phase (default 5m0s)

pais

Plugin for Private AI services platform.

Usage

CLI plugin: `pais` **| Primarily used for**: VCF CLI | Release Notes

Syntax

```
vcf pais [command]
```

Flags

-h, --help

Help text

vcf pais models

Manage models

Syntax

```
vcf pais models [command]
```

Flags

-h, --help

Help text

Available Commands

- vcf pais models delete
- vcf pais models delete-revision
- vcf pais models list
- vcf pais models list-revisions
- vcf pais models pull
- vcf pais models push

vcf pais models delete

Deletes a model (including all revisions) from the remote Model Store

Usage

```
vcf pais models delete --modelStore STORE --modelName NAME [flags]
```

Examples

Deletes a model with a model name

```
vcf pais models delete --modelStore harbor.corp/my-models --modelName meta-llama/llama-3-8b-instruct
```

Flags

-h, --help

Help text

-m, --modelName string

Model name (required)

-s, --modelStore string

Model store to use, e.g. myharbor.example.com/my-project (required, or set environment variable PAIS_MODELSTORE)

-y, --yes

Automatic 'yes' as answer to the interactive confirmation prompt.

vcf pais models delete-revision

Deletes a single revision of a model from the remote Model Store

Usage

```
vcf pais models delete-revision --modelStore STORE --modelName NAME {--tag TAG | --digest DIGEST} [flags]
```

Examples

Deletes a single revision of a model with a model name and tag

```
vcf pais models delete-revision --modelStore harbor.corp/my-models --modelName meta-llama/llama-3-8b-instruct
 --tag v1
```

Flags

-d, --digest string

Digest to identify a model revision

-h, --help

Help text

-m, --modelName string

Model name (required)

-s, --modelStore string

Model store to use, e.g. myharbor.example.com/my-project (required, or set environment variable PAIS_MODELSTORE)

-t, --tag string

Tag to identify a model revision (required)

-y, --yes

Automatic 'yes' as answer to the interactive confirmation prompt.

vcf pais models list

List the models in a remote Model Store

Usage

```
vcf pais models list --modelStore STORE [flags]
```

Examples

List the models from a store

```
vcf pais models list --modelStore harbor.corp/my-models
```

Flags

-h, --help

Help text

-s, --modelStore string

Model store to use, e.g. myharbor.example.com/my-project (required, or set environment variable PAIS_MODELSTORE)

vcf pais models list-revisions

List the revisions of a model

Usage

```
vcf pais models list --modelStore STORE [flags]
```

Examples

List the revisions of a model

```
vcf pais models list-revisions --modelStore harbor.corp/my-models --modelName meta-llama/llama-3-8b-instruct
```

Flags

-h, --help

Help text

-m, --modelName string

Model name (required)

-s, --modelStore string

Model store to use, e.g. myharbor.example.com/my-project (required, or set environment variable PAIS_MODELSTORE)

vcf pais models pull

Pull a model revision from a remote Model Store into the current working directory

Usage

```
vcf pais models pull --modelStore STORE --modelName NAME {--tag TAG | --digest DIGEST} [flags]
```

Examples

Pull a model revision from a remote Model Store

```
vcf pais models pull --modelStore harbor.corp/my-models --modelName meta-llama/llama-3-8b-instruct --tag v1
```

Flags

-d, --digest string

Digest to identify a model revision

-h, --help

Help text

-m, --modelName string

Model name (required)

-o, --outputDir string

Store model files here. Must be an existing empty directory. (default ".")

-s, --modelStore string

Model store to use, e.g. myharbor.example.com/my-project (required, or set environment variable PAIS_MODELSTORE)

-t, --tag string

Tag to identify a model revision (required)

vcf pais models push

Push a model revision from the current working directory into a remote Model Store

Usage

```
vcf pais models push --modelStore STORE --modelName NAME [flags]
```

Examples

Pull a model revision to remote Model Store

```
vcf pais models push --modelStore harbor.corp/my-models --modelName meta-llama/llama-3-8b-instruct --tag lat-
est
```

Flags

-h, --help

Help text

-m, --modelName string

Model name (required)

-s, --modelStore string

Model store to use, e.g. myharbor.example.com/my-project (required, or set environment variable PAIS_MODELSTORE)

-t, --tag string

Tag to identify a model revision (required)

-x, --exclude stringArray

Ignore files matching this glob pattern. May be repeated to ignore multiple patterns.

registry-secret

A plugin to manage registry secrets with v1/Secret resource of type `kubernetes.io/dockerconfigjson` in a workload cluster.

Usage

CLI plugin: `registry-secret` **| Primarily used for**: VCF CLI | Release Notes

This plugin works on a workload cluster.

For k8s contexts, create context with flags 'workload-cluster-name' and 'workload-cluster-namespace' and use the context with a format of 'ContextName:WorkloadClusterName'

For VCFA contexts, we need to create a context using the 'cluster kubeconfig' command before using it.

Example:

```
vcf cluster kubeconfig get cluster1
vcf context create your-namespace1-cluster1 --kubeconfig  ~/.kube/config --kubecontext vcf-cli-name-
space1-cluster1@namespace1-cluster1 --type k8s
vcf context use your-namespace1-cluster1
```

Syntax

```
vcf registry-secret [command]
```

Flags

-h, --help

Help text

--kubeconfig string

The path to the kubeconfig file, optional

--verbose int32

Number for the log level verbosity(0-9)

vcf registry-secret registry

Add, list, delete or update a registry secret. Registry secrets allow package and repository consumers to authenticate to and pull images from private registries

Syntax


```
vcf registry-secret registry [command]
```

Flags

-h, --help

Help text

-n, --namespace string

Target namespace for the registry secret, optional (default "default")

Available commands

- vcf registry-secret registry add
- vcf registry-secret registry delete
- vcf registry-secret registry list
- vcf registry-secret registry update

vcf registry-secret registry add

Create a v1/Secret resource of type kubernetes.io/dockerconfigjson

Usage

```
vcf registry-secret registry add SECRET_NAME --server REGISTRY_SERVER --username USERNAME --password PASSWORD
 [flags]
```

Examples

Add a registry secret

```
vcf registry-secret registry add test-secret --server projects.registry.broadcom.com --username test-user --
password-file test-file
```

Add a registry secret with 'export-to-all-namespaces' flag being set

```
vcf registry-secret registry add test-secret --server projects.registry.broadcom.com --username test-user --
password-file test-file --export-to-all-namespaces
```

Flags

--export-to-all-namespaces

Make the registry secret available across all namespaces (i.e. create SecretExport with toNamespace=*), optional

-h, --help

Help text

--password string

Password for authenticating to the private registry

--password-env-var string

Environment variable containing the password for authenticating to the private registry

--password-file string

File containing the password for authenticating to the private registry

--password-stdin

When provided, password for authenticating to the private registry would be taken from the standard input

--server string

Private registry server FQDN

--username string

Username for authenticating to the private registry

-y, --yes

In case the --export-to-all-namespaces flag was set, export the secret to all namespaces without asking for confirmation, optional

vcf registry-secret registry delete

Deletes a v1/Secret resource of type kubernetes.io/dockerconfigjson

Usage

```
vcf registry-secret registry delete SECRET_NAME [flags]
```

Examples

Delete a registry secret

```
vcf registry-secret registry delete test-secret
```

Flags

-h, --help

Help text

-y, --yes

Delete the registry secret without asking for confirmation, optional

vcf registry-secret registry list

Lists all v1/Secret of type kubernetes.io/dockerconfigjson

Usage

```
vcf registry-secret registry list [flags]
```

Examples

List registry secrets across all namespaces

```
vcf registry-secret registry list -A
```

List registry secrets from specified namespace

```
vcf registry-secret registry list -n test-ns
```

List registry secrets in json output format

```
vcf registry-secret registry list -n test-ns -o yaml
```

Flags

-A, --all-namespaces

If present, list registry secrets across all namespaces, optional

-h, --help

Help text

-o, --output

string Output format (yaml|json|table), optional

vcf registry-secret registry update

Updates the v1/Secret resource of type kubernetes.io/dockerconfigjson

Usage

```
vcf registry-secret registry update SECRET_NAME --username USERNAME --password PASSWORD [flags]
```

Examples

Update a registry secret. There will be no changes in the associated SecretExport resource

```
vcf registry-secret registry update test-secret --username test-user --password-file test-file
```

Update a registry secret with 'export-to-all-namespaces' flag being set

```
vcf registry-secret registry update test-secret --username test-user --password-file test-file --export-to-all-namespaces
```

Update a registry secret with 'export-to-all-namespaces' flag being clear. In this case, the associated SecretExport resource will get deleted

```
vcf registry-secret registry update test-secret --username test-user --password-file test-file --export-to-all-namespaces=false
```

Flags

--export-to-all-namespaces[=true]

If set to true, the secret gets available across all namespaces

-h, --help

Help text

--password string

Password for authenticating to the private registry

--password-env-var string

Environment variable containing the password for authenticating to the private registry

--password-file string

File containing the password for authenticating to the private registry

--password-stdin

When provided, password for authenticating to the private registry would be taken from the standard input

--username string

Username for authenticating to the private registry

-y, --yes

In case the --export-to-all-namespaces flag was provided, export/un-export of the secret will be performed without asking for confirmation, optional

secret

Plugin for managing secrets in a secret store service

Usage

CLI plugin: `secret` **| Primarily used for:** VCF CLI

Syntax

```
vcf secret [command]
```

Flags

-h, --help

Help text

--verbose int32

Sets the log level between 0 to 9 for this command invocation

vcf secret create

Create a secret

Usage

```
vcf secret create [flags]
```

Flags

-f, --file string

Yaml containing the request body required for the secret. This field is mandatory.

Sample secret yaml file -

```
kind: KeyValueSecret
apiVersion: secretstore.vmware.com/v1alpha1
metadata:
  name: db-cred
spec:
  name: db-cred
  data:
    - key: foo
      value: bar
    - key: foo1
      value: bar1
```

-h, --help

Help text

vcf secret delete

Delete a secret by name

Usage

```
vcf secret delete <secret-name> [flags]
```


Flags

-h, --help

Help text

-y, --yes

Confirm deletion without prompt

vcf secret get

Get a secret by name

Usage

```
vcf secret get <secret-name> [flags]
```

Flags

-h, --help

Help text

-o, --output string

output format: json, yaml (default) (default "yaml")

vcf secret list

List secrets

Usage

```
vcf secret list [flags]
```

Flags

-h, --help

Help text

-o, --output string

output format: json, yaml , table (default "yaml")

vcf secret update

Update a secret

```
vcf secret update [flags]
```

Flags

-f, --file string

Yaml containing the request body required for the secret. This field is mandatory

-h, --help

Help text

telemetry

Plugin to manage Telemetry configurations for CLI

Usage

CLI plugin: `telemetry` **| Primarily used for:** VCF CLI | Release Notes

Syntax

```
vcf telemetry [command]
```

Flags

-h, --help

Help text

--verbose int32

Sets the log level between 0 to 9 for this command invocation

Available Command

- vcf telemetry cli-usage-analytics collect
- vcf telemetry cli-usage-analytics status
- vcf telemetry cli-usage-analytics update

vcf telemetry cli-usage-analytics collect

Collects telemetry data and uploads to the central telemetry

Usage

```
vcf telemetry cli-usage-analytics collect [FLAGS]
```

Flags

-h, --help

Help text

-q, --quiet

disables logging for this command

-t, --timeout int

timeout in seconds for collect command (default 2)

vcf telemetry cli-usage-analytics status

Prints status of local telemetry configuration

Usage

```
vcf telemetry cli-usage-analytics status [FLAGS]
```

Flags

-h, --help

Help text

vcf telemetry cli-usage-analytics update

Opt-in or Opt-out of Broadcom's Customer Experience Improvement Program (CEIP)

Usage

```
vcf telemetry cli-usage-analytics update [FLAGS]
```

Examples

opt into Customer Experience Improvement Program (CEIP)

```
vcf telemetry cli-usage-analytics update --opt-in
```

opt out of Customer Experience Improvement Program (CEIP)

```
vcf telemetry cli-usage-analytics update --opt-out
```

Flags

-h, --help

Help text

--opt-in

opt into collection of local CLI telemetry

--opt-out

opt out of collection of local CLI telemetry

vm

Plugin to manage virtual machines

Usage

CLI plugin: `vm` **| Primarily used for:** VCF CLI | Release Notes

Syntax

```
vcf vm [command]
```

Flags

-h, --help

Help text

--verbose int32

Sets the log level between 0 to 9 for this command invocation

vcf vm web-console

Usage

```
vcf vm web-console [flags]
```

Example

To request a URL which can be opened in a web browser to access the web console of virtual machine with the name ***my-vm-name***.

```
vcf vm web-console my-vm-name
```

Flags

-h, --help

Help text

--short

Output the result in a shorter message

Access the IaaS Services Console or Local Consumption Interface

Depending on how your environment is configured, you can use vSphere Supervisor services from either the IaaS Services Console in VCF Automation or the Local Consumption Interface.

If you have access to the vSphere Client, you can log in to the Local Consumption Interface through the vSphere Client. Or your administrator can provide you with a link to log in to the standalone Local Consumption Interface.

Log in to the interface you are using to deploy services.

Option	Description
VCF Automation IaaS Services Console	1. Log in to VCF Automation. 2. Click the **Build and Deploy** tab. 3. In the **Services** section, click the **Overview** tab. 4. If multiple namespaces are available to you, select the namespace you want to use.
Local Consumption Interface through the vSphere Client	1. Log in to vCenter using the vSphere Client. 2. Navigate to **Supervisor Management**. 3. If multiple namespaces are available to you, select the namespace you want to use. 4. Click the **Resources** tab.
Standalone Local Consumption Interface	Go to the link that your administrator has provided for the Local Consumption Interface, and log in.

Provision and Manage Virtual Machines

You can use a variety of methods to provision VMs with the VM Service in vSphere Supervisor.

If you have access to VCF Automation for All Apps, the Iaas Services Console provides a graphical user interface for provisioning VMs using the VM Service.

If the Supervisor Admin has deployed the Local Consumption Interface, the Local Consumption Interface also provides a graphical user interface for provisioning VMs.

In addition, the VCF CLI and kubectl provide command-line interfaces for provisioning VMs. If you have the access to VCF Automation for All Apps, use VCF CLI and `kubectl` in the namespace of your organization, provided to by your organization administrator.

What to Read Next

- Provision a VM Using Self-Service
- Deploying and Managing Virtual Machines in vSphere Supervisor

Provision a VM Using Self-Service

You can provision VMs using the the VM Service in vSphere Supervisor.

- Verify that you have access to a namespace.
- The namespace must have access to the content library that contains the VM image.

If you have access to VCF Automation for All Apps, the Iaas Services Console provides a graphical user interface for provisioning VMs using the VM Service. As you go through the configuration procedure, the Kubernetes resource YAML manifest is generated in real time on the right.

If the Supervisor Admin has deployed the Local Consumption Interface, the Local Consumption Interface also provides a graphical user interface for provisioning VMs.

1. Log in to the interface you are using by following the steps in Access the IaaS Services Console or Local Consumption Interface.
2. On the **Virtual Machine Service** card, click **Create VM**.

   Alternatively, click **Go to Service**, then, on the Virtual Machines tab, click **Create VM**.
3. Select the source for the VM.

   The configuration options depend on what file type you choose. This procedure uses the OVF file type.
4. Click **Next**.
5. Configure VM class and storage class.

Option	Description
Zone	You have the following options: • Automatic. A zone is automatically selected based on the VM requirements. You cannot add storage volume later. • Select a specific zone, in which case you can add storage later.
VM Image	Select the image you want to provision. The VM images available for selection are pulled from the content library associated with the namespace.
VM Class	`best-effort-xsmall`
Storage Class	`global-storage-profile` The options that are available to you depend on what destinations your organization administrator defined.
Power State	Select if you want the VM to be initially powered on or off.

6. Click **Next** to configure an advanced VM.

   If you are provisioning a basic VM, you can skip steps 7 and 8 and click **Review and Confirm** to finalize your configuration.
7. Configure advanced settings.

Option	Description
Persistent Volume	Click **Attach Volume**. The volumes have their own life cycle and are managed independently from the VM. For example, if you provisioned a MySQL database and then you delete the VM, you can reattach the database to another VM. If you selected automatic zone selection in step 8, you cannot attach additional volumes.
Load Balancer	Click **Add** and select **New** or **Existing**.

Option	Description
Guest Customization	• **Cloud-init**. Recommended for Linux. • **Sysprep**. Recommended for Windows. • **Linuxprep**. Focused on VMware VM. • **vAppConfig**. OVA and OVF formats.

8. Configure networking for the VM.

Option	Description
Add Network Interface	Subnet set available for the VM.

9. On the **Review and Confirm** tab, click **Deploy VM**.

   Consider copying the Kubernetes resource YAML manifest to clipboard or downloading it as a .zip file. You can reuse the manifest to provision VMs with the same or modified configuration from the command line. You can also store the manifest in a source control repository like GitHub.

The provisioning process begins and the Virtual Machines tab on the Virtual Machine Service page opens with your current request at the top.

Review your provisioned VM.

1. Click the VM name to open the VM Details page, where you can perform day 2 operations on the VM like powering it off or on, attaching additional disk volumes, increasing volume capacity, or detaching volumes from the VM.
2. Click the double arrows next to the VM name to expand the details view.
3. Click View YAML to view the active configuration of the VM Kubernetes object. You can edit certain fields in the manifest directly. For example, if the VM was Powered On in the original configuration, you can change the `powerState` to PoweredOff.
4. Click the vertical ellipsis next to the VM name to view your options, such as opening the remote or web console or deleting the VM.

Deploying and Managing Virtual Machines in vSphere Supervisor

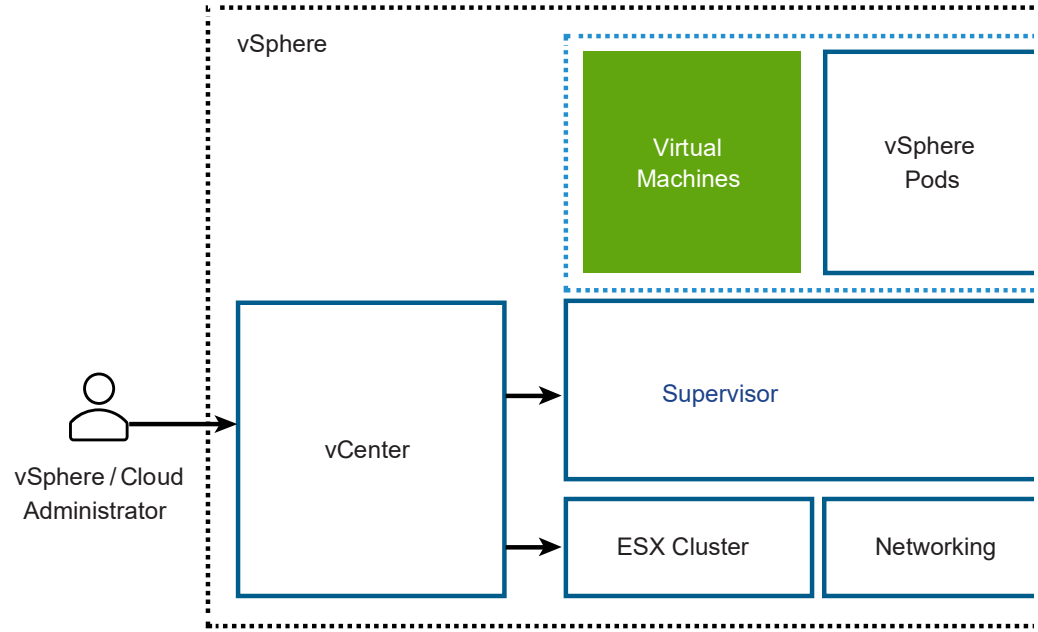
With the VM Service functionality, DevOps engineers can deploy and run VMs, in addition to containers, in a common, shared Kubernetes environment. You can use the VM Service to manage the lifecycle of virtual machines in a namespace. VM Service manages stand-alone VMs and VMs that make up VKS clusters.

The VM Service addresses the needs of DevOps teams that use Kubernetes, but have existing VM-based workloads that cannot be easily containerized. It also helps users reduce the overhead of managing a non-Kubernetes platform alongside a container platform. When running containers and VMs on a Kubernetes platform, DevOps teams can consolidate their workload footprint to just one platform.

Note:

In addition to stand-alone VMs, the VM Service manages the VMs that make up VKS clusters. For information about the clusters, Managing vSphere Kubernetes Service Clusters and Workloads.

Figure 251: VM Service Architecture



Each VM deployed through the VM Service functions as a complete machine running all the components, including its own operating system, on top of the Supervisor infrastructure. The VM has access to networking and storage that the Supervisor provides, and is managed using the standard Kubernetes `kubectl` command. The VM runs as a fully isolated system that is immune to interference from other VMs or workloads in the Kubernetes environment.

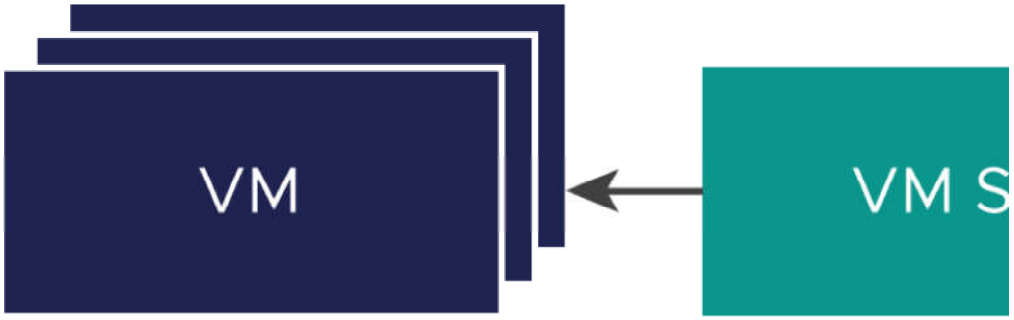
When to Use Virtual Machines on a Kubernetes Platform?

Generally, a decision to run workloads in a container or in a VM depends on your business needs and goals. Among the reasons to use VMs appear the following:

- Your applications cannot be containerized.
- You have specific hardware requirements for your project.
- Applications are designed for a custom kernel or custom operating system.
- Applications are better suited to running in a VM.
- You want to have a consistent Kubernetes experience and avoid overhead. Rather than running separate sets of infrastructure for your non-Kubernetes and container platforms, you can consolidate these stacks and manage them with a familiar `kubectl` command.

Concepts of VM Service

To describe the state of a VM to be deployed in a vSphere Namespace, you use such parameters as a VM class, a VM image, and a storage class. The VM Service then brings together these specifications to create stand-alone VMs or VMs that support VKS clusters.



VM Service

The VM Service provides a declarative, Kubernetes-style API for management of VMs and associated vSphere resources. The VM Service enables the vSphere administrators to deliver resources and provide templates, such as VM classes and VM images,

to Kubernetes. DevOps engineers can use these resources to describe the desired state of a VM. After the DevOps engineers specify the VM state, the VM Service converts the desired state in to a realized state against backing infrastructure resources.

A VM created through the VM Service can be managed only from the Kubernetes namespace with the `kubectl` commands. vSphere administrators cannot manage the VM from the vSphere Client, but can display its details and monitor resources it uses. For information, see Monitor Virtual Machines Available in vSphere Supervisor.

VM Class

The VM class is a VM specification that can be used to request a set of resources for a VM. The VM class is controlled and managed by a vSphere administrator, and defines such parameters as the number of virtual CPUs, memory capacity, and reservation settings. The defined parameters are backed and guaranteed by the underlying infrastructure resources of a Supervisor.

A vSphere administrator can create custom VM classes.

In addition, the Supervisor offers several default VM classes. Generally, each default class type comes in two editions: guaranteed and best effort. A guaranteed edition fully reserves resources that a VM specification requests. A best effort class edition does not and allows resources to be overcommitted. Typically, a guaranteed type is used in a production environment.

Examples of default VM classes include the following.

Class	CPU	Memory (GB)	Reserved CPU and Memory
`guaranteed-large`	4	16	Yes
`best-effort-large`	4	16	No
`guaranteed-small`	2	4	Yes
`best-effort-small`	2	4	No

The vSphere administrator can assign any number of existing VM classes to make them available to DevOps engineers within a specific namespace.

The VM class provides a simplified experience for the DevOps engineers. The DevOps don't need to understand the full configuration of each VM that they plan to create. Instead, they can select a VM class from available options, and the VM Service manages the VM configuration.

On the Kubernetes side, the VM classes appear as `VirtualMachineClass` resource.

VM Image

A VM image is a template that contains a software configuration, including an operating system, applications, and data.

	When DevOps engineers create VMs, they can select images from the content library associated with the namespace. To the DevOps, the images are exposed as `VirtualMachineImage` objects. A vSphere administrator can create VM images that are compatible with vSphere and upload them to a content library.
Content Library	A DevOps engineer uses a content library as a source of images to create a VM. Similar to VM classes, a vSphere administrator assigns existing content libraries to a namespace or a cluster to make them available to DevOps engineers. The vSphere administrator can also make the namespace content library writable. This additional permission allows DevOps users to publish their images to the library.
Storage Class	VM Service uses storage classes for placing virtual disks and dynamically attaching persistent volumes. For more information about storage classes, see Persistent Storage for Workloads.
VM Specification	DevOps engineers describe the desired state of a VM in a YAML file that brings together the VM image, the VM class, and the storage class.
VM Operator for Kubernetes	VM operator enables management of virtual machines with a Kubernetes-style, declarative API. **Note:** Starting with VCF 9.0, the v1alpha1 version of the VM Operator API is deprecated. Use v1alpha2 or v1alpha3 instead, as both are supported on Supervisor and are fully backward compatible. Support for v1alpha1 will be phased out. For information about the VM Operator API, see https://vm-operator.readthedocs.io/en/latest/ref/api.
Networking	VM Service does not have any specific requirements and relies on the network configuration available in the Supervisor. VM Service supports both types of networking, the vSphere networking or NSX. When VMs are deployed, an available network provider allocates static IP addresses to the VMs. For information, see the Supervisor documentation.

VM Service and Supervisor with vSphere Zones

When you create VMs on a three-zone Supervisor, the VM instance is replicated across all available zones. To control the VMs placement through the YAML file, the DevOps team can use the Kubernetes label `topology.kubernetes.io/zone`. For example, `topology.kubernetes.io/zone: zone-a02`.

Workflow for Provisioning and Monitoring a VM

As a vSphere administrator, you set guardrails for the policy and governance of the VMs, and deliver VM resources, such as VM classes and VM templates to DevOps engineers. After a VM is deployed, you can monitor it using the vSphere Client.

Step	Performed by	Description
1	vSphere administrator	Create content libraries and assign them to a namespace or Supervisor

Step	Performed by	Description
2	vSphere administrator	Create a VM class and assign it to a namespace To use NVIDIA vGPU, configure a PCI device in the VM class. See Deploying a VM with vGPU and Other PCI Devices in a Supervisor.
3	DevOp engineer	Provision a VM in a Kubernetes namespace For VKS cluster VMs, see the VKS cluster documentation.
4	vSphere administrator	Monitor deployed VMs
5	DevOps engineer	Publish a VM image to a content library

Creating and Managing Content Libraries for Stand-Alone VMs in vSphere Supervisor

To deploy virtual machines in the Supervisor environment, DevOps users must have access to VM images, or templates, that contain software configurations, including operating systems, applications, and data. To provide access to images, a vSphere administrator configures a VM content library, and then associates it with the namespace where the VMs are deployed. Starting with vSphere 8.0 Update 2, the vSphere administrator can also assign the content library at a Supervisor level, so that it becomes available to all namespaces.

Create a Content Library for Stand-Alone VMs in the Supervisor Environment

As a vSphere administrator, create a content library to store and manage VM templates.

Create a Content Library for Stand-Alone VMs
You can create a local content library and populate it with templates and other types of files. You can also create a subscribed library to use the contents of an already existing published local library.

Required privileges:

- **Content library** > **Create local library** or**Content library** > **Create subscribed library** on the vCenter instance where you want to create the library.
- **Datastore** > **Allocate space** on the destination datastore.

To protect the items of a content library, you can apply an OVF security policy. The OVF security policy enforces strict validation when you deploy or update a content library, import items to a content library, or synchronize templates. To make sure that the templates are signed by a trusted certificate, you can add the OVF signing certificate from a trusted CA to a content library.

For more information about content libraries and VM templates in vSphere, see *Using Content Libraries* in *vSphere Virtual Machine Administration*.

1. Navigate to the **VM Service** page.
   a) From the vSphere Client home menu, select **Supervisor Management**.
   b) Click the **Services** tab and click **Manage** on the **VM Service** card.
2. On the **VM Service** page, click **Content Libraries** > **Create a content library**.
   This action takes you to the content library section in the vSphere Client.
3. Click **Create**.
   The **New Content Library** wizard opens.
4. On the **Name and location** page, enter a name, select a vCenter instance for the content library and click **Next**.
   Make sure to use an informative name for the content library, so that your DevOps team can easily find and access the library items.

5. On the **Configure content library** page, select the type of content library that you want to create and click **Next**.

Option	Description
Local content library	A local content library is accessible only in the vCenter instance where you create it by default. 1. To make the content of the library available to other vCenter instances, select **Enable publishing**. 2. If you want to require a password for accessing the content library, select **Enable authentication** and set a password.
Subscribed content library	A subscribed content library originates from a published content library. Use this option to take advantage of existing content libraries. You can synchronize the subscribed library with the published library to see up-to-date content, but you cannot add or remove content from the subscribed library. Only an administrator of the published library can add, modify, and remove contents from the published library. Provide the following information to subscribe to a library: 1. In the **Subscription URL** text box, enter the URL address of the published library. 2. If authentication is enabled on the published library, select **Enable authentication** and enter the publisher password. 3. Select a download method for the contents of the subscribed library. – If you want to download a local copy of all the items in the published library immediately after subscribing to it, select **immediately**. – If you want to save storage space, select **when needed**. You download only the metadata for the items in the published library. If you need to use an item, synchronize the item or the entire library to download its content. 4. If prompted, accept the SSL certificate thumbprint. The SSL certificate thumbprint is stored on your system until you delete the subscribed content library from the inventory.

6. Optional: On the **Apply security policy** page, select **Apply Security Policy** and select **OVF default policy**.

   For the subscribed library, this option appears only if the library supports security policies.

   If you select this option, the system performs a strict OVF certificate verification when importing an OVF item to the library from the local host or synchronizing an item. The OVF items that do not pass the certificate validation cannot be imported.

   If the item does not pass the validation during synchronization, it is marked with the **Verification Failed** tag. Only the item and metadata will be kept, but not the files in the item.

7. On the **Add storage** page, select datastore as a storage location for the content library contents and click **Next**.

8. On the **Ready to complete** page, review the details and click **Finish**.

Populate a Content Library with VM Images for Stand-Alone VMs

After you create the content library, populate it with VM templates in OVA or OVF format. Your DevOps engineers can use the templates to provision new stand-alone virtual machines.

- Create VM images that are compatible with the vSphere Supervisor environment.

  The image specification requires that all VM images include VMware Tools or an equivalent open source package. The images must use one of the following to bootstrap the guest OS and its networking stack. For more information, see Bootstrap Providers.

  – Linux + Cloud-Init version 17.9-21.2 with DataSourceVMwareGuestInfo.
  – Linux + Cloud-Init version 21.3+
  – Windows + Cloudbase-Init version 1.1.0+
  – Windows + Sysprep (System Preparation)

  For information about Cloud-Init, see the official documentation at The standard for customising cloud instances.

  For information about Sysprep, see the official documentation at Sysprep overview.
- If your library is protected by a security policy, make sure that all library items are complaint. If a protected library includes a mix of compliant and non-compliant items, the `kubectl get virtualmachineimages` fails to present VM images to the DevOps engineers.
- Required privilege:**Content library** > **Add library item** and**Content library** > **Update files** on the library.

You can use several methods to populate the library. This topic describes how to add items to a local content library by importing files from your local machine or from a Web server. For other ways to populate the content library, see *Populating Libraries with Content* in *vSphere Virtual Machine Administration*.

Note: There are no restrictions on the VM images that you use. If you want to test ready-to-use OVA images, you can download them from the Recommended Images page. Keep in mind that these images are for POC use only. In the production environment, create images with latest patches and required security settings that follow corporate security policies.

1. From the vSphere Client home menu, select **Content Libraries**.
2. Right-click a local content library and select **Import Item**.

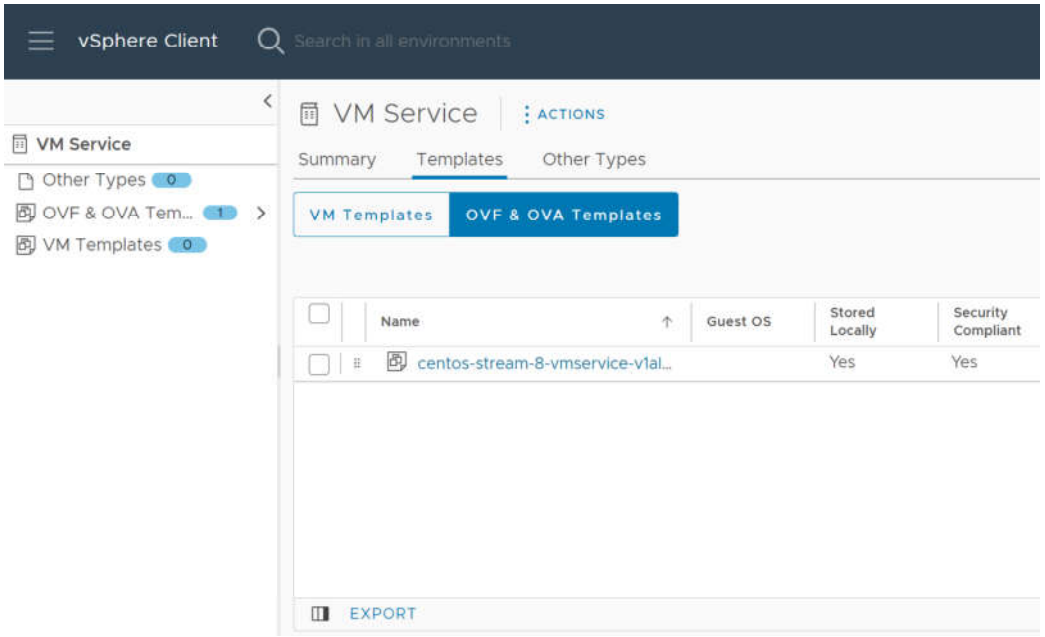
   The **Import Library Item** dialog box opens.
3. In the **Source** section, select the source of the item.

Option	Description
URL	Enter the path to the Web server where the item is. **Note:** You can import either an `.ovf` or `.ova` file. The resulting content library item is of the OVF Template type.
Local File	Click **Upload File** to navigate to the file that you want to import from your local system. You can use the drop-down menu to filter files in your local system. **Note:** You can import either an `.ovf` or `.ova` file. When you import an OVF template, first select the OVF descriptor file (`.ovf`). Next, you are prompted to select the other files in the OVF template, for example the `.vmdk` file. The resulting content library item is of the OVF Template type.

   vCenter reads and validates the manifest and certificate files in the OVF package during importing. A warning is displayed in the **Import Library Item** wizard, if certificate issues exist, for example if vCenter detects an expired certificate.

   Note: vCenter does not read signed content, if the OVF package is imported from an `.ovf` file from your local machine.
4. In the **Destination** section, enter a name and a description for the item.
5. Click **Import**.

The item appears on the **Templates** tab or on the **Other Types** tab.



Add and Manage VM Content Libraries in vSphere Supervisor

After you create the content library and populate it with VM templates, use the vSphere Client to add the library to the namespace. By adding the library to the namespace, you give your DevOps users access to the library. In addition, you can use the Data Center CLI (DCLI) commands to add a writable or read-only content library to the namespace, or assign a read-only library at a cluster level.

Add a VM Content Library to a Namespace Using the vSphere Client
The content library that you add with the vSphere Client is read-only. The DevOps users can access images from this content library, but cannot publish VM images to this library.

Required privileges:

- **Namespaces** > **Modify cluster-wide configuration**
- **Namespaces** > **Modify namespace configuration**

You can add multiple content libraries to a single namespace. You can add the same content library to different namespaces.

Note:

This procedure applies only to content libraries for VM Service. A content library for VKS must be managed from the VKS card.

1. In the vSphere Client, go to the namespace.
   a) From the home menu, select **Supervisor Management**.
   b) Click the **Namespaces** tab and click the namespace.
2. Add a content library.
   a) On the **VM Service** card, click **Add Content Library**.
   b) Select one or several content libraries and click **OK**.

Manage VM Content Libraries on a Namespace Using the vSphere Client
After you associate the library with the namespace, you can use the vSphere Client to remove it from the namespace. You can also add more libraries.

Required privileges:

- **Namespaces** > **Modify cluster-wide configuration**
- **Namespaces** > **Modify namespace configuration**

Removing a content library from a namespace does not affect VMs that were previously deployed with the library images.
Note:

This procedure applies only to content libraries for VM Service. VKS content libraries must be managed from the VKS card.

1. In the vSphere Client, go to the namespace.
   a) From the home menu, select **Supervisor Management**.
   b) Click the **Namespaces** tab and click the namespace.
2. Add or remove a content library.
   a) On the **VM Service** card, click **Manage Content Library**.
   b) Perform one of the following operations.

Remove a content library	Deselect the content library and click **OK**.
Add a content library	Select one or several content libraries and click **OK**.

OVF templates from the library become available in the Kubernetes namespace as VM images and can be used by DevOps to self-service VMs. See Deploy a Virtual Machine on a Namespace in a Supervisor.

Note: Only OVF templates from the library show up in the namespaces. Other types of content do not show up in the namespace.

Add a VM Content Library to a Namespace Using Data Center CLI
As a vSphere administrator, you can use the Data Center CLI (DCLI) command to assign the content library to a namespace. When assigning the library, you can make the library associated with the namespace writable. When the library is writable, in addition to viewing the library and the images in the library, DevOps users can publish new VM images to it.

With the DCLI commands, you can add any type of the library, including local, published, and subscribed, to the namespace. However, only local and published can be linked as writable libraries.? Content libraries and library items are available only in the associated namespace?.

1. Log in to vCenter using the root user account.
2. Type `dcli +i` to use the DCLI in interactive mode.
3. Obtain the ID of the content library to be associated with the namespace.
   ```
   dcli > namespacemanagement content library list
   ```

4. Run the following command to associate the content library with the namespace.

   The update operation is not incremental. Only libraries specified in the list will be associated with the namespace and the libraries that were added previously will be removed, unless their IDs are specified. For example, if you update `'[{"content_library": "CLA", "writable": "true"}]'` and then later update `'[{"content_library": "CLB", "writable": "true"}]'`, CLA will be removed and only CLB will be added. If you want both CLA and CLB to be associated, you have to specify both libraries: `'[{"content_library": "CLA", "writable": "true"}, {"content_library": "CLB", "writable": "true"}]'`.

   ```
   dcli > namespaces instances update --namespace namespace_name --content-libraries '[{"content_library":
     "content_library_ID", "writable": "true | false"}]'
   ```

   Use the following arguments:

   - `--namespace` *`namespace_name`* – Name of the namespace.
   - `--content_libraries` *`content_library_ID`* `writable: true | false` – ID of the content library to be associated with the namespace and whether the library is writable or not.

   For example,

   ```
   dcli > namespaces instances update --namespace lb-edit-ns --content-libraries '[{"content_library": "cl-
   b585915ddxxxxxxxx", "writable": "true"}]'
   ```

5. To delete the content library from the namespace, repeat the `namespaces instances update` command removing the content library entry from the array list.

   For example,

   ```
   dcli > namespaces instances update --namespace lb-edit-ns --content-libraries '[]'
   ```

The added content library becomes available in the DevOps namespace view.
The DevOps user can run the following commands to verify that the content library has been added or deleted.

```
kubectl get cl -n lb-edit-ns
   NAMESPACE    NAME                  VSPHERENAME   TYPE    WRITABLE   STORAGETYPE   AGE
   lb-edit-ns   cl-b585915ddxxxxxxxx  Test-ns-cl    Local   true       Datastore     3m9s
kubectl describe cl cl-b585915ddxxxxxxxx -n lb-edit-ns
kubectl get clitem -n lb-edit-ns
```

Add a VM Content Library to Supervisor Using Data Center CLI
In addition to assigning the content library at a namespace level, the vSphere administrator can use the Data Center CLI (DCLI) command to associate the library with a Supervisor cluster. The content library becomes available to all namespaces in the Supervisor.

For more information about the DCLI commands, see VMware Data Center CLI.

You can associate all types of libraries, including local, published, and subscribed.

Note: The content library associated with the Supervisor is read-only. The DevOps users can only access VM images from this content library, but cannot publish VM images to this library.

1. Log in to vCenter using the root user account.
2. Type `dcli +i` to use the DCLI in interactive mode.
3. Obtain the name of the Supervisor and the ID of the content library to connect to the Supervisor.
   a) Get the name of the Supervisor from the list of clusters.

      The command lists all clusters available on vCenter.

      ```
      dcli > namespacemanagement clusters list
      ```

   b) List IDs of all content libraries of any type available on vCenter.

      ```
      dcli > library list
      ```

   c) Verify details for the specific library.

      ```
      dcli > library get --library-id content_library_ID
      ```

4. Associate one or more content libraries with the Supervisor.

   The update operation is not incremental. Only libraries specified in the list will be associated with the namespace and the libraries that were added previously will be removed, unless their IDs are specified. For example, if you update `'[{"content_library": "CLA", "writable": "true"}]'` and then later update `'[{"content_library": "CLB", "writable": "true"}]'`, CLA will be removed and only CLB will be added. If you want both CLA and CLB to be associated, you have to specify both libraries: `'[{"content_library": "CLA", "writable": "true"}, {"content_library": "CLB", "writable": "true"}]'`.

   ```
   dcli > namespacemanagement clusters update --cluster cluster_name --content-libraries '[{"content_li-
   brary": content_library_ID_1}, {"content-library": content_library_ID_2}]'
   ```

   Use the following arguments:

   - `--cluster` *`cluster_name`* – Identifier for the Supervisor cluster.
   - `--content-libraries` *`content_library_ID`* – An ID of a content library to be associated with the Supervisor. You can list several IDs.

     For example,

     ```
     dcli > namespacemanagement clusters update --cluster cluster_name --content-libraries '[{"con-
     tent_library": 535d4b3d-xxxx-xxxx-xxxx-xxxxxxxxxxxx}, {"content-library": b5aa7f68-xxxx-xxxx-xxxx-
     xxxxxxxxxxxx}]'
     ```

5. Verity that the content libraries are connected to the cluster.

   ```
   dcli > namespacemanagement clusters get --cluster cluster_name
   ```

   The output must include the IDs of the connected content libraries.

6. To delete the associated content library from the cluster, repeat the `namespacemanagement clusters update` command removing the content library entry from the content library array list.

   For example,

   ```
   dcli > namespacemanagement clusters update --cluster cluster_name --content-libraries '[]'
   ```

The newly added content libraries become available in the DevOps cluster view. Any changes that the vSphere administrator makes to the content libraries are reflected in the DevOps view. The DevOps user can run the following commands to list the content libraries and describe their contents:

- `kubectl get ccl` – List of all content libraries available at the cluster level. The output can look similar to the following.

  ```
  NAME                  VSPHERENAME                          TYPE         STORAGETYPE   AGE
  cl-f28af8153fb849bd7  Kubernetes Service Content Library    Subscribed   Datastore     6d5h
  cl-knounwp7xxxxxxxxx  Image Registry Content Library        Local        Datastore     6d4h
  ```

- `kubectl get cclitem` – List of all items in the content libraries at the cluster level.
- `kubectl describe ccl` *`NAME`* – Detailed information for a specific content library at the cluster level.

Manage and Publish Content Library Images in vSphere Supervisor

After a vSphere administrator assigns content libraries to a namespace or cluster, DevOps users can access the library and use its items to deploy VMs from VM images in the library. If the library assigned to the namespace is writable, the DevOps users with Edit permissions can also manage the library items and publish new VM images.

As a DevOps user, make sure that you follow these requirements:

- You have `Edit` permissions on the vSphere Namespace.
- The vSphere administrator has assigned a writable content library to the namespace. See Add a VM Content Library to Supervisor Using Data Center CLI .
- The content library is local or published. Subscribed libraries cannot be edited.

Note: There are no restrictions on the VM images that you use. If you want to test ready-to-use OVA images, you can download them from the Recommended Images page. Keep in mind that these images are for POC use only. In the production environment, create images with latest patches and required security settings that follow corporate security policies.

1. Manage the library items.
   a) Verify that the content libraries are available in the namespace.

      Note: If you want to manage library items in the library or publish VM images to the library, make sure that its writable status is true.

      ```
      kubectl get cl -n <namespace-name>
         NAME                  VSPHERENAME   TYPE    WRITABLE   STORAGETYPE   AGE
         cl-b585915ddxxxxxxxx  Test-ns-cl-1  Local   true       Datastore     3m9s
         cl-535d4b3dnxxxyyyyy  Test-ns-cl-1  Local   false      Datastore     3m9s
      ```

   b) Check the contents of the library.

      ```
      kubectl get clitem -n <namespace-name>
       NAME                VSPHERENAME     CONTENTLIBRAYREF        TYPE   READY   AGE
       clitem-d2wnmq.....  item 1          cl-b585915ddxxxxxxxx    Ovf    True    26c
       clitem-55088d.....  item 2          cl-b585915ddxxxxxxxx    Ovf    True    26c
       clitem-xyzxyz.....  xyzxyz          cl-535d4b3dnxxxyyyyy    Ovf    True    26c
      ```

   c) Delete an image from the content library.

      Note: You can delete an item only from the library that is writable and if you have `Edit` permissions or permissions of a higher level.

      Once you delete the clitem, the corresponding vmi resource is also deleted.

      ```
      kubectl delete clitem clitem-55088d.....
       NAME                VSPHERENAME     CONTENTLIBRAYREF        TYPE   READY   AGE
       clitem-d2wnmq.....  item 1          cl-b585915ddxxxxxxxx    Ovf    True    26c
       clitem-xyzxyz.....  xyzxyz          cl-535d4b3dnxxxyyyyy    Ovf    True    26c
      ```

   d) Get image details.

      ```
      kubectl get vmi -n <namespace-name>
       NAME              PROVIDER-NAME      CONTENT-LIBRAY-NAME    IMAGE-NAME VERSION   OS-TYPE        FORMAT
         AGE
       vmi-d2wnmq.....   clitem-d2wnmq..... cl-b585915ddxxxxxxxx   item 1               ubuntu64guest  ovf
         26c
       vmi-55088d.....   clitem-55088d..... cl-b585915ddxxxxxxxx   item 2               otherguest     ovf
         26c
      ```


2. Publish an image to the content library.
   a) Create a yaml file to deploy a source VM.

      Make sure that the imageName in the VirtualMachine spec references one of the VM images from the content library.

      For example, `source-vm.yaml` .

      ```
      apiVersion: vmoperator.vmware.com/v1alpha2
      kind: VirtualMachine
      metadata:
        name: source-vm
        namespace: test-publish-ns
      spec:
        className: best-effort-small
        storageClass: wcpglobal-storage-profile
        imageName: vmi-d2wnmq.....
        powerState: poweredOn
        vmMetadata:
          transport: CloudInit
      ```

   b) Obtain information about the deployed VM and connect to the VM to make sure it's running.

      You see the output similar to the following.

      ```
      kubectl get vm -n <namespace-name>
      NAME         POWER-STATE   CLASS             IMAGE           PRIMARY-IP        AGE
      source-vm    poweredOn     best-effort-small vmi-d2wnmq..... 192.168.000.00   9m32s
      ```

   c) Create a publish request for a new target image.

      For example, `vmpub.yaml` . In the request, indicate the name of the source VM and target content library where you want to publish your image. Make sure that the library is writable.

      ```
      apiVersion: vmoperator.vmware.com/v1alpha2
      kind: VirtualMachinePublishRequest
      metadata:
        name: vmpub-1
        namespace: test-publish-ns
      spec:
        source:
          apiVersion: vmoperator.vmware.com/v1alpha2
          kind: VirtualMachine
          name: source-vm # If empty, the name of this VirtualMachinePublishRequest will be used as the
       source VM name ("vmpub-1" in this example).
        target:
          item:
            name: publish-image-1 # If empty, the target item name is <source-vm-name>-image by default
          location:
            apiVersion: imageregistry.vmware.com/v1alpha2
            kind: ContentLibrary
            name: cl-b585915ddxxxxxxxx
      ```

   d) Describe the publish request.

      Make sure that the publish request in a `Ready` status with the `ImageName` set.

      ```
      kubectl describe vmpub vmpub-1 -n <namespace-name>
      ==================
      Status:
        imageName: vmi-12980cddd...
        ready: true
      ```

```
==================
```

e) Verify that the new image is added to the content library after the publish request is complete.

```
kubectl get vmi
 NAME              PROVIDER-NAME      CONTENT-LIBRAY-NAME    IMAGE-NAME      VERSION   OS-TYPE
 FORMAT   AGE
 vmi-12980cddd..   clitem-12980cddd.. cl-b585915ddxxxxxxxx   publish-image-1           ubuntu64guest
 ovf      7m12s
 vmi-d2wnmq.....   clitem-d2wnmq..... cl-b585915ddxxxxxxxx   item 1                    ubuntu64guest
 ovf      26m
 vmi-55088d.....   clitem-55088d..... cl-b585915ddxxxxxxxx   item 2                    otherguest
 ovf      26m
```

You can use this new image to deploy a new VM.

Working with VM Classes in vSphere Supervisor

To be able to self-service VMs in a Supervisor, DevOps users must have access to VM classes. A VM class is a template that defines CPU, memory, and reservations for VMs. The VM class helps to set guardrails for the policy and governance of VMs by anticipating development needs and accounting for resource availability and constraints.

The platform offers several default VM classes. A vSphere administrator can use them as is or create custom VM classes. To make the classes available to the DevOps users, the vSphere administrator adds them to a namespace. The VM classes assigned to the namespace can be used by stand-alone VMs and by the VMs that make up VKS clusters.

Create a Custom VM Class Using the vSphere Client

As a vSphere administrator, you can use available default classes. You can also create custom VM classes instead of the default and use them for VM deployment in a namespace.

Required privileges:

- **Namespaces** > **Modify cluster-wide configuration**
- **Namespaces** > **Modify namespace configuration**
- **Virtual Machine Classes** > **Manage Virtual Machine Classes**

When you create new classes, keep in mind the following considerations.

- VM classes that you create in a vCenter instance are available to all vCenter clusters and all namespaces in these clusters.
- VM classes are available to all namespaces in vCenter. However, DevOps engineers can use only those VM classes that you associate with a particular namespace.

Note: You can also create VM classes using the DCLI command. See Create and Manage VM Classes Using the Data Center CLI.

1. Go to the **VM Service** page.
   a) From the vSphere Client home menu, select **Supervisor Management**.
   b) Click the **Services** tab and click **Manage** on the **VM Service** pane.
2. On the **VM Service** page, click **VM Classes** and click **Create VM Class**.
3. On the **Name** page, specify the VM class name and click **Next**.
   The VM class name Identifies the VM class. Enter a unique DNS compliant name that follows these requirements:
   - Use a unique name that does not duplicate the names of default or custom VM classes in your environment.
   - Use alphanumeric string with maximum length of 63 characters.
   - Do not use uppercase letters or spaces.
   - Use a dash anywhere except as a first or last character. For example, `vm-class1`.

   After you create the VM class, you cannot change its name.
4. On the **Compatibility** page, select the VM class hardware compatibility and click **Next**.
   For more information, see *Virtual Machine Compatibility* in the vSphere documentation.
   Note: You can only set the hardware compability of a VM class during its creaiton and you cannot change it later.
5. On the **Configuration** page, leave the defaults.
6. On the **Review and Confirm** page, review the details and click **Finish**.

Edit the VM class configuration such as VM hardware and VM options.

Edit a VM Class Using the vSphere Client

Checkout how to edit a VM class after its creation. You can configure hardware resources such as CPU, memory, and devices as well as edit VM options and advanced parameters. You can also edit default VM classes that Supervisors offer.

Required privileges:

- **Namespaces** > **Modify cluster-wide configuration**
- **Namespaces** > **Modify namespace configuration**
- **Virtual Machine Classes** > **Manage Virtual Machine Classes**

Editing a VM class does not result in automatic reconfiguring of the VMs that were previously deployed from this class. For example, if a DevOps user created a VKS cluster with the VM class and you later change the VM class definition, existing VKS VMs remain unaffected. New VKS VMs will use the modified class definition.

CAUTION:

If you scale out a VKS cluster after editing a VM class used by that cluster, new cluster nodes use the updated class definition, but existing cluster nodes continue to use the initial class definition, resulting in a mismatch. Both control plane and worker nodes can be scaled. For information about scaling, see *Scale a Workload Cluster* in the vSphere Supervisor Services and Standalone Components documentation.

When you delete a VM class, it is removed from all associated namespaces. DevOps users can no longer self-service VMs using this VM class. VMs that have already been created with this VM class are not affected.

1. In the vSphere Client, display available VM classes.
   a) From the vSphere Client home menu, select **Supervisor Management**.
   b) Click the **Services** tab and click the **VM Service** pane.
   c) On the **VM Service** page, click **VM Classes**.
      All default or user-created VM classes appear under **Available VM Classes**.

2. In the selected VM class pane, click **Manage** and click **Edit**.
3. In the **Virtual Hardware** page, configure the hardware resources of the VM class, such as memory, CPU, and different devices.

   All the VM hardware settings are applied when a DevOps user assigns the VM class to a VM. For example, the CPU configuration values become the CPU resources dedicated to all the VMs that DevOps user creates by using the VM class.

   Note: Starting from vSphere 8.0 Update 2b, the wizard for creating and editing VM classes changes from setting CPU and memory resources in percentages to numeric values in MB, GB, TB, and HMz. For all previously created VM classes you will see the CPU and memory in percentages, but now you can edit these values in the new numeric formats.

VM Configuration Option	Description
CPU	Define the CPU resources dedicated to the VM. For more information on configuring CPU resources, see *Virtual CPU Configuration and Limitations* and *Configure CPU Resources of a Virtual Machine* in *vSphere Virtual Machine Administration*.
Memory	Define the memory configured for a VM in MB, GB, or TB. For more information on VM memory resources, see *Virtual Memory Configuration* in *vSphere Virtual Machine Administration*.
Video Card	Configure 3D graphics to take advantage of Windows AERO, CAD, Google Earth, and other 3D design, modeling, and multimedia applications. For more information on the video card setting, see *How do I Configure 3D Graphics* in *vSphere Virtual Machine Administration*.
Security Devices	Provide additional security to the VM class by configuring ® Software Guard Extensions (vSGX). See *Securing Virtual Machines with Intel Software Guard Extensions* in *vSphere Virtual Machine Administration*.

4. In the **Virtual Hardware**option, click **Add New Device** to add and configure devices to the VM class.

   You configure different devices to the VM class, such as storage controllers, network adapters, USB, and PCI devices.

   For more information about the VM configuration options, see the *vSphere Virtual Machine Administration* documentation.

VM Configuration Option	Description
RDM Disk	Add a raw device mapping (RDM) to store virtual machine data directly on a SAN LUN, instead of storing it in a virtual disk file. See *Add an RDM Disk to a Virtual Machine* in *vSphere Virtual Machine Administration*.
Host USB Device	Add one or more USB passthrough devices from an ESXi host to a virtual machine if the physical devices are connected to the host on which the virtual machine runs. See *Add USB Devices from an ESXi Host to a Virtual Machine* in *vSphere Virtual Machine Administration*.
NVDIMM	Configure a virtual NVDIMM device to the VM class to enable it to use non-volatile, or persistent, computer memory. See *Add an NVDIMM Device to a Virtual Machine* in *vSphere Virtual Machine Administration*.
CD/DVD Drive	Configure a CD/DVD device to the VM class. See *How do I Add or Modify a Virtual Machine CD or DVD Drive* in *vSphere Virtual Machine Administration*.
NVMe Controller, **SATA Controller**, **SCSI Controller**	Configure storage controllers to the VM class. See *SCSI, SATA, and NVMe Storage Controller Conditions, Limitations, and Compatibility* in *vSphere Virtual Machine Administration* .

VM Configuration Option	Description
USB Controller	Add a USB controller to the VM class to support USB passthrough from an ESXi host or from a client compute. See *Add a USB Controller to a Virtual Machine* in *vSphere Virtual Machine Administration*.
PCI Device	Configure VMs to use the NVIDIA GRID virtual GPU (vGPU) technology, if ESXi hosts in your Supervisor environment have one or more NVIDIA GRID GPU graphics devices. You can also configure other PCI devices on an ESXi host to make them available to a VM in a passthrough mode. If you select this option, the memory resource reservation value automatically changes to 100%. For more information and additional requirements, see Deploying a VM with PCI Devices in Supervisor Environment.
Watchdog Timer	Add a virtual Watchdog Timer (VWDT) device to ensure self-reliance related to the system performance within a virtual machine. See *How do I Add a Virtual Watchdog Timer Device to a Virtual Machine* in *vSphere Virtual Machine Administration*.
Precision Clock	Add a precision clock device to the VM. A precision clock is a virtual clock device that provides a virtual machine with access to the system time of the primary ESXi host. See *How do I Add a Precision Clock Device to a Virtual Machine* in *vSphere Virtual Machine Administration*.
Serial Port	Configure a connection of the virtual serial port to a physical serial port or to a file on the host computer. See *Change the Serial Port Configuration* in *vSphere Virtual Machine Administration*.
Instance Storage	Configure instance storage to the VM. Along with persistent storage volumes, a VM can use instance storage. Unlike persistent volumes that exist separately from the VM, instance storage volumes depend on the lifecycle of a VM instance. Using the **Instance Storage** option, you can add appropriate storage policies and configure volumes to be used with the VM. For additional requirements, see Deploying a VM with Instance Storage in vSphere Supervisor.
Network Adapter	Configure a network adapter to the VM class. When the DevOps user deploys a VM by using the VM class, she can specify a workload network to the adapter. The workload network must be configured to the vSphere Namespace where the VM runs. For more information on the supported adapter types, see *Network Adapter Basics* in *vSphere Virtual Machine Administration*.

5. On the **VM Options** page, set or change VM options to run VMware Tools scripts, control user access to the remote console, configure startup behavior, and more.

   For more information the VM options that you can configure to the VM class, see *Configuring Virtual Machine Options* in *vSphere Virtual Machine Administration*.
6. On the **Advanced Parameters** page, change or add VM configuration parameters when instructed by a VMware technical support representative, or if you see VMware documentation that instructs you to add or change a parameter to fix a problem with your system.

   For more information the VM advanced parameters, see *Configure Virtual Machine Advanced File Parameters* in *vSphere Virtual Machine Administration*.
7. Once you are ready with editing the VM class, review and confirm your changes and click **Finish**.

Associate a VM Class with a Namespace Using the vSphere Client

As a vSphere administrator, add a default or custom VM class to one or more namespaces on a Supervisor. When you add a VM class to a namespace, you make the class available to DevOps users, so that they can start self-servicing VMs

in the Kubernetes namespace environment. The VM classes you assign to the namespace are also used by the VMs that make up VKS clusters.

Required privileges:

- **Namespaces** > **Modify cluster-wide configuration**
- **Namespaces** > **Modify namespace configuration**
- **Virtual Machine Classes** > **Manage Virtual Machine Classes**

You can add multiple VM classes to a single namespace. Different VM classes serve as indicators of different levels of service. If you publish multiple VM classes, DevOps users can select between all custom and default classes when creating and managing virtual machines in the namespace.

Note:

To be able to deploy a VKS cluster in a newly created namespace, DevOps engineers need to have access to VM classes. As a vSphere administrator, you must explicitly associate default or custom VM classes to any new namespace where the VKS cluster is deployed.

1. In the vSphere Client, go to the namespace.
   a) From the home menu, select **Supervisor Management**.
   b) Click the **Namespaces** tab and click the namespace.
2. Add a VM class.
   a) On the **VM Service** pane, click **Add VM Class**.
   b) Select one or several VM classes and click **OK**.

The VM classes you added become available in the namespace for the DevOps to self-service VMs. These classes can also be used by the VMs that make up VKS clusters.

Manage VM Classes on a Namespace Using the vSphere Client

After you associate a VM class with a namespace, you can add more VM classes or remove the class to unpublish it from the Kubernetes namespace.

- If you want to remove a VM class from a namespace, verify that it is not used by VKS. Removing it can affect operations VKS.
- Required privileges:
  – **Namespaces** > **Modify cluster-wide configuration**
  – **Namespaces** > **Modify namespace configuration**
  – **Virtual Machine Classes** > **Manage Virtual Machine Classes**

1. In the vSphere Client, go to the namespace.
   a) From the home menu, select **Supervisor Management**.
   b) Click the **Namespaces** tab and click the namespace.
2. Add or remove a VM class.
   a) On the **VM Service** pane, click **Manage VM Class**.
   b) Perform one of the following operations.

Remove a VM class	Deselect the VM class and click **OK**.
Add a VM class	Select one or several VM classes and click **OK**.

Create and Manage VM Classes Using the Data Center CLI

In addition thevSphere Client, you can use the Data Center CLI (DCLI) commands to create and manage the VM classes. The DCLI commands give you more flexibility and access to VM configuration options not available in thevSphere Client.

Prerequisites

Log in tovCenter using the root user account and type `dcli +i` to use the DCLI in interactive mode.

For information about DCLI commands, see Overview of Running DCLI Commands.

Available DCLI Commands

Command	Description
namespacemanagement virtualmachineclasses create	Create a VM class object.
namespacemanagement virtualmachineclasses delete	Delete the VM class object.
namespacemanagement virtualmachineclasses get	Return information about a VM class.
namespacemanagement virtualmachineclasses list	Return information about all VM classes.
namespacemanagement virtualmachineclasses update	Update configuration of the VM class object.

Create a VM Class Using Data Center CLI

As a vSphere administrator, use the DCLI `com vmware vcenter namespacemanagement virtualmachineclasses create` command to create a VM class. You can configure such VM properties as CPU, memory, memory reservations, network adapters, and so on.

The command takes the following arguments.

Argument	Description
-h, --help	Show this help message and exit.
--config-spec CONFIG_SPEC	A VirtualMachineConfigSpec associated with the VM class (json input).
--cpu-count CPU_COUNT	Required. The number of CPUs configured for virtual machine of this class (integer).
--cpu-reservation CPU_RESERVATION	The percentage of total available CPUs reserved for a virtual machine (integer).
--description DESCRIPTION	Description for the VM class (string).
--devices-dynamic-direct-path-io-devices DEVICES_DYNAMIC_DIRECT_PATH_IO_DEVICES	List of Dynamic DirectPath I/O devices (json input).
--devices-vgpu-devices DEVICES_VGPU_DEVICES	List of vGPU devices (json input).
--id ID	Required. Identifier of the virtual machine class (string).
--instance-storage-policy INSTANCE_STORAGE_POLICY	Storage policy corresponding to the instance storage (string).
--instance-storage-volumes INSTANCE_STORAGE_VOLUMES	List of instance storage volumes (json input).
--memory-mb MEMORY_MB	Required. The amount of memory in MB configured for virtual machine of this class (integer).

Argument	Description
--memory-reservation MEMORY_RESERVATION	The percentage of available memory reserved for a virtual machine of this class.

Use the following examples to create VM classes with different properties.

CPU and Memory

```
com vmware vcenter namespacemanagement virtualmachineclasses create --id cpu-mem-class --cpu-count 2 --memo-
ry-mb 2048 --config-spec '{"_typeName":"VirtualMachineConfigSpec","numCPUs":2,"memoryMB":2048}'
```

Setting `numCPUs` and `memoryMB` in the Config Spec is optional. If you chose to set them, they must have the same values as the mandatory `--cpu-count` and `--memory-mb` vAPI fields.

CPU and Memory Reservations

When you create a VM Class by using a Config Spec having CPU and memory reservation, the memory reservation or limit is in MB for `memoryAllocation`, MHz for `cpuAllocation`.

```
com vmware vcenter namespacemanagement virtualmachineclasses create --id cpu-res-class-1 --config-spec
 '{"_typeName":"VirtualMachineConfigSpec","numCPUs":2,"memoryMB":2048,"cpuAllocation":{"_typeName":"Re-
sourceAllocationInfo","reservation":200,"limit":200},"memoryAllocation":{"_typeName":"ResourceAllocationIn-
fo","reservation":200,"limit":200}}' --cpu-count 2 --memory-mb 2048
```

Network Adapter

The following command creates a network adapter of type E1000.

```
com vmware vcenter namespacemanagement virtualmachineclasses create --id class-w-e1000 --cpu-count 2 --memo-
ry-mb 2048 --config-spec '{"_typeName":"VirtualMachineConfigSpec","deviceChange":[{"_typeName":"VirtualDevice-
ConfigSpec","operation":"add","device":{"_typeName":"VirtualE1000","key":-100}}]}'
```

vGPUs

In these examples, the first command creates a VM class with a vGPU by using the field `--devices-vgpu-devices`. The second command creates a VM class with a vGPU by using a configSpec.

```
com vmware vcenter namespacemanagement virtualmachineclasses create --id vmclass-1 --devices-vgpu-devices
 '[{"profile_name": "mockup-vmiop-8c"}]' --memory-reservation 100 --cpu-count 2 --memory-mb 4096
com vmware vcenter namespacemanagement virtualmachineclasses create --id vmclass-2 --cpu-count 2 --memory-mb
 4096 --config-spec '{"_typeName":"VirtualMachineConfigSpec","deviceChange":[{"_typeName":"VirtualDeviceCon-
figSpec","operation":"add","device":{"_typeName":"VirtualPCIPassthrough","key":20,"backing":{"_typeName":"Vir-
tualPCIPassthroughVmiopBackingInfo","vgpu":"mockup-vmiop-8c"}}}]}' --memory-reservation 100
```

Instance Storage

The following examples create VM classes that use instance storage by using the `--instance-storage-volumes` and `--instance-storage-policy` fields.

```
com vmware vcenter namespacemanagement virtualmachineclasses create --id vmclass-ist-1 --instance-storage-vol-
umes '[{"size":47}]' --instance-storage-policy "e28d4352-1d1e-431b-b3f7-528bef5838a0" --cpu-count 2 --memo-
ry-mb 4096
```

The ID field in this example is a well-known virtualDisk ID that represents an instance storage device in the VM Service VMs.


```
com vmware vcenter namespacemanagement virtualmachineclasses create --id vmclass-ist-2 --cpu-count
 2 --memory-mb 2048 --config-spec '{"_typeName":"VirtualMachineConfigSpec","deviceChange":[{"_type-
Name":"VirtualDeviceConfigSpec","operation":"add","fileOperation":"create","device":{"_typeName":"Vir-
tualDisk","key":0,"backing":{"_typeName":"VirtualDiskFlatVer2BackingInfo","fileName":"","diskMod-
e":"","thinProvisioned":false},"capacityInKB":0,"capacityInBytes":49283072,"vDiskId":{"_type-
Name":"ID","id":"e28d4352-1d1e-431b-b3f7-528bef5838a0"}},"profile":[{"_typeName":"VirtualMachineDefinedPro-
fileSpec","profileId":"e28d4352-1d1e-431b-b3f7-528bef5838a0","profileData":{"_typeName":"VirtualMachineProfil-
eRawData","extensionKey":"com.vmware.vim.sps"}}]}]}'
```

Update a VM Class Using Data Center CLI

As a vSphere administrator, use the DCLI `com vmware vcenter namespacemanagement virtualmachineclasses update` command to modify a VM class.

Use the following as examples.

Modify CPU and Memory

```
com vmware vcenter namespacemanagement virtualmachineclasses get --vm-class cpu-mem-class
com vmware vcenter namespacemanagement virtualmachineclasses update --vm-class cpu-mem-class --cpu-count 4 --
memory-mb 4096
```

Modify CPU and Memory Reservations

When you create a VM Class by using a Config Spec having CPU and memory reservation, the memory reservation or limit is in MB for `memoryAllocation`, MHz for `cpuAllocation`.

```
com vmware vcenter namespacemanagement virtualmachineclasses get --vm-class cpu-res-class-1
com vmware vcenter namespacemanagement virtualmachineclasses update --vm-class cpu-res-class-1 --cpu-reserva-
tion 100 --memory-reservation 100
com vmware vcenter namespacemanagement virtualmachineclasses get --vm-class cpu-res-class-1
```

You can also use Config Spec to update the CPU and memory reservations. Any existing CPU or memory reservations will be overwritten. Use the following as an example:

```
com vmware vcenter namespacemanagement virtualmachineclasses update --vm-class  cpu-res-class-1 --config-spec
 '{"_typeName":"VirtualMachineConfigSpec","cpuAllocation":{"_typeName":"ResourceAllocationInfo","reser-
vation":200,"limit":200},"memoryAllocation":{"_typeName":"ResourceAllocationInfo","reservation":200,"lim-
it":200}}'
```

Add vGPUs

```
com vmware vcenter namespacemanagement virtualmachineclasses get --vm-class class-w-e1000
com vmware vcenter namespacemanagement virtualmachineclasses update --vm-class class-w-e1000 --devices-vg-
pu-devices '[{"profile_name": "mockup-vmiop-8c"}]'
com vmware vcenter namespacemanagement virtualmachineclasses get --vm-class class-w-e1000


com vmware vcenter namespacemanagement virtualmachineclasses get --vm-class vmclass-1
com vmware vcenter namespacemanagement virtualmachineclasses update --vm-class vmclass-1 --config-spec
 '{"_typeName":"VirtualMachineConfigSpec","deviceChange":[{"_typeName":"VirtualDeviceConfigSpec","oper-
ation":"add","device":{"_typeName":"VirtualPCIPassthrough","key":20,"backing":{"_typeName":"VirtualPCI-
PassthroughVmiopBackingInfo","vgpu":"mockup-vmiop-8c"}}},{"_typeName":"VirtualDeviceConfigSpec","oper-
ation":"add","device":{"_typeName":"VirtualPCIPassthrough","key":20,"backing":{"_typeName":"VirtualPCI-
PassthroughVmiopBackingInfo","vgpu":"mockup-vmiop"}}}]}'
```

```
com vmware vcenter namespacemanagement virtualmachineclasses get --vm-class vmclass-1
```

Remove vGPU

```
com vmware vcenter namespacemanagement virtualmachineclasses update --vm-class vmclass-1 --devices-vgpu-de-
vices '[]'
 com vmware vcenter namespacemanagement virtualmachineclasses get --vm-class vmclass-1
```

Add Instance Storage

```
com vmware vcenter namespacemanagement virtualmachineclasses update --vm-class vmclass-1 --instance-stor-
age-volumes '[{"size":47}]' --instance-storage-policy "e28d4352-1d1e-431b-b3f7-528bef5838a0"
 com vmware vcenter namespacemanagement virtualmachineclasses get --vm-class vmclass-1

com vmware vcenter namespacemanagement virtualmachineclasses get --vm-class vmclass-ist-2
com vmware vcenter namespacemanagement virtualmachineclasses update --vm-class vmclass-ist-2 --instance-stor-
age-volumes '[{"size":51},{"size":50}]' --instance-storage-policy "e28d4352-1d1e-431b-b3f7-528bef5838a0"
com vmware vcenter namespacemanagement virtualmachineclasses get --vm-class vmclass-ist-2
com vmware vcenter namespacemanagement virtualmachineclasses update --vm-class vmclass-ist-2 --config-spec
 '{"_typeName":"VirtualMachineConfigSpec","deviceChange":[{"_typeName":"VirtualDeviceConfigSpec","opera-
tion":"add","fileOperation":"create","device":{"_typeName":"VirtualDisk","key":0,"backing":{"_typeName":"Vir-
tualDiskFlatVer2BackingInfo","fileName":"","diskMode":"","thinProvisioned":false},"capacityInKB":0,"ca-
pacityInBytes":52428800,"vDiskId":{"_typeName":"ID","id":"cc737f33-2aa3-4594-aa60-df7d6d4cb984"}},"pro-
file":[{"_typeName":"VirtualMachineDefinedProfileSpec","profileId":"e28d4352-1d1e-431b-b3f7-528be-
f5838a0","profileData":{"_typeName":"VirtualMachineProfileRawData","extensionKey":"com.vmware.vim.sps"}}]}]}'
com vmware vcenter namespacemanagement virtualmachineclasses get --vm-class vmclass-ist-2
```

Remove Instance Storage

```
com vmware vcenter namespacemanagement virtualmachineclasses update --vm-class vmclass-ist-1 --instance-stor-
age-volumes '[]' --instance-storage-policy ""
com vmware vcenter namespacemanagement virtualmachineclasses get --vm-class vmclass-ist-1
```

Add Network Adapters

This command adds both an instance storage and an e1000 NIC to the VM class.

```
com vmware vcenter namespacemanagement virtualmachineclasses update --vm-class vmclass-ist-2 --config-spec
 '{"_typeName":"VirtualMachineConfigSpec","deviceChange":[{"_typeName":"VirtualDeviceConfigSpec","opera-
tion":"add","fileOperation":"create","device":{"_typeName":"VirtualDisk","key":0,"backing":{"_typeName":"Vir-
tualDiskFlatVer2BackingInfo","fileName":"","diskMode":"","thinProvisioned":false},"capacityInKB":0,"ca-
pacityInBytes":52428800,"vDiskId":{"_typeName":"ID","id":"cc737f33-2aa3-4594-aa60-df7d6d4cb984"}},"pro-
file":[{"_typeName":"VirtualMachineDefinedProfileSpec","profileId":"e28d4352-1d1e-431b-b3f7-528be-
f5838a0","profileData":{"_typeName":"VirtualMachineProfileRawData","extensionKey":"com.vmware.vim.sp-
s"}}]},{"_typeName":"VirtualDeviceConfigSpec","operation":"add","device":{"_typeName":"Vir-
tualE1000","key":-100}}]}'
com vmware vcenter namespacemanagement virtualmachineclasses get --vm-class vmclass-ist-2
```

Empty Config Spec

```
com vmware vcenter namespacemanagement virtualmachineclasses update --vm-class class-w-e1000 --config-spec ''
```

What to do next

The VM classes that you create with the DCLI, become available in vCenter. You can use the vSphere Client to assign these VM classes to a namespace. See Associate a VM Class with a Namespace Using the vSphere Client.

Deploying a Stand-Alone VM in vSphere Supervisor

As a DevOps engineer, use the `kubectl` command to review available VM resources and provision a stand-alone Linux or Windows VM in a namespace on a Supervisor. If the VM includes a PCI device configured for vGPU, after you create and boot the VM in your a Supervisor environment, you can install the NVIDIA vGPU graphics driver to enable GPU operations.

Prerequisites

To be able to deploy a stand-alone VM, a DevOps engineer must have access to specific VM resources. Make sure that a vSphere administrator has performed these steps to make VM resources available:

- Create a namespace and assign storage policies to it. See the Supervisor documentation.
- Create a content library and associate it with the namespace. See Creating and Managing Content Libraries for Stand-Alone VMs in vSphere Supervisor.
  - If a content library is protected by a security policy, all library items must be complaint. If the protected library includes a mix of compliant and non-compliant items, the `kubectl get virtualmachineimages` command fails to present VM images to the DevOps engineers.
  - If you plan to deploy VMs with vGPU devices, you must have access to images with the boot mode set to EFI, such as CentOS.
- Associate default or custom VM classes with a namespace. See Working with VM Classes in vSphere Supervisor.

If you plan to use NVIDIA vGPU or other PCI devices for your VMs, you must follow additional requirements. For information, see Deploying a VM with PCI Devices in Supervisor Environment.

For information about VM operator and supported fields, see Concepts of VM Service and https://vm-operator.readthedocs.io/en/stable/ref/api/v1alpha2/.

View VM Resources Available on a Namespace in a Supervisor

As a DevOps engineer, verify that you can access VM resources on your namespace, and view VM classes and VM templates available in your environment. You can also list storage classes and other items you might need to self-service a VM.

This task covers commands you use to access resources available for a deployment of a stand-alone VM. For information about resources necessary to deploy VKS clusters and VMs that make up the clusters, see *Using VM Classes with VKS Clusters* in the vSphere Supervisor Services and Standalone Components documentation.

1. Access your namespace in the Kubernetes environment.

   See Connecting to Supervisor and VKS Clusters.

2. To view VM classes available in your namespace, run the following command.

   ```
   kubectl get virtualmachineclass
   ```

   You can see the following output.

   Note: Because the best effort VM class type allows resources to be overcommitted, you can run out of resources if you have set limits on the namespace where you are provisioning the VMs. For this reason, use the guaranteed VM class type in the production environment.

   ```
   NAME                    VIRTUALMACHINECLASS     AGE

   best-effort-large       best-effort-large       44m
   best-effort-medium      best-effort-medium      44m
   best-effort-small       best-effort-small       44m
   best-effort-xsmall      best-effort-xsmall      44m
   ```

```
custom                  custom                  44m
```

3. To view details of a specific VM class, run the following commands.
   - `kubectl describe virtualmachineclasses` *`name_vm_class`*

     If a VM class includes a vGPU device, you can see its profile under `spec: hardware: devices: vgpuDevices`.

     ```
     ......
     spec:
       hardware:
         cpus: 4
         devices:
           vgpuDevices:
           - profileName: grid_v100-q4
     ......
     ```

   - `kubectl get virtualmachineclasses -o wide`

     If the VM class includes a vGPU or a passthrough device, the output shows it in the `VGPUDevicesProfileNames` or `PassthroughDeviceIDs` column.
4. View the VM images.

   `kubectl get virtualmachineimages`

   The output you see is similar to the following. The image name, such as `vmi-xxxxxxxxxxxx` is auto-generated by the system.

   ```
   NAME                                          VERSION   OSTYPE               FORMAT   IMAGESUPPORTED
    AGE
   vmi-xxxxxxxxxxxx                                        centos8_64Guest      ovf      true
    4d3h
   ```

5. To describe a specific image, use the following command.

   `kubectl describe virtualmachineimage vmi-xxxxxxxxxxxx`

   VMs with vGPU devices require images that have boot mode set to EFI, such as CentOS. Make sure to have access to these images.
6. Verify that you can access storage classes.

   `kubectl get resourcequotas`

   For more information, see Display Storage Classes in a vSphere Namespace.

   ```
   NAME                       AGE   REQUEST
                             LIMIT
   my-ns-ubuntu-storagequota   24h   wcpglobal-storage-profile.storageclass.storage.k8s.io/requests.storage:
    0/9223372036854775807
   ```

Deploy a Virtual Machine on a Namespace in a Supervisor

As a DevOps engineer, provision a VM and its guest OS in a declarative manner by writing VM deployment specifications in a Kubernetes YAML file.

If you use NVIDIA vGPU or other PCI devices for your VMs, see Deploying a VM with PCI Devices in Supervisor Environment.

Note:

Starting with VCF 9.0, the v1alpha1 version of the VM Operator API is deprecated. Use v1alpha2 or v1alpha3 instead, as both are supported on Supervisor and are fully backward compatible. Support for v1alpha1 will be phased out. For information, see https://vm-operator.readthedocs.io/en/latest/ref/api/.

1. Prepare a VM YAML file.

   In the file, specify the following parameters:

Option	Description
apiVersion	Specifies the version of the VM Service API. Such as `vmoperator.vmware.com/v1alpha2`.
kind	Specifies the type of Kubernetes resource to create. The only available value is `VirtualMachine`.
spec.imageName	Specifies the virtual machine image resource name in Kubernetes cluster.
spec.storageClass?	Specifies the storage class to be used for storage of the persistent volumes.
spec.className	Specifies the name of the VM class that describes the virtual hardware settings to be used.
spec.networkInterfaces	Specifies network-related settings for the VM. • `networkType`. Values for this key can be `nsx-t` or `vsphere-distributed`. • `networkName`. If needed, specify the name or leave the default name.
spec.vmMetadata	Includes additional metadata to pass to the VM. You can use this key to customize the guest OS image and set such items as the `hostname` of the VM and `user-data`, including passwords, ssh keys, and so on. For additional information, including details on how to bootstrap and customize Windows VMs using the Microsoft System Preparation tool (Sysprep), see Customizing a Guest.
topology.kubernetes.io/zone	Controls the VM placement on a three-zone Supervisor. For example, `topology.kubernetes.io/zone: zone-a02`.

   The following example VM YAML file `my-vm` uses CloudInit as a bootstrapping method. The example shows a VirtualMachine resource that specifies user data in a Secret resource `my-vm-bootstrap-data` . The Secret will be used to bootstrap and customize the guest OS.

   The data in the Secret includes the CloudInit cloud-config. For more information on the cloud-config format, see the Cloud config examples official documentation.

   For examples with Sysprep as a bootstrapping method, see Sysprep.

   ```
   apiVersion: vmoperator.vmware.com/v1alpha2
   kind: VirtualMachine
   metadata:
     name:      my-vm
     namespace: my-namespace
   spec:
     className:    small
     imageName:    vmi-xxxxxxxxxxxx
     storageClass: iscsi
     vmMetadata:
       transport: CloudInit
   ```

```
    secretName: my-vm-bootstrap-data
```

```
apiVersion: v1
kind: Secret
metadata:
  name:      my-vm-bootstrap-data
  namespace: my-namespace
stringData:
  user-data: |
    #cloud-config
    users:
    - default
    - name: xyz..
      primary_group: xyz..
      groups: users
      ssh_authorized_keys:
      - ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAABAQDSL7uWGj...
    runcmd:
    - "ls /"
    - [ "ls", "-a", "-l", "/" ]
    write_files:
    - path: /etc/my-plaintext
      permissions: '0644'
      owner: root:root
      content: |
        Hello, world.
```

Use the following example if you are deploying a VM in an environment with zones.

To get the values for *ZONE_NAME*, run the `kubectl get vspherezones` command.

```
apiVersion: vmoperator.vmware.com/v1alpha2
kind: VirtualMachine
metadata:
  name: <vm-name>
  namespace: <vm-ns>
  labels:
    topology.kubernetes.io/zone: ZONE_NAME
...
```

2. Deploy the VM.

```
kubectl apply -f my-vm.yaml
```

3. Verify that the VM has been created.

```
kubectl get vm
NAME              AGE
my-vm             28s
```

4. Check the VM details and its status.

```
kubectl describe virtualmachine my-vm
```

The output is similar to the following. From the output, you can also obtain the IP address of the VM, which appears in the `Vm Ip` field.


```
Name:         my-vm
Namespace:    my-namespace
API Version:  vmoperator.vmware.com/v1alpha2
Kind:         VirtualMachine
Metadata:
  Creation Timestamp:  2021-03-23T19:07:36Z
  Finalizers:
    virtualmachine.vmoperator.vmware.com
  Generation:  1
  Managed Fields:
 ...
 ...
Status:
  Bios UUID:              4218ec42-aeb3-9491-fe22-19b6f954ce38
  Change Block Tracking:  false
  Conditions:
    Last Transition Time:  2021-03-23T19:08:59Z
    Status:                True
    Type:                  VirtualMachinePrereqReady
  Host:                    10.185.240.10
  Instance UUID:           50180b3a-86ee-870a-c3da-90ddbaffc950
  Phase:                   Created
  Power State:             poweredOn
  Unique ID:               vm-73
  Vm Ip:                   10.161.75.162
Events:                    <none>
...
```

5. Verify that the VM IP is reachable.

```
ping 10.161.75.162
PING 10.161.75.162 (10.161.75.162): 56 data bytes
64 bytes from 10.161.75.162: icmp_seq=0 ttl=59 time=43.528 ms
64 bytes from 10.161.75.162: icmp_seq=1 ttl=59 time=53.885 ms
64 bytes from 10.161.75.162: icmp_seq=2 ttl=59 time=31.581 ms
```

A VM created through the VM Service can be managed only by DevOps from the Kubernetes namespace. Its life cycle cannot be managed from the vSphere Client, but vSphere administrators can monitor the VM and its resources. For more information, see Monitor Virtual Machines Available in vSphere Supervisor.
For additional details, see the Introducing Virtual Machine Provisioning blog.

Deploying a VM with vGPU and Other PCI Devices in a Supervisor

If ESX hosts in your environment have one or more NVIDIA GRID GPU graphics devices, you can configure VMs to use the NVIDIA GRID virtual GPU (vGPU) technology. You can also configure other PCI devices on an ESX host to make them available to a VM in a passthrough mode.

Deploying a VM with vGPU in vSphere Supervisor

NVIDIA GRID GPU graphics devices are designed to optimize complex graphics operations and enable them to run at high performance without overloading the CPU. NVIDIA GRID vGPU provides unparalleled graphics performance, cost-effectiveness, and scalability by sharing a single physical GPU among multiple VMs as separate vGPU-enabled passthrough devices.

Considerations

The following considerations apply when you use NVIDIA vGPU:

- Three-zone Supervisor supports VMs with vGPU, but you cannot leave the zone field empty. You must make sure that the VM and the PV use the same zone. If you deploy a VM without a zone, VM Service places it only by VM class constraints and might not place it in the same zone as the PV.
  - When you create the VM and PVC together in the same manifest, specify the zone in the PVC. For example, add this annotation: `csi.vsphere.volume-requested-topology: '[{"topology.kubernetes.io/zone":"zone-1"}]'`.
  - When the PV already exists, specify the zone in the VM. The VM needs this label: `topology.kubernetes.io/zone: zone-1`.
- VMs with vGPU devices that are managed by VM Service are automatically powered off when an ESX host enters maintenance mode. This might temporarily affect workloads running in the VMs. The VMs are automatically powered on after the host exists the maintenance mode.
- DRS distributes vGPU VMs in a breadth-first manner across cluster's hosts. For more information, see *DRS Placement of vGPU VMs* in the *vSphere Resource Management* guide.

Requirements

To configure NVIDIA vGPU, follow these requirements:

- Verify that the ESX is supported in the VMware Compatibility Guide, and check with the vendor to verify the host meets power and configuration requirements.
- Configure ESX host graphics settings with at least one device in **Shared Direct** mode. See *Configuring Host Graphics* in the *vSphere Resource Management* documentation.
- The content library you use for VMs with vGPU devices must include images with the boot mode set to EFI, such as CentOS.
- Install NVIDIA vGPU software. NVIDIA provides a vGPU software package that includes the following components. For more information, see appropriate NVIDIA Virtual GPU Software documentation.
  - vGPU Manager that a vSphere administrator installs on the ESX host. See VMware Knowledge Base article 2033434.
  - Guest VM driver that a DevOps engineer installs in the VM after deploying and booting the VM. See the *Install the NVIDIA Guest Driver in a VM* section.

Add a vGPU Device to a VM Class Using the vSphere Client

Create or edit an existing VM class to add an NVIDIA GRID virtual GPU (vGPU).

Required privileges:

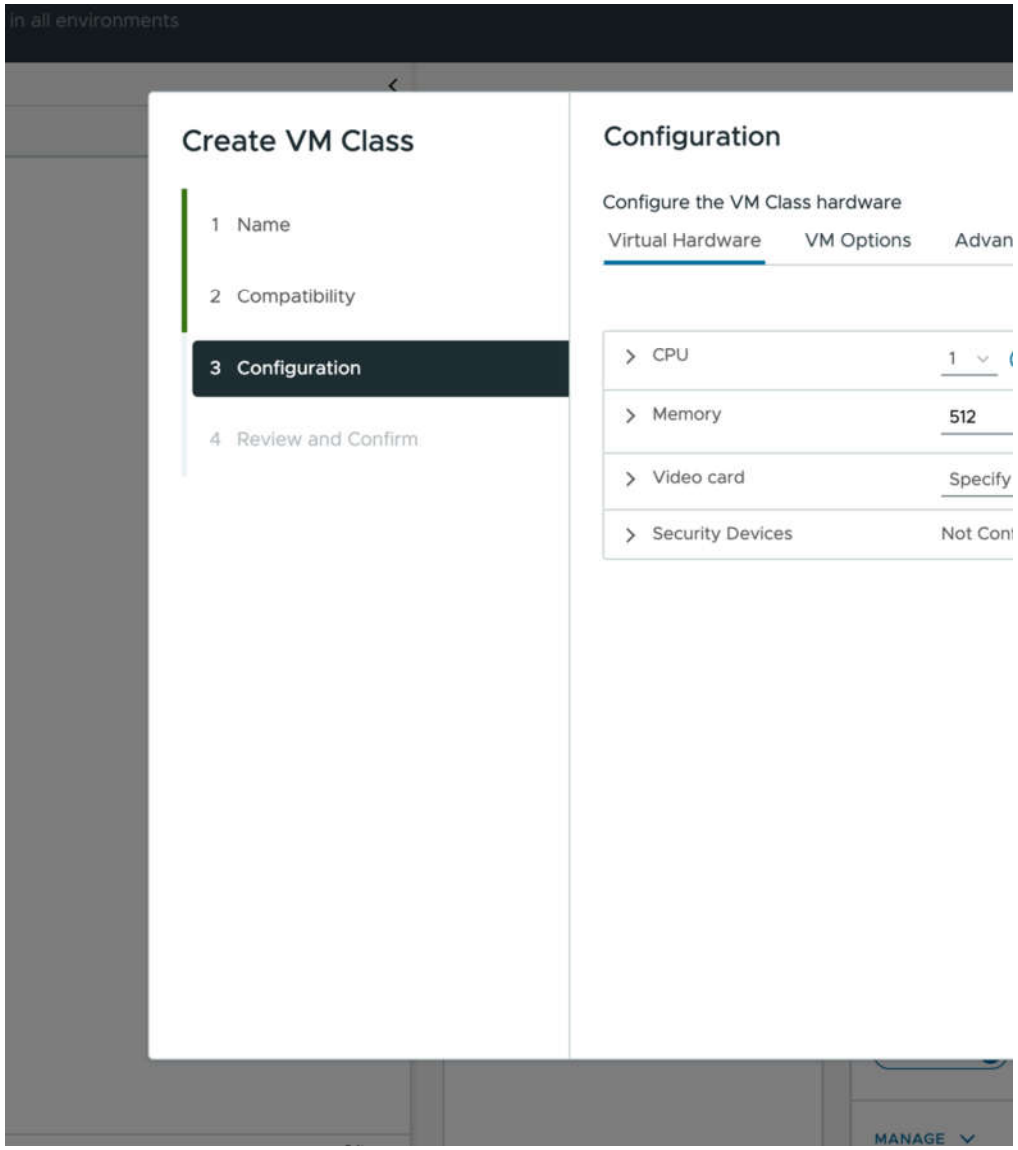
- **Namespaces** > **Modify cluster-wide configuration**
- **Namespaces** > **Modify namespace configuration**
- **Virtual Machine Classes** > **Manage Virtual Machine Classes**

1. Create or edit an existing VM class.

Option	Action
Create a new VM class	1. From the vSphere Client home menu, select **Supervisor Management**. 2. Click the **Services** tab and click **Manage** on the **VM Service** pane. 3. On the **VM Service** page, click **VM Classes** and click **Create VM Class**. 4. Follow the prompts.
Edit a VM class	1. From the vSphere Client home menu, select **Supervisor Management**. 2. Click the **Services** tab and click **Manage** on the **VM Service** pane. 3. On the **VM Service** page, click **VM Classes**. 4. In the existing VM class pane, click **Manage** and click **Edit**.

Option	Action
	5. Follow the prompts.

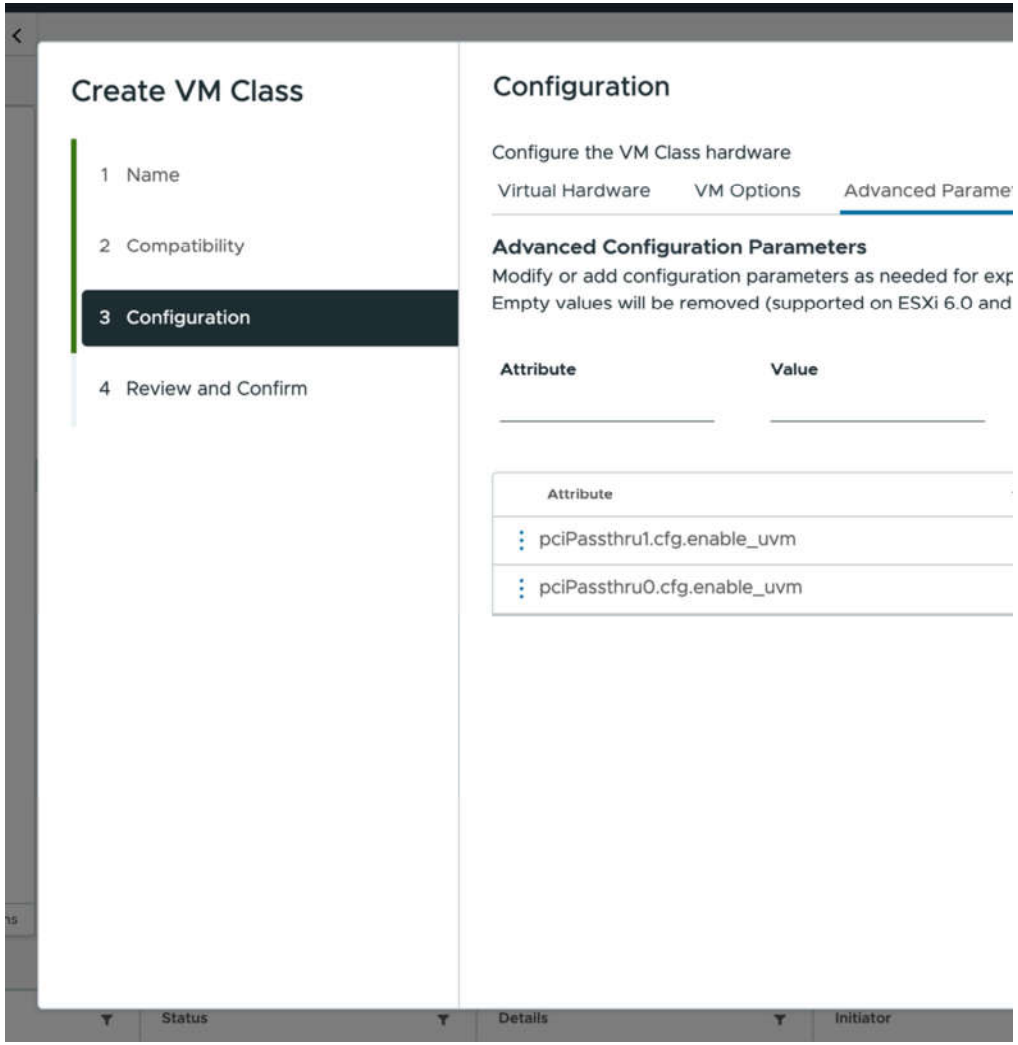
2. On the **Configuration** page, click the **Virtual Hardware**tab, click **Add New Device**, and select **PCI Device**.



3. From the list of available devices on the **Device Selection** page, select the NVIDIA GRID vGPU and click **Select**. The device appears on the Virtual Hardware page.
4. Click the **Advanced Parameters** tab and set the parameters with the following attributes and values.

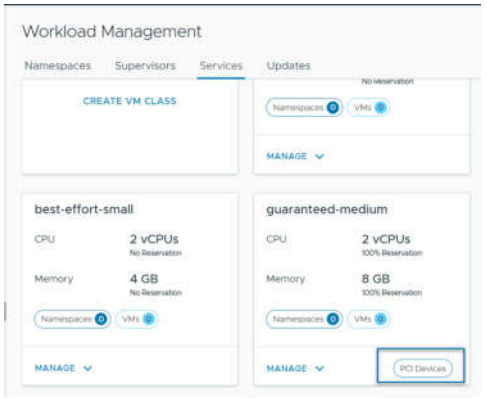
Option	Description
Parameter	Value
pciPassthru0.cfg.enable_uvm	1

Option	Description
pciPassthru1.cfg.enable_uvm	1



5. Review your configuration and click **Finish**.

A **PCI Devices** tag on the VM class pane indicates that the VM class is vGPU-enabled.



Add a vGPU Device to a VM Class Using Data Center CLI
In addition to the vSphere Client, you can use the Data Center CLI (DCLI) command to add vGPUs and advanced configurations.

For more information about DCLI commands, see Create and Manage VM Classes Using the Data Center CLI.

1. Log in to vCenter using the root user account and type `dcli +i` to use the DCLI in interactive mode.
2. Run the following command to create a VM class.
   In the following example, the *my-class* VM class includes two CPUs, 2048 of memory, and a VirtualMachineConfigSpec with two sample vGPUs profiles, `mockup-vmiop-8c` and `mockup-vmiop`. The extraConfig fields `pciPassthru0.cfg.enable_uvm` and `pciPassthru1.cfg.enable_uvm` are set to 1.

```
dcli +i +show-unreleased com vmware vcenter namespacemanagement virtualmachineclasses create --id
 my-class --cpu-count 2 --memory-mb 2048 --config-spec '{"_typeName":"VirtualMachineConfigSpec","de-
viceChange":[{"_typeName":"VirtualDeviceConfigSpec","operation":"add","device":{"_typeName":"Virtu-
alPCIPassthrough","key":20,"backing":{"_typeName":"VirtualPCIPassthroughVmiopBackingInfo","vgpu":"mock-
up-vmiop-8c"}}},{"_typeName":"VirtualDeviceConfigSpec","operation":"add","device":{"_typeName":"Virtu-
alPCIPassthrough","key":20,"backing":{"_typeName":"VirtualPCIPassthroughVmiopBackingInfo","vgpu":"mock-
up-vmiop"}}}],"extraConfig":[{"_typeName":"OptionValue","key":"pciPassthru0.cfg.enable_uvm","val-
ue":{"_typeName":"string","_value":"1"}},{"_typeName":"OptionValue","key":"pciPassthru1.cfg.enable_u-
vm","value":{"_typeName":"string","_value":"1"}}]}'
```

Install the NVIDIA Guest Driver in a VM
If the VM includes a PCI device configured for vGPU, after you create and boot the VM in your environment, install the NVIDIA vGPU graphics driver to fully enable GPU operations.

- Deploy the VM with vGPU. Make sure that the VM YAML file references the VM class with vGPU definition. See Deploy a Virtual Machine on a Namespace in a Supervisor.

- Verify that you downloaded the vGPU software package from the NVIDIA download site, uncompressed the package, and have the guest drive component ready. For information, see the appropriate NVIDIA Virtual GPU Software documentation.

  Note:

  The version of the driver component must correspond to the version of the vGPU Manager that a vSphere administrator installed on the host.

1. Copy the NVIDIA vGPU software Linux driver package, for example, `NVIDIA-Linux-x86_64-version-grid.run`, to the guest VM.
2. Before attempting to run the driver installer, terminate all applications.
3. Start the NVIDIA vGPU driver installer.

   ```
   sudo ./NVIDIA-Linux-x86_64-version-grid.run
   ```

4. Accept the NVIDIA software license agreement and select **Yes** to update the X configuration settings automatically.
5. Verify that the driver has been installed.

   For example,

```
~$ nvidia-smi
Wed May 19 22:15:04 2021
+-----------------------------------------------------------------------------+
| NVIDIA-SMI 460.63       Driver Version: 460.63       CUDA Version: 11.2     |
|-------------------------------+----------------------+----------------------+
| GPU  Name        Persistence-M| Bus-Id        Disp.A | Volatile Uncorr. ECC |
| Fan  Temp  Perf  Pwr:Usage/Cap|         Memory-Usage | GPU-Util  Compute M. |
|                               |                      |               MIG M. |
|===============================+======================+======================|
|   0  GRID V100-4Q        On   | 00000000:02:00.0 Off |                  N/A|
| N/AN/AP0    N/A/  N/A|    304MiB /  4096MiB |      0%      Default |
|                               |                      |                  N/A|
+-------------------------------+----------------------+----------------------+

+-----------------------------------------------------------------------------+
| Processes:                                                                  |
|  GPU   GI   CI        PID   Type   Process name                  GPU Memory |
|        ID   ID                                                   Usage      |
|=============================================================================|
|  No running processes found                                                 |
+-----------------------------------------------------------------------------+
```

Deploying a VM with PCI Devices in Supervisor Environment

In addition to vGPU, you can configure other PCI devices on an host to make them available to a VM in a passthrough mode.

Supervisor environment supports Dynamic DirectPath I/O devices. Using Dynamic DirectPath I/O, the VM can directly access the physical PCI and PCIe devices connected to a host. You can use Dynamic DirectPath I/O to assign multiple PCI passthrough devices to a VM. Each passthrough device can be specified by its PCI vendor and device identifier.

Note:

When configuring Dynamic DirectPath I/O for PCI passthrough devices, connect the PCI devices to the host and mark them as available for passthrough. See *Enable Passthrough for a Network Device on a Host* in the *vSphere Networking* documentation.

Deploying a VM with Instance Storage in vSphere Supervisor

Along with persistent storage volumes, a VM can use instance storage. Unlike persistent volumes that exist separately from the VM, instance storage volumes depend on the lifecycle of a VM instance. This storage is typically located on high-speed devices, such as NVMe, that are local to the ESX host.

Instance Storage Lifecycle

At the VM creation, the system creates instance storage volumes and attaches them to the VM. Data in the instance storage volume persists only during the lifetime of its associated VM instance. The volume is deleted when the VM is deleted.

VMs with instance storage support ESX host maintenance mode. The VM is turned off when the ESX host enters maintenance mode and is turned on after the host exits maintenance mode.

Instance Storage VM Considerations

Consider the following items when you use VMs with instance storage:

- Supervisor with a VDS networking stack does not support instance storage.
- Three-zone Supervisor does not support instance storage.
- A warning appears if a vSphere administrator applies a VM class with instance storage to a namespace that misses an appropriate storage policy required for the instance storage.
- VMs with instance volumes cannot migrate to other ESX hosts.
- You cannot edit the instance storage volumes when the volumes are already in use.
- If the vSphere administrator removes the instance storage policy from the namespace after the VM creation, the VM continues to run.
- As the DevOps engineer, you cannot delete or update instance storage resources. You cannot detach the instance storage volume from one VM instance and attach it to a different instance.

Workflow for Provisioning and Monitoring an Instance Storage VM

Step	Performed by	Description
1	vSphere administrator	Create content libraries and assign them to the namespace you use for the VM.
2	vSphere administrator	Create a vSAN Direct datastore.
3	vSphere administrator	Create a storage policy compatible with vSAN Direct and assign it to the namespace.
4	vSphere administrator	Create an instance storage VM class and assign it to the namespace.
5	DevOps engineer	Provision a VM with instance storage in the namespace.
6	vSphere administrator	Monitor deployed VMs.

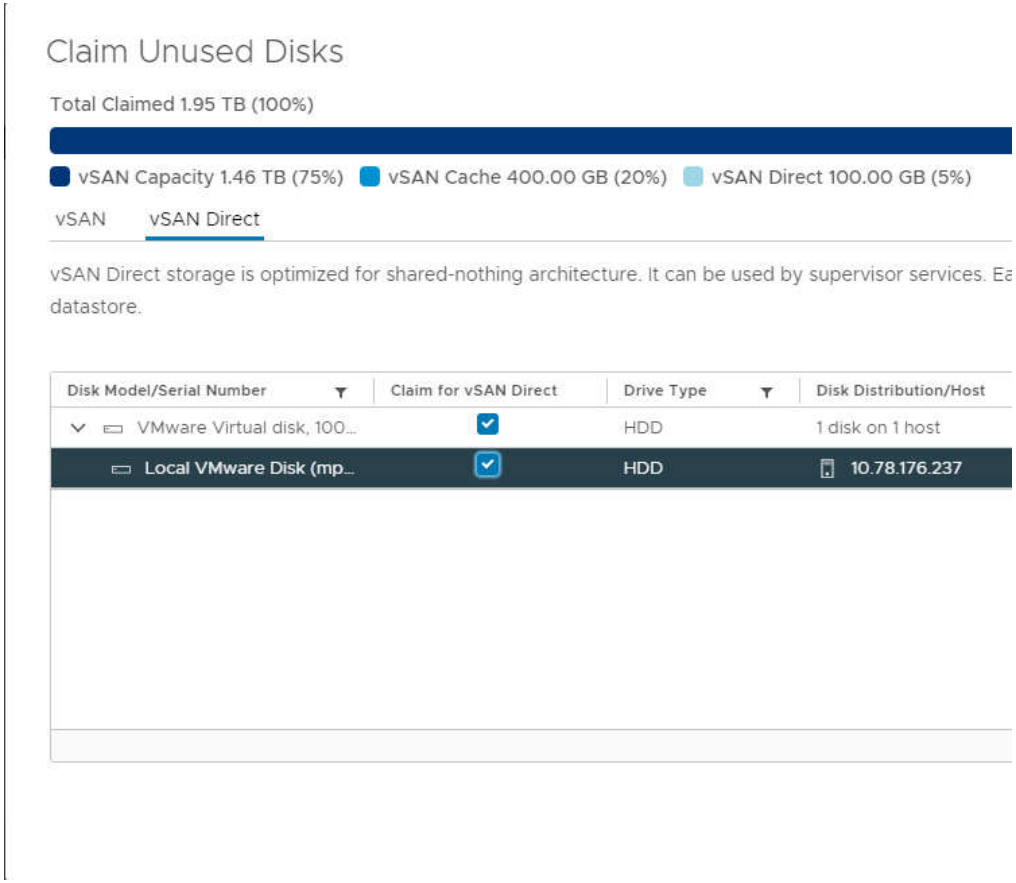
Create a vSAN Direct Datastore

As a vSphere administrator, set up a vSAN Direct datastore to be used with such functionality asvSAN Data Persistence platform or VM instance storage. To create the datastore, use unclaimed storage devices local to your ESX host.

You can create the vSAN Direct datastore when enabling vSAN for your Supervisor. The following task shows how to claim local storage devices as vSAN Direct when vSAN is already enabled on the cluster.

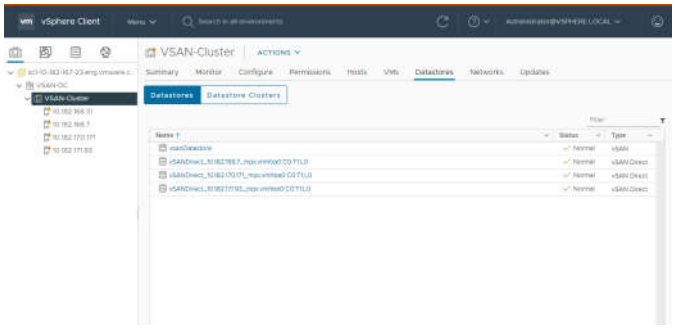
1. In the vSphere Client, navigate to the vSAN cluster.
2. Click the **Configure** tab.
3. Under vSAN, click **Disk Management**.
4. Click **Claim Unused Disks**.
5. On the **Claim Unused Disks** dialog box, click the **vSAN Direct** tab.
6. Select a device to claim and select a check box in the **Claim for vSAN Direct** column.

   Note: If you claimed the devices for a regular vSAN datastore, these devices do not appear in the **vSAN Direct** tab.



7. Click **Create**.

   For each device you claim, vSAN Direct creates a new datastore.
8. Click the **Datastores** tab to display all vSAN Direct datastores in your cluster.



Create vSAN Direct Storage Policy

If you use vSAN Direct, create a storage policy to be used with a Supervisor namespace. On the namespace that you associate with this storage policy, you can run workloads compatible with vSAN Direct, for example, stateful services or instance storage VMs.

1. In the vSphere Client, open the **Create VM Storage Policy** wizard.
   a) In the **Home** menu, click **Policies and Profiles**.
   b) Under **Policies and Profiles**, click **VM Storage Policies**.
   c) Click **Create**.
2. Enter the policy name and description.

Option	Action
vCenter	Select the vCenter instance.
Name	Enter the name of the storage policy.
Description	Enter the description of the storage policy.

3. On the **Policy structure** page under **Datastore specific rules**, enable rules for vSAN Direct storage placement.
4. On the **vSAN Direct rules** page, specify vSAN Direct as a storage placement type.
5. On the **Storage compatibility** page, review the list of vSAN Direct datastores that match this policy.
6. On the **Review and finish** page, review the storage policy settings and click **Finish**.

   To change any settings, click **Back** to go to the relevant page.

Create a VM Class with Instance Storage

In the VM class, you reference the vSAN Direct storage policy and set the size of the volumes to be used for instance storage. After you create the VM class, assign it to the namespace that you plan to use for the instance storage VM.

- Create a storage policy compatible with the vSAN Direct datastore.
- Add the vSAN Direct storage policy to the namespace you use for the instance storage VM. For information, see the Supervisor documentation.
- Required privileges:

- **Namespaces** > **Modify cluster-wide configuration**
- **Namespaces** > **Modify namespace configuration**
- **Virtual Machine Classes** > **Manage Virtual Machine Classes**

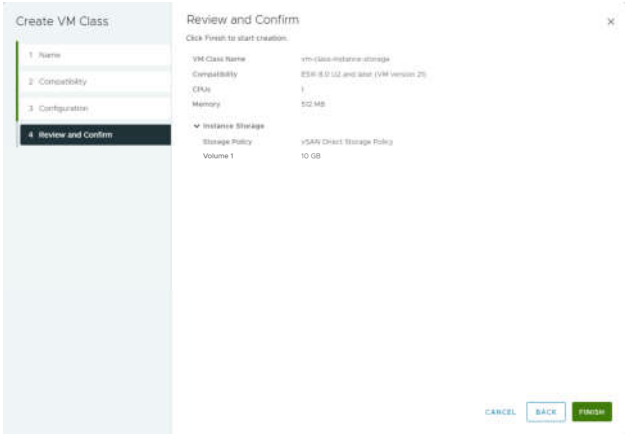
1. Add instance storage when you create or edit a VM class.

Option	Action
Create a VM class	1. From the vSphere Client home menu, select **Supervisor Management**. 2. Click the **Services** tab and click **Manage** on the **VM Service** card. 3. On the **VM Service** page, click **Create VM Class**. 4. Configure the VM Class as needed, see Edit a VM Class Using the vSphere Client for the available options. 5. To add instance storage, on the **Configuration** page, select **Virtual Hardware** then select **Add New Device** > **Instance Storage**. The **Instance Storage** option appears in Virtual Hardware.
Edit an existing VM class	1. From the vSphere Client home menu, select **Supervisor Management**. 2. Click the **Services** tab and click **Manage** on the **VM Service** pane. 3. On the **VM Service** page, click **VM Classes**. 4. In the existing VM class card, click **Manage** and click **Edit**. 5. To add instance storage, select **Virtual Hardware** then select **Add New Device** > **Instance Storage**. The **Instance Storage** option appears in Virtual Hardware.

2. Expand the **Instance Storage** option to edit the instance storage settings.

Option	Action
Storage Policy	Select the vSAN Direct storage policy.
Volume	Specify the size of the volume. You can add multiple storage volumes.

3. On the **Review and Confirm** page, review the details and click **Finish**.



4. Assign the VM class you created to the namespace you use for the instance storage VM.

   See Associate a VM Class with a Namespace Using the vSphere Client.

Deploy a VM with Instance Storage

As a DevOps engineer, verify that you can access VM resources necessary to create an instance storage VM. Use the resources to deploy the VM.

When you deploy the instance storage VM, you follow general VM deployment steps. See Deploying a Stand-Alone VM in vSphere Supervisor. This procedure covers additional specific items that apply to the instance storage VM.

Verify the following items specific to the instance storage VM:

- Your namespace includes the storage class compatible with the vSAN Direct datastore.
- The instance storage VM class references this storage class.

  When reviewing details of the instance storage VM class, make sure that it includes the `instanceStorage` section.

```
kubectl describe virtualmachineclasses vm-class-instance-storage
 apiVersion: vmoperator.vmware.com/v1alpha2
 kind: VirtualMachineClass
 metadata:
   name: vm-class-instance-storage
 spec:
   hardware:
     cpus: 8
     memory: 64Gi
     devices:
...
     instanceStorage:
       storageClass: vsan-direct
       volumes:
       - size: 256Gi
       - size: 512Gi

...
```

- The VM YAML file points to the appropriate instance storage VM class.

Deploy VMs with Configurable OVF Properties in vSphere Supervisor

Typically, when a DevOps engineer provisions a VM in the Supervisor environment, an OVF template includes hard coded details such as basic network configuration. However, you might not know and often cannot assign certain values to the VM's OVF properties, such as such as IPAM provided network data, until after the VM CR is created. With template strings support, you don't need to know network information in advance. You can use Golang based templating to populate the OVF property values and configure the VM's network stack.

1. Make sure that for all properties to be configured, your OVF file includes the `ovf:userConfigurable="true"` entry.

   This entry enables the system to substitute networking value placeholders, such as nameservers and management IP, with real data after the data gets collected.

   Use the following example.

```
<Property ovf:key="hostname" ovf:type="string" ovf:userConfigurable="true" ovf:value="ubuntuguest">
      <Description>Specifies the hostname for the appliance</Description>
</Property>
<Property ovf:key="nameservers" ovf:type="string" ovf:userConfigurable="true" ovf:value="1.1.1.1,
 1.0.0.1">
      <Label>2.2. DNS</Label>
```

```
      <Description>A comma-separated list of IP addresses for up to three DNS servers</Description>
</Property>
<Property ovf:key="management_ip" ovf:type="string" ovf:userConfigurable="true">
      <Label>2.3. Management IP</Label>
      <Description>The static IP address for the appliance on the Management Port Group in CIDR format
 (Eg. ip/subnet mask bits). This cannot be DHCP.</Description>
</Property>
```

2. Create the VM YAML file with template strings.
   The template strings for bootstrap resources will collect the data necessary to populate the OVF property values.
   You can use one of the following methods to construct template strings.

   - Use vm-operator-api.

     For more information, see the following page on GitHub: https://github.com/vmware-tanzu/vm-operator/blob/main/api/v1alpha2/virtualmachinetempl_types.go.

     The following is a sample YAML file:

```
apiVersion: vmoperator.vmware.com/v1alpha2
kind: VirtualMachine
metadata:
  name: template-vm
  namespace: test-ns
  annotations:
    vmoperator.vmware.com/image-supported-check: disable
spec:
  className: best-effort-xsmall
  imageName: vmi-xxxx0000
  powerState: poweredOn
  storageClass: wcpglobal-storage-profile
  vmMetadata:
    configMapName: template-vm-1
    transport: vAppConfig

---
apiVersion: v1
kind: ConfigMap
metadata:
  name: template-vm-1
  namespace: test-ns
data:
  nameservers: "{{ (index .v1alpha2.Net.Nameservers 0) }}"
  hostname: "{{ .v1alpha2.VM.Name }} "
  management_ip: "{{ (index (index .v1alpha2.Net.Devices 0).IPAddresses 0) }}"
  management_gateway: "{{ (index .v1alpha2.Net.Devices 0).Gateway4 }}"
```

   - Use the following functions.

Function name	Signature	Description
V1alpha2_FirstIP	func () string	Get the first non-loopback IP from first NIC.
V1alpha2_FirstIPFromNIC	func (index int) string	Get non-loopback IP address from the ith NIC. If index is out of bound, template string will not be parsed.

Function name	Signature	Description
V1alpha2_FormatIP	func (IP string, netmask string) string	Format an IP address with network length. A netmask can be either the length, for example, /24, or the decimal notation, for example, 255.255.255.0. If input netmask is not valid or different from the default mask, it will not be parsed.
V1alpha2_FormatNameservers	func (count int, delimiter string) string	Format the first occurred count of nameservers with specific delimiter. A negative number for count means all nameservers.
V1alpha2_IP	func(IP string) string	Format a static IP address with default netmask CIDR. If IP is not valid, template string will not be parsed.
V1alpha2_IPsFromNIC	func (index int) []string	List all IPs from the ith NIC. If index is out of bound, template string will not be parsed.

If you use the functions, the YAML file looks like the following:

```
apiVersion: vmoperator.vmware.com/v1alpha2
kind: VirtualMachine
metadata:
  name: template-vm
  namespace: test-ns
spec:
  className: best-effort-xsmall
  imageName: vmi-xxxx0000
  powerState: poweredOn
  storageClass: wcpglobal-storage-profile
  vmMetadata:
    configMapName: template-vm-2
    transport: vAppConfig

---
apiVersion: v1
kind: ConfigMap
metadata:
  name: template-vm-2
  namespace: test-ns
data:
  nameservers: "{{ V1alpha2_FormatNameservers 2 \",\" }}"
  hostname: "{{ .v1alpha2.VM.Name }} "
  management_ip: "{{ V1alpha2_FormatIP \"192.168.1.10\" \"255.255.255.0\" }}"
  management_gateway: "{{ (index .v1alpha2.Net.Devices 0).Gateway4 }}"
```

3. Deploy the VM.

```
kubectl apply -f file_name.yaml
```

If the customization fails and the VM does not get an IP address, inspect the VM using the vSphere VM web console. See Troubleshoot VMs Using the vSphere VM Web Console.

Import VMs into vSphere Supervisor

As a vSphere administrator, you can import an existing VM that runs in your vSphere environment into a namespace on Supervisor and have this VM managed by VM Service.

Required privilege: Namespaces.View, Namespaces.Edit, and VirtualMachine.Supervisor.Transfer.

1. In the vSphere Client, navigate to the VM you want to import.
2. Select **Migrate** from the right-click menu, and select **Change Management Layer**.
3. Review the VM to be migrated, and click **Next**.
4. On the **Select destination** page, select the destination namespace.
5. On the **Configure inputs** page, select appropriate storage policy, network and guest OS inputs.
6. Verify that the destination information is correct and click **Finish** to start the migration.

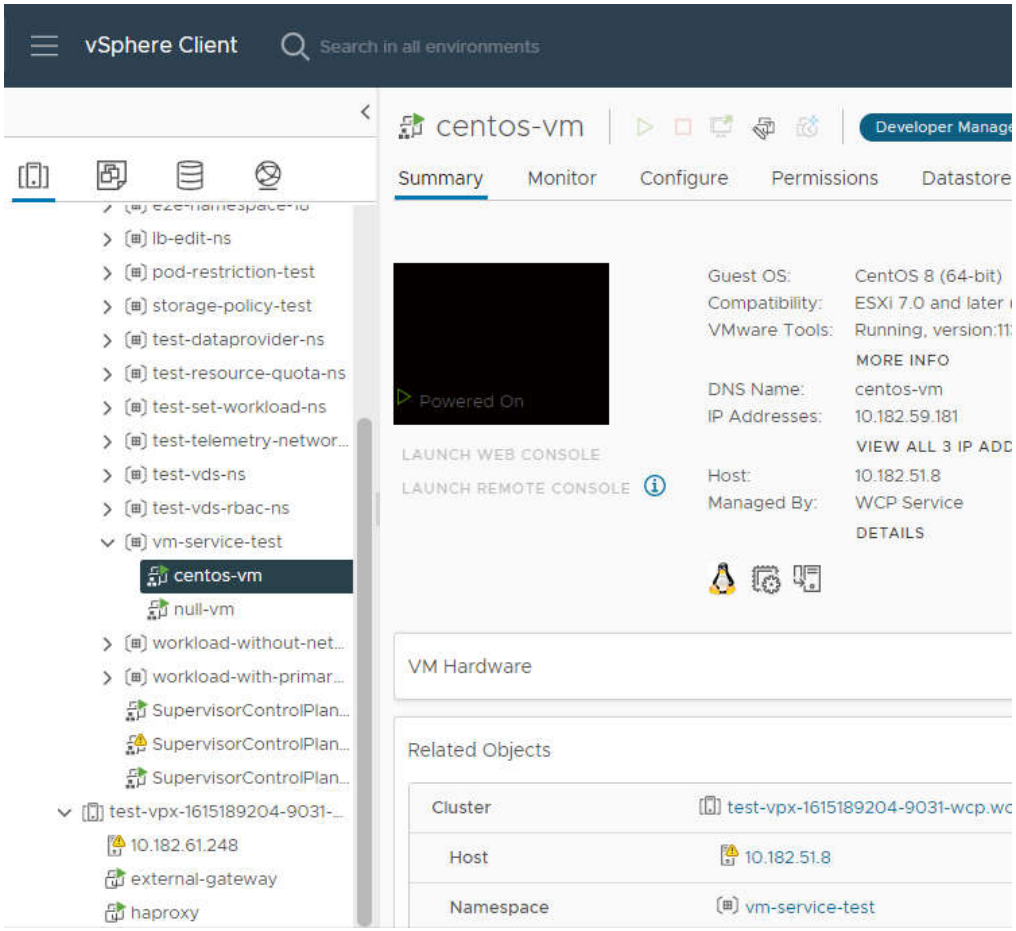
Monitor Virtual Machines Available in vSphere Supervisor

As a vSphere administrator, use the vSphere Client to monitor a VM that was deployed by DevOps in the Supervisor Kubernetes environment.

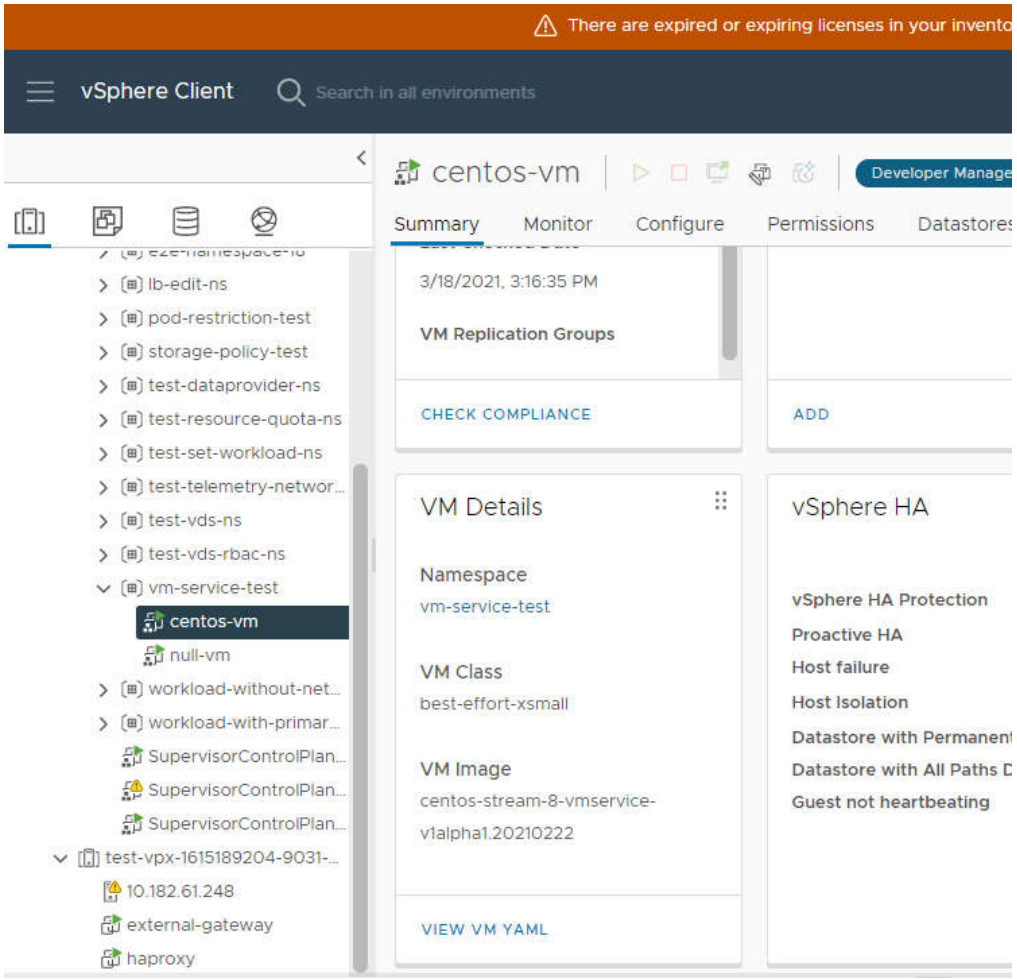
A DevOps engineer has deployed a VM. See Deploying a Stand-Alone VM in the Environment.

You cannot manage the VM lifecycle from the vSphere Client.

1. In the vSphere Client, navigate to the host cluster that has the Supervisor enabled.
2. Under **Namespaces**, expand the namespace where a VM was deployed.
3. Select the VM to view and click the **Summary** tab.
   Make sure that you see the **Developer Managed** tag at the top of the **Summary** page.
   The page displays information about the VM, including its guest OS and IP addresses.



4. Click **Switch to New View** in the upper-right corner of the page to display additional details such as the VM class and VM image, as well as the namespace, where the VM runs.



Troubleshoot VMs Using the vSphere VM Web Console

As a DevOps engineer, you can use the vSphere VM web console to access and troubleshoot problematic VMs. The VM web console can be a useful tool for troubleshooting VMs that are unreachable over the network. For example, when a guest OS fails to configure network settings during its initial boot.

Have edit or owner permissions on the namespace where the problematic VM is deployed. For information, see the Supervisor documentation.

The VM web console becomes especially useful when you deploy VMs with configurable OVF properties. For more information, see Deploy VMs with Configurable OVF Properties in vSphere Supervisor.

1. Access your vSphere Namespace.

   See Connecting to Supervisor and VKS Clusters.

2. Verify the target VM status.

   ```
   kubectl get vm -n namspace-name
   ```

   The output is similar to the following. If the POWERSTATE is poweredOff, the console is not accessible.

   ```
   NAME       POWERSTATE        AGE
   vm-name    poweredOn         175m
   ```

3. Generate the URL to the VM web console.

   ```
   kubectl vsphere vm web-console vm-name -n namspace-name
   ```

   The command returns an authenticated URL to the VM's web console as output.

4. Launch the console.

   Copy the generated URL and paste it into your web browser.

   Note:

   You must use the link within two minutes of generation, otherwise it expires. Once you open the console, the session is managed by WebMKS and remains active for an extended period, even after the initial two-minute URL window closes.

Provision and Manage Kubernetes Clusters

You can use a variety of methods to provision Kubernetes clusters in VCF Automation and vSphere Supervisor.

If you have access to VCF Automation for All Apps, the Iaas Services Console provides a graphical user interface for provisioning Kubernetes clusters.

If the Supervisor Admin has deployed the Local Consumption Interface, the Local Consumption Interface also provides a graphical user interface for provisioning Kubernetes clusters.

In addition, the VCF CLI and `kubectl` provide command-line interfaces for provisioning Kubernetes clusters. If you have the access to VCF Automation for All Apps, use VCF CLI and `kubectl` in the namespace of your organization, provided by your organization administrator.

What to Read Next

- Provision a Kubernetes Cluster Using Self-Service
- Managing vSphere Kubernetes Service Clusters and Workloads

Provision a Kubernetes Cluster Using Self-Service

You can use a variety of methods to provision Kubernetes clusters with the Kubernetes Service in vSphere Supervisor.

- Verify that you have access to a namespace.

If you have access to VCF Automation for All Apps, the Iaas Services Console provides a graphical user interface for provisioning Kubernetes clusters using the Kubernetes Service.

If the Supervisor Admin has deployed the Local Consumption Interface, the Local Consumption Interface also provides a graphical user interface for provisioning Kubernetes clusters.

1. Log in to the interface you are using by following the steps in Access the IaaS Services Console or Local Consumption Interface.
2. On the **Kubernetes Service** card, click **Create**.

   Alternatively, click **Go to Service**, then, on the Kubernetes Clusters tab, click **Create**.
3. Select a cluster type.

Option	Description
ClusterClass API	Define and reuse configuration templates for clusters.
TanzuKubernetesCluster API	Configure and manage Kubernetes clusters in a declarative fashion.

4. Select a configuration type.
5. On the General Settings pane, define storage and networking for the cluster.
   a) Enter a new name for the cluster.
   b) Select a cluster class.
   c) Select a Tanzu Kubernetes release version.
   d) Select a Container Network Interface (CNI).

      The CNI provides networking for pods, services, and ingress within the cluster.

      - Antrea uses Open vSwitch.
      - Calico uses the Linux bridge with BGP.

   e) Enter CIDR for the pods.
   f) Enter services CIDR.
   g) Enter service domain.
   h) Optional: Select a storage class.
   i) Click **Next**.
6. Define the topology of the cluster controller.
   a) Select a number of replica nodes.
   b) Select a VM class.
   c) Select a storage class.
   d) Select TKR OSimage format.

      - Photon
      - Ubuntu

   e) Optional: Add volumes.
   f) Click **Next**.
7. Add nodepools.
   a) Click **Add nodepool**.
   b) Configure the nodepool.
   c) Configure volumes.
   d) On the **Review and Confirm** pane, click **Finish**.
8. Click **Next**.
9. Review your configuration and click **Finish**.

   Consider copying the Kubernetes resource YAML manifest to clipboard or downloading it as a .zip file. You can reuse the manifest to provision clusters with the same or modified configuration from the command line. You can also store the manifest in a source control repository like GitHub.

The provisioning process begins and the Kubernetes Cluster tab on the Kubernetes Cluster Service page opens with your current request at the top.

Review your provisioned cluster.

1. Click the cluster name to open the Details page, where you can perform day 2 operations on the cluster like..
2. Click the double arrows next to the cluster name to expand the details view.
3. Click View YAML to view the active configuration of the Kubernetes object. You can edit certain fields in the manifest directly.
4. Click the vertical ellipsis next to the cluster name to view your options, such as deleting the cluster.

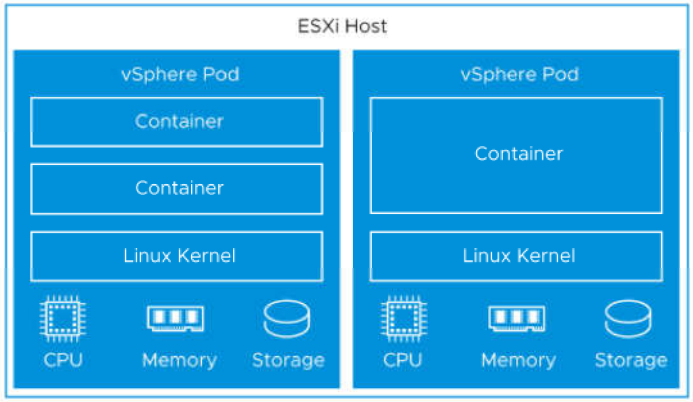
Deploying Workloads to vSphere Pods

As a DevOps engineer, you can deploy and manage the life cycle of vSphere Pods within the resources boundaries of a vSphere Namespace that is running on a Supervisor.

What Is a vSphere Pod?

A vSphere Pod is equivalent of a Kubernetes pod. It is a VM with a small footprint that runs one or more Linux containers. Each vSphere Pod is sized precisely for the workload that it accommodates and has explicit resource reservations for that workload. It allocates the exact amount of storage, memory, and CPU resources required for the workload to run.

Figure 252: vSphere Pods



vSphere Pods are objects enable the following capabilities for workloads:

- Strong isolation. A vSphere Pod is isolated in the same manner as a virtual machine. Each vSphere Pod has its own unique Linux kernel that is based on the kernel used in Photon OS. Rather than many containers sharing a kernel, as in a bare metal configuration, in a vSphere Pod, each container has a unique Linux kernel
- Resource Management. vSphere DRS handles the placement of vSphere Pods on the Supervisor.
- High performance. vSphere Pods get the same level of resource isolation as VMs, eliminating noisy neighbor problems while maintaining the fast start-up time and low overhead of containers.
- Diagnostics. As a vSphere administrator you can use all the monitoring and introspection tools that are available with vSphere on workloads.

vSphere Pods are Open Container Initiative (OCI) compatible and can run containers from any operating system as long as these containers are also OCI compatible.

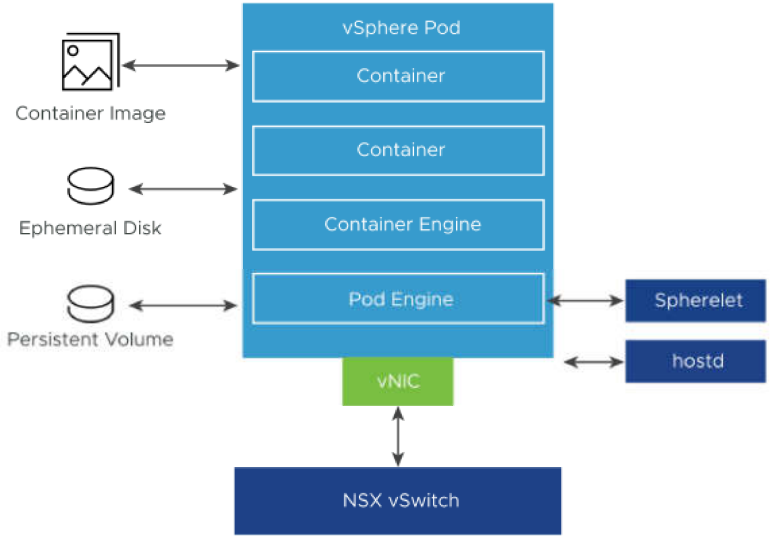
Guidelines for Deploying vSphere Pods

Before deploying vSphere Pods, make sure that your environment meets the following requirements.

Namespace	For information about creating a namespace, see Create and Configure a vSphere Namespace on the Supervisor. For information about assigning permissions, see Identity and Access Management.

Networking	For networking, vSphere Pods use the topology provided by NSX. For details, see Supervisor Networking. Spherelet is an additional process that is created on each host. It is a kubelet that is ported natively to ESX and allows the ESX host to become part of the Kubernetes cluster.
Storage	vSphere Pods use three types of storage depending on the objects that are stored, that are ephemeral VMDKs, persistent volume VMDKs, and containers image VMDKs. As a vSphere administrator, you configure storage policies for placement of container image cache and ephemeral VMDKs when you enable the Supervisor . On a vSphere Namespace level, you configure storage policies for placement of persistent volumes. See Persistent Storage for Workloads for details about the storage requirements and concepts.

vSphere Pod Networking and Storage



Display Storage Classes in a vSphere Namespace

After a vSphere administrator creates a storage policy and assigns it to the vSphere Namespace, the storage policy appears as a matching Kubernetes storage class in the vSphere Namespace. It is also replicated to any available VKS cluster. As a DevOps engineer, you can verify that the storage class is available.

Make sure that your vSphere administrator has created an appropriate storage policy and assigned the policy to the vSphere Namespace.

Your ability to run the commands depends on your permissions.

Use one of the following commands to verify that the storage classes are available.

- `kubectl get storageclass`

  Note: This command is available only to a user with administrator privileges.

  You get the output similar to the following. The name of the storage class matches the name of the storage policy on the vSphere side.

  On a three-zone Supervisor, you can see two storage classes for each storage policy assigned to a namespace, one for `Immediate` and another for `WaitForFirstConsumer` volume binding modes.

```
NAME     PROVISIONER              AGE
silver   csi.vsphere.vmware.com   2d
gold     csi.vsphere.vmware.com   1d
```

- Use `kubectl get storagepolicyquota -namespace namespace -o yaml`, to see the name of the storage class and view other details.

  You get the output similar to the following.

```
---
apiVersion: v1
items:
  - apiVersion: cns.vmware.com/v1alpha1
    kind: StoragePolicyQuota
    metadata:
      creationTimestamp: 2024-01-02T21:48:55Z
      generation: 2
      name: wcp-profile-f4v2dlw23p-storagepolicyquota
      namespace: storage-policy-test
      resourceVersion: "1878935"
      uid: d32aae59-911f-4215-a2b8-47ab3640e36e
    spec:
      limit: 100Gi
      storagePolicyId: 09126b80-e0e9-4224-9fb6-c9bb2904414f
    status:
      extensions:
        - extensionName: volume.cns.vsphere.vmware.com
          extensionQuotaUsage:
            - scQuotaUsage:
                reserved: "0"
                used: "0"
              storageClassName: silver
        - extensionName: volume.cns.vsphere.vmware.com
          extensionQuotaUsage:
            - scQuotaUsage:
                reserved: "0"
                used: "0"
              storageClassName: gold
      total:
        - scQuotaUsage:
            reserved: "0"
            used: "0"
          storageClassName: silver
        - scQuotaUsage:
            reserved: "0"
```



```
              used: "0"
            storageClassName: gold
kind: List
metadata:
  resourceVersion: ""
```

The names of the storage classes appear under `Status`.

You can also see such details as storage policy Id and storage resource limits under `spec`:

```
limit: 100Gi
storagePolicyId: 09126b80-e0e9-4224-9fb6-c9bb2904414f
```

Deploy an Application to a vSphere Pod on a vSphere Namespace

You can deploy an application on a vSphere Namespace in a Supervisor. Once the application is deployed, the respective number of vSphere Pods are created on the Supervisor within the namespace.

- Get the IP address of the Kubernetes control plane on the from your vSphere administrator.
- Get your user account in vCenter Single Sign-On.
- Verify with your vSphere administrator that you have permissions to access the contexts that you need.

1. Access your namespace in the Supervisor environment.

   See Connecting to Supervisor and VKS Clusters.

2. Deploy the application.

```
kubectl apply -f <application name>.yaml
```

Scale a Container by using VCF CLI

You can scale up and down the number of replicas for each application that is deployed using containers.

- Get the IP address of the Kubernetes control plane on the from your vSphere administrator.
- Get your user account in vCenter Single Sign-On.
- Verify with your vSphere administrator that you have permissions to access the contexts that you need.

1. Access your namespace in the Supervisor environment.

   See Connecting to Supervisor and VKS Clusters.

2. Scale up or scale down an application.

```
kubectl get deployments
kubectl scale deployment <deployment-name> --replicas=<number-of-replicas>
```

Deploy a Confidential vSphere Pod

You can run confidential vSphere Pods on a Supervisor. A confidential vSphere Pod uses a hardware technology that keeps the guest OS memory encrypted, protecting it against access from the hypervisor.

To enable SEV-ES on an ESX host, a vSphere administrator must follow these guidelines:

- Use the hosts that support the SEV-ES functionality.
- Use the ESX version of 7.0 Update 2 or later.
- Enable SEV-ES in an ESX system's BIOS configuration. See your system's documentation for more information about accessing the BIOS configuration.

- When enabling SEV-ES in the BIOS, enter a value for the **Minimum SEV non-ES ASID** setting equal the number of SEV-ES VMs and confidential vSphere Pods on the host plus one. For example, if you plan to run 100 SEV-ES VMs and 128 vSphere Pods, enter at least 229. You can enter a setting as high as 500.

You can create confidential vSphere Pods by adding Secure Encrypted Virtualization-Encrypted State (SEV-ES) as an extra security enhancement. SEV-ES prevents CPU registers from leaking information in registers to components like the hypervisor. SEV-ES can also detect malicious modifications to a CPU register state. For more information about using SEV-ES technology in the vSphere environment, see *Securing Virtual Machines with AMD Secure Encrypted Virtualization-Encrypted State* in the *vSphere Security* documentation.

1. Create a YAML file that contains the following parameters.
   a) In annotations, enable the confidential vSphere Pods feature.

```
...
annotations:
       vmware/confidential-pod: enabled
...
```

   b) Specify memory resources for containers.

   Make sure to set memory requests and memory limits to the same value, as in this example.

```
resources:
     requests:
       memory: "512Mi"
     limits:
       memory: "512Mi"
```

   Use the following YAML file as an example:

```
apiVersion: v1
kind: Pod
metadata:
 name: photon-pod
 namespace: my-podvm-ns
 annotations:
   vmware/confidential-pod: enabled
spec:  # specification of the pod's contents
 restartPolicy: Never
 containers:
 - name: photon
   image: wcp-docker-ci.artifactory.eng.vmware.com/vmware/photon:1.0
   command: ["/bin/sh"]
   args:    ["-c", "while true; do echo hello, world!; sleep 1; done"]
   resources:
     requests:
       memory: "512Mi"
     limits:
       memory: "512Mi"
```

2. Access your namespace in the Supervisor environment.

   See Connecting to Supervisor and VKS Clusters.

3. Deploy a confidential vSphere Pod from the YAML file.

```
kubectl apply -f <yaml file name>.yaml
```

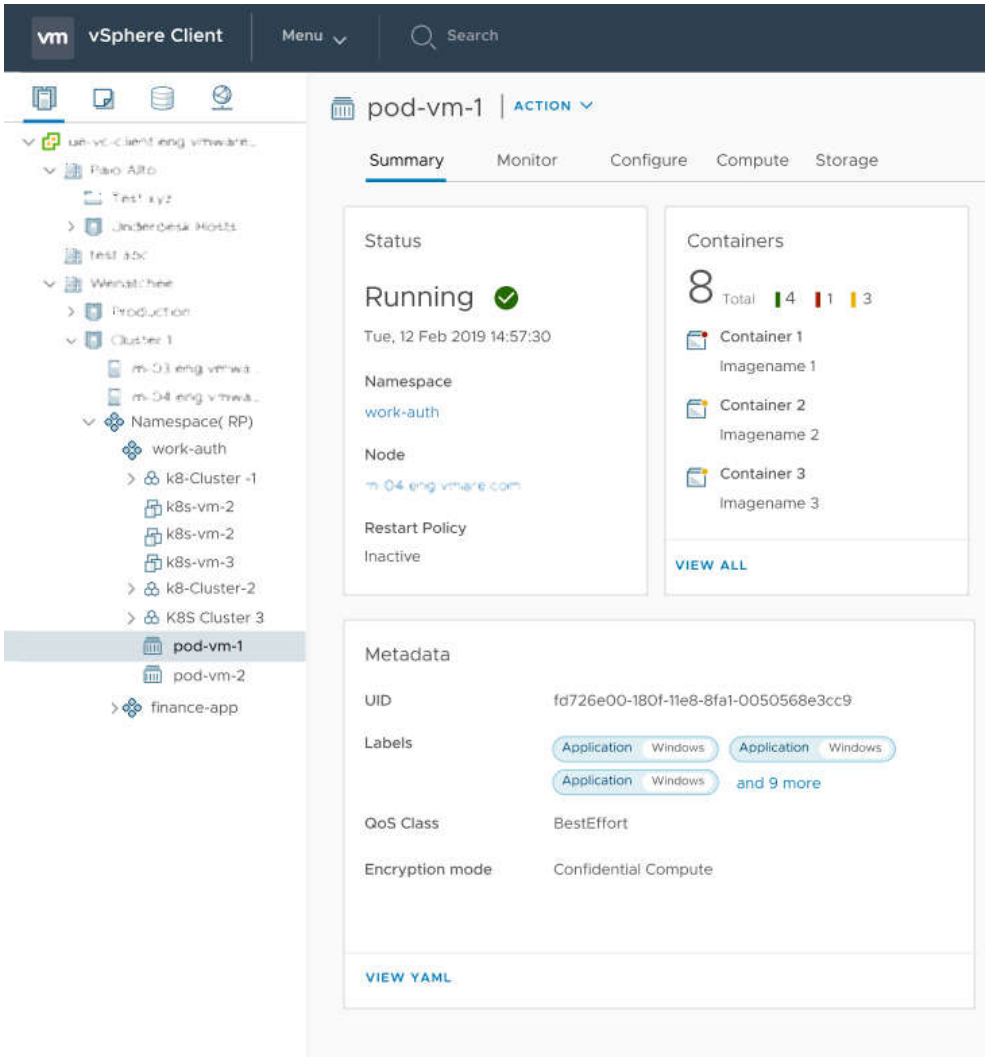
   Note: When the vSphere Pod is deployed, DRS places it to the ESX node that supports SEV-ES. If no such node is available, the vSphere Pod is marked as failed.

   The confidential vSphere Pod that is launched provides hardware memory encryption support for all workloads that are running on that pod.

4. Run the following command to verify that the confidential vSphere Pod has been created.

```
kubectl describe pod/<yaml name>
```

A vSphere administrator can view the confidential vSphere Pod. In the vSphere Client, it appears with the **Encryption Mode: Confidential Compute** tag.



vSphere Pod Workload Deployment in vSphere Supervisor

This example tutorial describes how to deploy the WordPress application using vSphere Pods.

The WordPress deployment includes containers for the WordPress frontend and the MySQL backend, and services for both. Secret objects are also required.

This tutorial uses a Deployment object. In production environment, you typically use StatefulSets for both WordPress and MySQL containers.

Prerequisites

- Activate a Supervisor.
- Create a vSphere Namespace for deploying vSphere Pods. See Create and Configure a vSphere Namespace on the Supervisor.
- Create a storage policy, for example, `vwt-storage-policy` and assign it to the namespace.
- Download the VCF Consumption CLI tools. See Download and Install the VCF CLI.
- Create YAML files required for this tutorial and verify command line access to the files.

Category	Files
Storage	• mysql-pvc.yaml • wordpress-pvc.yaml **Note:** Make sure that the files reference correct storage class.
Secrets	• regcred.yaml • mysql-pass.yaml
Services	• mysql-service.yaml • wordpress-service.yaml
Deployments	• mysql-deployment-vsphere-pod.yaml • wordpress-deployment.yaml

Deploy WordPress

Use this workflow to deploy the WordPress application using vSphere Pods.

Part 1. Access Your Namespace

Use these steps to access your namespace.

1. Log in to the Supervisor and switch to the your namespace.
   See Connecting to Supervisor and VKS Clusters.
2. Verify that the storage policy you created, `vwt-storage-policy`, is available in the namespace as a storage class.
   See Display Storage Classes in a vSphere Namespace.

Part 2. Create WordPress PVCs

Use these commands to create WordPress PVCs.

1. Create the MySQL PVC.
   ```
   kubectl apply -f mysql-pvc.yaml
   ```
2. Create the WordPress PVC.
   ```
   kubectl apply -f wordpress-pvc.yaml
   ```
3. Verify the PVCs.
   ```
   kubectl get pvc,pv
   ```

Part 3. Create Secrets

The public Docker Hub is the default container registry for Kubernetes. Docker Hub now limits image pulls. You need to have a paid account and add the account key to the secret YAML in the `data.dockerconfigjson` field.

1. Create Docker Hub registry secret.
   ```
   kubectl apply -f regcred.yaml
   ```
2. Create the mysql password secret.
   The MySQL DB password is required. Within the secret the password needs to be base64-encoded.
   ```
   kubectl apply -f mysql-pass.yaml
   ```
3. Verify secrets.
   ```
   kubectl get secrets
   ```

Part 4. Create Services

Follow these steps to create services.

1. Create the MySQL service.
   ```
   kubectl apply -f mysql-service.yaml
   ```
2. Create the WordPress service.
   ```
   kubectl apply -f wordpress-service.yaml
   ```
3. Verify services.
   ```
   kubectl get services
   ```

Part 5. Create Pod Deployments

Use this task to create pod deployments.

This tutorial uses Deployment objects. In production environment, you should use StatefulSets for both WordPress and MySQL containers.

1. Create the MySQL deployment.
   ```
   kubectl apply -f mysql-deployment-vsphere-pod.yaml
   ```
   Note: When a vSphere Pod is created, the system creates a VM for the containers in the pod. By default, the VM has a 512 MB RAM limit. The MySQL container requires more memory. The pod deployment spec `mysql-deployment-vsphere-pod.yaml` includes a section that increases the memory given to the vSphere Pod VM. If you do not include this section, the pod deployment fails with an out of memory (OOM) exception. You don't need to increase the RAM when deploying a MySQL pod to a TKG cluster.
2. Create the WordPress deployment.
   ```
   kubectl apply -f wordpress-deployment.yaml
   ```
3. Verify your deployment.
   ```
   kubectl get deployments
   ```

Part 6. Test WordPress

Follow these steps to test your WordPress deployment.

1. Verify all objects are created and running.
   ```
   kubectl get pv,pvc,secrets,rolebinding,services,deployments,pods
   ```
2. Get the EXTERNAL-IP address from the WordPress service.
   ```
   kubectl get service wordpress

   NAME        TYPE           CLUSTER-IP    EXTERNAL-IP     PORT(S)         AGE
   wordpress   LoadBalancer   10.96.9.180   10.197.154.73   80:30941/TCP   87s
   ```
3. Browse to the EXTERNAL-IP address.
4. Configure the WordPress instance.
   Username: administrator
   Password: use the provided strong password

Example YAML Files for the WordPress Deployment

Use these example YAML files when you deploy the WordPress application with vSphere Pods.

mysql-pvc.yaml

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: mysql-pvc
  labels:
    app: wordpress
spec:
  accessModes:
    - ReadWriteOnce
  storageClassName: vwt-storage-policy
  resources:
    requests:
      storage: 20Gi
```

wordpress-pvc.yaml

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: wordpress-pvc
  labels:
    app: wordpress
spec:
  accessModes:
    - ReadWriteOnce
  storageClassName: vwt-storage-policy
  resources:
    requests:
      storage: 20Gi
```

regcred.yaml

```
apiVersion: v1
kind: Secret
metadata:
  name: regcred
data:
  .dockerconfigjson: ewoJImF1dGhzIjog....zZGlKcE5WUmtXRUozWpc
type: kubernetes.io/dockerconfigjson
```

mysql-pass.yaml

```
apiVersion: v1
data:
  password: YWRtaW4=  #admin base64 encoded
kind: Secret
metadata:
  name: mysql-pass
```

mysql-service.yaml

```
apiVersion: v1
kind: Service
metadata:
  name: wordpress-mysql
  labels:
    app: wordpress
spec:
  ports:
    - port: 3306
  selector:
    app: wordpress
    tier: mysql
  clusterIP: None
```

wordpress-service.yaml

```
apiVersion: v1
kind: Service
metadata:
  name: wordpress
  labels:
    app: wordpress
spec:
  ports:
    - port: 80
  selector:
    app: wordpress
    tier: frontend
  type: LoadBalancer
```

mysql-deployment-vsphere-pod.yaml

```
apiVersion: apps/v1
```



```
kind: Deployment
metadata:
  name: wordpress-mysql
  labels:
    app: wordpress
spec:
  replicas: 1
  strategy:
    type: Recreate
  selector:
    matchLabels:
      app: wordpress
      tier: mysql
  template:
    metadata:
      labels:
        app: wordpress
        tier: mysql
    spec:
      containers:
      - image: mysql:5.6
        name: mysql
        #increased resource limits required for this pod vm
        #default pod VM RAM is 512MB; MySQL container needs more
        #without extra RAM OOM error prevents deployment
        #extra RAM not required for Kuberentes cluster
        resources:
          limits:
            memory: 1024Mi
            cpu: 1
        env:
        - name: MYSQL_ROOT_PASSWORD
          valueFrom:
            secretKeyRef:
              name: mysql-pass
              key: password
        ports:
        - containerPort: 3306
          name: mysql
        volumeMounts:
        - name: mysql-persistent-storage
          mountPath: /var/lib/mysql
      volumes:
      - name: mysql-persistent-storage
        persistentVolumeClaim:
          claimName: mysql-pvc
      imagePullSecrets:
      - name: regcred
```

wordpress-deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
```

```
metadata:
  name: wordpress
  labels:
    app: wordpress
spec:
  selector:
    matchLabels:
      app: wordpress
      tier: frontend
  strategy:
    type: Recreate
  template:
    metadata:
      labels:
        app: wordpress
        tier: frontend
    spec:
      containers:
      - image: wordpress:4.8-apache
        name: wordpress
        env:
        - name: WORDPRESS_DB_HOST
          value: wordpress-mysql
        - name: WORDPRESS_DB_PASSWORD
          valueFrom:
            secretKeyRef:
              name: mysql-pass
              key: password
        ports:
        - containerPort: 80
          name: wordpress
        volumeMounts:
        - name: wordpress-persistent-storage
          mountPath: /var/www/html
      volumes:
      - name: wordpress-persistent-storage
        persistentVolumeClaim:
          claimName: wordpress-pvc
      imagePullSecrets:
      - name: regcred
```